

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 11.09.2026, 20:51:46
Файлов исходного кода: 87
Суммарный объём кода: 1.21 МВ
Глав/билетов в пособии: 30

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts

25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts

27. src/data/tickets/ticket14_20.ts

28. src/data/tickets/ticket21_24.ts

29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx

31. src/components/BellmanFordViz.tsx

32. src/components/BfsViz.tsx

33. src/components/ChapterImage.tsx

34. src/components/ChapterNav.tsx

35. src/components/ChapterView.tsx

36. src/components/DfsBridgesSimulator.tsx

37. src/components/DijkstraViz.tsx

38. src/components/DPVisualizer.tsx

39. src/components/EulerSimulator.tsx

40. src/components/ExpressQuiz.tsx

41. src/components/FloydViz.tsx

42. src/components/GraphTraversalViz.tsx

43. src/components/HeapViz.tsx

44. src/components/hints/demoKit.tsx

45. src/components/hints/demosExtra.tsx

46. src/components/hints/demosGraph.tsx

47. src/components/hints/demosStruct.tsx

48. src/components/hints/MiniDemo.tsx

49. src/components/hints/SimulatorModal.tsx

50. src/components/hints/TermPopover.tsx

51. src/components/JohnsonViz.tsx

52. src/components/KosarajuViz.tsx

53. src/components/KruskalSimulator.tsx

54. src/components/MnemonicCards.tsx

55. src/components/MobileToc.tsx

56. src/components/Navbar.tsx

57. src/components/PlanarityDemo.tsx

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
 push:
```

```
 branches:
```

```
- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
```

```
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
+       name: github-pages
+       url: ${ steps.deployment.outputs.page_url }
+     steps:
+       - name: Deploy
+         id: deployment
+         uses: actions/deploy-pages@v5
+     ,,,
+

## `github/workflows/rebuild-pdf.yml`

### Полное содержимое после изменений

```yaml
name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:
#
исходный код → full_code_all_pages.txt (script)
|
→ page-0001.png ... page-NNNN.png (
|
→ full_code_all_page
#
Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.

on:
 push:
 branches:
 - "*"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бескон
```

```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
На Ubuntu политика ImageMagick иногда запре
разрешаем то, что нужно пайплайну (text ->
for policy in /etc/ImageMagick-6/policy.xml /
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
done
convert -version
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.densi  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`\${public}/export/full\_code\_all\_pages  
 echo "| \`\${public}/export/full\_code\_all\_pages

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB\_STEP\_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```

 <body class="bg-slate-950 text-slate-100 min-h-screen
....
```

```
`public/favicon.svg`
```

```
Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
</svg>  
....
```

```
### Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
....
```

```
`src/App.tsx`
```

```
Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from  
import { chapters } from "../data/content";
```



```

    }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
      {activeTab === "guide" ? (
        <>
          <MobileToc
            selectedChapterId={activeChapterId}
            setSelectedChapterId={goTo}
            currentTitle={activeChapter.title}
            index={index}
            total={chapters.length}
          />

          <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
            {/* Сайдбар только на десктопе – на мобильном скрывается */}
            <div className="hidden lg:block lg:w-80 shrink-0">
              <Sidebar selectedChapterId={activeChapterId}>
            </div>

            <main id="main-content" className="flex-1 min-w-0">
              {/* Шапка главы с быстрыми переходами */}
              <div className="flex items-center justify-between">
                <div className="min-w-0">
                  <div className="text-[10px] uppercase">
                    {activeChapter.category || "Универсальное пособие"}
                  </div>
                  <h2 className="text-base sm:text-xl font-medium">
                    {activeChapter.title}
                  </h2>
                </div>
                <ChapterNav prev={prev} next={next} index={index}>
              </div>
            </main>
          </div>
        </>
      ) : (
        <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
          <div className="hidden lg:block lg:w-80 shrink-0">
            <Sidebar selectedChapterId={activeChapterId}>
          </div>

          <main id="main-content" className="flex-1 min-w-0">
            {/* Шапка главы с быстрыми переходами */}
            <div className="flex items-center justify-between">
              <div className="min-w-0">
                <div className="text-[10px] uppercase">
                  {activeChapter.category || "Универсальное пособие"}
                </div>
                <h2 className="text-base sm:text-xl font-medium">
                  {activeChapter.title}
                </h2>
              </div>
              <ChapterNav prev={prev} next={next} index={index}>
            </div>
          </main>
        </div>
      )}
    </Navbar>
  </div>
);

```

```

        </footer>
    </div>
  );
}
....

### Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
- const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
 const [activeChapterId, setActiveChapterId] = useState<string>("");
 const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
 </main>
 </div>
 </>
-) : (
+) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
 <SimulatorHub
 onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
 </>
 </main>
) : (
 <main>
 <ChapterNav />
 <ChapterView />
 </main>
)
}

```

```

<div className="flex items-center gap-3 w-full" >
 <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1" >
 <h1 className="text-xl font-extrabold text-slate-900" >
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium" >
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2
 text-slate-900 font-medium rounded-md shadow-sm transition-colors
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2
 text-slate-900 font-medium rounded-md shadow-sm transition-colors
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Симулятор
 </button>
</div>

```

```


 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
 </header>
);
};
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

```

Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.asm.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в интерпретатор
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
 }
 return runtimePromise;
}
```

```

 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.end`);
 setRuntimeMessage("Не удалось выполнить код – про
 } finally {
 setRunning(false);
 }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>
 </div>
 </div>
 </main>

```

```

 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] res
 -none focus:ring-2 focus:ring-inset focus:r
 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 b
 <label className="block px-4 pt-3 text-[1
 Ввод для input() • по одной строке на к
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.tar
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-tra
 eholder:text-slate-700"
 />
 </div>
</section>

<section className="rounded-2xl border border
 <div className="flex items-center justify-b
 <div className="flex items-center gap-2 t
 <Terminal className="w-4 h-4 text-emera
 </div>
 <span className={`inline-flex items-cente
 {runtimeReady && <CheckCircle2 classNam
 {runtimeReady ? "готов" : "не загружен"

 </div>
 <pre className="min-h-[360px] max-h-[560px]
 x] leading-6 text-slate-300">
 {output}

```

Универсальное пособие • полный исходный код

```

+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`
+
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) => void }) => void
+ setStderr: (options: { batched: (message: string) => void }) => void
+ setStdin: (options: { stdin: () => string | number | null }) => void
+};
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {
+ (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL })
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
```



```

+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:p
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:i
+ <div>
+ <div className="inline-flex items-center g
+ <Terminal className="w-4 h-4" /> Боковая
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-e
+ <p className="text-slate-400 mt-2 max-w-2x
+ Пишите небольшой код и запускайте его пр
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5
+ >
+ Как это работает <ExternalLink className="
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1.
+ <section className="rounded-2xl border borde
+ <div className="flex flex-wrap items-cente
+ <div className="flex items-center gap-2"
+ <span className="w-2.5 h-2.5 rounded-f
+ <span className="text-sm font-bold tex
+ <span className="text-[11px] text-slate

```

```

+
+ <div className="border-t border-slate-800 p-4" style={{border: 1px solid #444; border-radius: 10px; margin: 10px 0;}}>
+ <label className="block px-4 pt-3 text-slate-600 font-medium">Ввод для input() • по одной строке на строку</label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-transparent border border-slate-800 rounded-md p-2"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-800 p-4" style={{border: 1px solid #444; border-radius: 20px; margin: 10px 0;}}>
+ <div className="flex items-center justify-between" style={{border: 1px solid #444; border-radius: 10px; padding: 5px; margin-bottom: 10px;}}>
+ <div className="flex items-center gap-2" style={{border: 1px solid #444; border-radius: 10px; padding: 5px; flex: 1;}}>
+ <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+ </div>
+
+ {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
+ {runtimeReady ? "готов" : "не загружен"}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" style={{border: 1px solid #444; border-radius: 10px; padding: 10px; margin-top: 10px;}}>
+ {output}
+ </pre>
+ <div className="border-t border-slate-800 p-4" style={{border: 1px solid #444; border-radius: 10px; margin-top: 10px;}}>
+ </div>
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" style={{border: 1px solid #444; border-radius: 10px; padding: 10px; margin-top: 10px;}}>
+ <div className="flex gap-2 rounded-xl border border-slate-800 p-4" style={{border: 1px solid #444; border-radius: 10px; margin-bottom: 10px;}}>

```

```

plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
},
server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
},
preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
},
});

```

### Patch относительно `main`

```

````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {

```

Универсальное пособие • полный исходный код

-
2. ****Структура JSON/TS:**** Каждая новая малая страница должна иметь объект `{ "content": ... }` для вставки в ``src/data/content.ts``.
 3. ****Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
 - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``)
 - Обязательная сетка аналогий (2 колонки: неформальный текст и код)
 - "Душные" детали (код, формулы, сложные доказательства)
 4. ****Не ломай верстку:**** Не придумывай свои CSS-классы. Используй только классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.

ФАЙЛ 4/87: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискретке
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/87: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "version": "1.0.0",
  "description": "A guide to the ExamPrep Reader, a tool for reading and understanding the ExamPrep Reader.",
  "keywords": "ExamPrep Reader, ExamPrep, ExamPrep Reader, ExamPrep Reader",
  "author": "Remix Remix: ExamPrep Reader",
  "license": "MIT",
  "repository": "https://github.com/remix-remix/exam-prep-reader",
  "scripts": {
    "dev": "vite dev",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.14.2",
    "vite": "^4.3.9"
  },
  "devDependencies": {
    "@types/react": "^18.2.14",
    "@types/react-dom": "^18.2.6",
    "@types/react-router-dom": "^6.14.2",
    "typescript": "^5.0.4"
  }
}
```

```
"devDependencies": {
  "@tailwindcss/vite": "4.1.17",
  "@types/node": "22.19.17",
  "@types/pdfkit": "^0.17.6",
  "@types/react": "19.2.7",
  "@types/react-dom": "19.2.3",
  "@vitejs/plugin-react": "5.1.1",
  "esbuild": "^0.28.2",
  "tailwindcss": "4.1.17",
  "typescript": "5.9.3",
  "vite": "7.3.2",
  "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 КБ

algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с подсветкой кода и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash
npm install
npm run dev # предпросмотр на http://0.0.0.0:5174
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- * ****Содержание**** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- * ****Переходы между темами**** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- * ****Всплывающие подсказки по терминам**** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симуля
- Словарь – `src/data/glossary.ts`, мини-анимации – `src
- * ****Реестр интерактивов**** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

Структура

...

| | |
|--------------------|----------------------------------|
| src/ | |
| App.tsx | – оболочка, привязка глав к визу |
| components/ | – интерактивные симуляторы (Дейк |
| data/ | – контент пособия (главы/билеты |
| scripts/export/ | – пайплайн код → txt → png → pdf |
| public/export/ | – сгенерированные TXT и PDF (обн |
| .github/workflows/ | – автоматизация |
| ... | |

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен
азуемые теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.

Универсальное пособие • полный исходный код

```
-----
> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |
|--------------------|-----------|----------------------|
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет | Тема | Статус |
|---------------|--------------------------------|---|
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. Контент в холостую. |
| **7** | Планарность и раскраска графов | Номерной главы нет; есть блок аналогии. |
| **11** | Мосты в графах | Номерной главы нет; есть блок аналогии. |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; есть блок аналогии. |

Дополнительно – «осиротевшие» визуализаторы без глав (контент в холостую)

- * `stack-dfs` → `StackViz.tsx` – ****Стек и DFS****
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – ****Очередь и BFS****
- * `heap-beam-search` → `HeapViz.tsx` – ****Куча и лучевой поиск****
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – ****Дерево отрезков****
- * `dynamic-programming` → `DPVisualizer.tsx` – ****Динамическое программирование****

| | | | |
|----|---|---|-------|
| 9 | 9. Графы. Топологическая сортировка | 1 | - |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | |
| 12 | 12. Графы. Точки сочленения | 1 | - - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | |
| 16 | 16. Графы. Кратчайший путь. Флойд | - | - |
| 17 | 17. Графы. Алгоритм Джонсона | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - - |
| 22 | 22. Z-функция | 4 | - - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - - |
| 24 | 24. Классы сложности, сведение задач | 4 | - - |
| - | Обзор: Кратчайшие пути и Графы | - | - - |
| - | Алгоритмы на карте | - | - - |
| - | Эйлер: Путь ≠ Цикл | - | - - |
| - | Мосты в графах: код + визуализация | 1 | - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш
3. ****Добавить 2D**** к 4 главам «Уровня 4» — минимум `inli`
4. ****Решить судьбу `alg-map`**** — раскрыть или удалить.

ФАЙЛ 9/87: `tsconfig.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
```



```
-----
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локаль
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

```

const toggleSidebar = useCallback(() => {
  setSidebarCollapsed((c) => {
    window.localStorage.setItem("sidebarCollapsed", c);
    return !c;
  });
}, []);

// Desktopный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.innerWidth - compilerWidth;
  };
  apply();
  window.addEventListener("resize", apply);
  return () => window.removeEventListener("resize", apply);
}, [compilerOpen, compilerWidth]);

const index = Math.max(
  0,
  chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefined;
const next = index < chapters.length - 1 ? chapters[index + 1] : undefined;

const goTo = useCallback((id: string) => {
  setActiveChapterId(id);
  window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
  setCompilerOpen(true);
  requestAnimationFrame(() => {
    requestAnimationFrame(() => {

```

```

        total={chapters.length}
      />

      <div className="flex flex-col lg:flex-row m
        /* Сайдбар только на десктопе – на мобил
        <div
          className={`hidden lg:block shrink-0 bo
tion-all duration-200 ${sidebarCollapsed ?
        >
          {!sidebarCollapsed && <Sidebar selected
        </div>
        /* Вкладка-переключатель содержания у ле
        <button
          type="button"
          onClick={toggleSidebar}
          className="hidden lg:flex items-center
l-0 border-slate-700 bg-slate-900/90 px-1 p
          aria-label={sidebarCollapsed ? "Разверн
          title={sidebarCollapsed ? "Развернуть с
        >
          {sidebarCollapsed ? <PanelLeftOpen clas
          <span className="text-[10px] font-bold
        </button>

      <main id="main-content" className="flex-1
        /* Шапка главы с быстрыми переходами */
        <div className="flex items-center justi
          <div className="min-w-0">
            <div className="text-[10px] upperca
              {activeChapter.category || "Универ
            </div>
            <h2 className="text-base sm:text-xl
              {activeChapter.title}
            </h2>
          </div>
          <ChapterNav prev={prev} next={next} in
        </div>

```

```

<SimulatorModal vizId={modalVizId} onClose={() =>

  {compilerOpen && (
    <PythonCompiler
      chapterId={activeChapter.id}
      chapterTitle={activeChapter.title}
      width={compilerWidth}
      onWidthChange={setCompilerWidth}
      onOpenGuide={() => setActiveTab("guide")}
      onClose={() => setCompilerOpen(false)}
    />
  )}
</div>
);
}

```

ФАЙЛ 12/87: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```

import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/

```

```

for (const textNode of targets) {
  const text = textNode.nodeValue ?? "";
  re.lastIndex = 0;
  if (!re.test(text)) continue;
  re.lastIndex = 0;

  const frag = document.createDocumentFragment();
  let last = 0;
  let m: RegExpExecArray | null;

  while ((m = re.exec(text)) !== null) {
    const surface = m[1].toLowerCase();
    const id = labelToId.get(surface);
    if (!id) continue;

    const usedTerm = perTerm.get(id) ?? 0;
    const usedSurface = perSurface.get(surface) ?? 0;
    if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
    perTerm.set(id, usedTerm + 1);
    perSurface.set(surface, usedSurface + 1);

    if (m.index > last) frag.appendChild(document.createElement("span"));
    const span = document.createElement("span");
    span.className = "term-hint";
    span.dataset.termId = id;
    span.tabIndex = 0;
    span.setAttribute("role", "button");
    span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
    span.textContent = m[1];
    frag.appendChild(span);
    last = m.index + m[1].length;
  }

  if (last === 0) continue;
  if (last < text.length) frag.appendChild(document.createElement("span"));
  textNode.parentNode?.replaceChild(frag, textNode);
}

```

```
@import "tailwindcss";

@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
    box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
  }
  .no-scrollbar::-webkit-scrollbar {
    display: none;
  }
  .no-scrollbar {
    scrollbar-width: none;
  }
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
```

```
.term-hint:hover,  
.term-hint:focus-visible {  
  background: rgba(99, 102, 241, 0.22);  
  color: #fff;  
  outline: none;  
}  
@media (hover: none) {  
  .term-hint {  
    border-bottom-style: solid;  
    background: rgba(99, 102, 241, 0.12);  
  }  
}
```

/* Контент главы приходит готовым HTML — чиним его под

```
.chapter-body {  
  max-width: 100%;  
  overflow-wrap: anywhere;  
}  
  
.chapter-body :where(section, div, p, li) {  
  min-width: 0;  
}
```

```
.chapter-body :where(pre, code) {  
  white-space: pre-wrap;  
  word-break: break-word;  
}
```

```
.chapter-body pre {  
  overflow-x: auto;  
  max-width: 100%;  
  font-size: clamp(11px, 2.9vw, 13px);  
}
```

```
.chapter-body table {  
  display: block;  
  width: 100%;  
  overflow-x: auto;
```

```
    height: 8px;
  }
  ::-webkit-scrollbar-track {
    background: #0f172a;
  }
  ::-webkit-scrollbar-thumb {
    background: #334155;
    border-radius: 4px;
  }
  ::-webkit-scrollbar-thumb:hover {
    background: #475569;
  }

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
  from {
    width: 0%;
  }
  to {
    width: 100%;
  }
}
```

ФАЙЛ 14/87: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```



```
const src: string;
export default src;
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 17/87: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const additionalTickets: Chapter[] = [
  {
```

```
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
```

```
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">Новое</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">2. Двумерная разреженная матрица (Sparse Table)</h2>
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
    <h3 class="text-xl font-bold text-blue-400">2.1. Алгоритм построения</h3>
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">Пусть  $ST[L][k]$  — значение минимума на отрезке  $[L, R]$  длины  $2^k$ . Тогда для вычисления  $ST[R][k]$  рассмотрим отрезки  $[L, L + 2^{k-1}]$  и  $[L + 2^{k-1}, R]$ . Тогда:
      <p class="text-xl font-mono text-white">

$$ST[R][k] = \min(ST[L][k], ST[R - 2^{k-1}][k])$$

      </p>
    </div>
  </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
```

```

    id: "treap",
    title: "3. Декартово дерево (Treap)",
    type: "html",
    description: "Дерево поиска по ключу и Куча по приоритету",
    category: "Продвинутые структуры",
    content: `
<section id="treap" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">3
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Декартово дерево (Treap)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">Описание
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Декартово дерево — это бинарное дерево поиска, в котором каждый узел имеет два значения: ключ (key) и приоритет (priority). Ключ используется для поиска, а приоритет используется для поддержания структуры дерева. Узлы с более высоким приоритетом поднимаются вверх по дереву.
        <p class="text-xl font-mono text-white">Алгоритмы: Insert, Delete, Find.
        метода: Split и Merge.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Удар катаной (Split)
          </div>
          <p class="text-slate-300 text-sm mb-2">Представим, что мы ударяем катаной по дереву в точке X. Все узлы с приоритетом меньше X улетают влево, а все с приоритетом больше X — вправо.
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Аналогия 2: Слияние капель (Merge)
          </div>
          <p class="text-slate-300 text-sm mb-2">Представим, что мы сливаем две кучи. Мы просто смотрим на корни: у кого Y меньше X, тот становится родителем.
          <p class="text-slate-300 text-sm mb-2">вого). Мы просто смотрим на корни: у кого Y
          как потомок.</p>
        </div>
      </div>
    </div>
  </div>
`

```

```
<div class="space-y-8">
```

```
<!-- 0. Определение -->
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-500">
  <h3 class="text-xl font-bold text-blue-400 mb-4">Определение
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-blue-300 italic">Определение
    <p class="text-lg sm:text-xl font-mono">
      поиска <span class="text-slate-500">без</span>
      : серия поворотов, которая поднимает v в ко
      <p class="text-sm text-slate-400 text-c">
      т – поворот.</p>
  </div>
```

```

  <div class="grid grid-cols-1 lg:grid-cols-3 gap-4">
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-xs uppercase tracking-wider">Определение
      <p class="text-white font-bold">мож
      <p class="text-slate-400 text-xs mt-1">
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-xs uppercase tracking-wider">Определение
      <p class="text-emerald-300 font-bold">мож
      <p class="text-slate-400 text-xs mt-1">
    </div>
    <div class="bg-slate-900 rounded-lg p-4">
      <p class="text-xs uppercase tracking-wider">Определение
      <p class="text-white font-bold">0(n
      <p class="text-slate-400 text-xs mt-1">
    </div>
  </div>
</div>
```

```
<!-- 1. Зачем -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-500">
  <h3 class="text-lg font-bold text-white mb-4">Зачем
  <p class="text-slate-300 text-sm mb-4">В об
  ючи по возрастанию – и дерево вырождается в
```

Аналогия 2: тропинка через газон

Ты идёшь по дорожке, а дорожка сама укорачивалась вдвое: чем больше идёшь, тем меньше остаётся до конца. И так оплачивает все следующие. Пойдёшь один раз

<!-- 3. Поворот -->

<div class="bg-slate-800/60 p-6 rounded-xl bord

class="text-lq font-bold text-white mb-

`<p class="text-slate-300 text-sm mb-4">Пово
движение, три перевешенные ссылки и`

```

<pre class="bg-slate-950 p-4 rounded-lg overflow-relaxed">
    40

```

обход слева направо ДО: А 20 В 40 С

обход слева направо ПОСЛЕ: A 20 B 40 C ← тот же шаг

```
<div class="grid grid-cols-1 md:grid-cols-2
```

```
<div class="bg-slate-900 rounded-lg p-4
```

<p class="text-emerald-400 font-bol

<p class="text-slate-300 text-xs">Π

сла поворотов – сломать его поворотами нево.

```
<div class="bg-slate-900 rounded-lg p-4
```

text-amber-400 font-bold

<p class="text-slate-300 text-xs">Π

одно и то же место, поэтому В просто перев

<!-- 4. Три случая -->

<div class="bg-slate-800/60 p-6 rounded-xl bord

```
<p class="text-slate-300 text-sm mt-4">И та  
нова смотрим на тройку. Zig может случиться  
<p class="text-slate-400 text-xs mt-3"> Ни  
вороты можно поделат руками.</p>
```

```
</div>
```

```
<!-- 5. Главная ловушка -->
```

```
<div class="bg-rose-950/30 p-6 rounded-xl borde
```

```
<h3 class="text-lg font-bold text-rose-300 m
```

```
<p class="text-slate-300 text-sm mb-4">Это  
ми.</p>
```

```
<div class="grid grid-cols-1 lg:grid-cols-2
```

```
<div class="bg-slate-950 rounded-lg p-4
```

```
<p class="text-rose-400 font-bold t
```

```
<pre class="overflow-x-auto text-[1
```

```

      20
     /
  40 /
 /
20

→

      20
     /
  20 /
   \|
   40

→

      \|
      60
     /
  40
```

было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!</pre>

```
<p class="text-slate-400 text-xs mt
мортизации не получится.</p>
```

```
</div>
```

```
<div class="bg-slate-950 rounded-lg p-4
```

```
<p class="text-emerald-400 font-bol
```

```
<pre class="overflow-x-auto text-[1
```

```

      20
     /  \|
  40 /   60
 /
20

→

      20
     /  \|
  20 /   60
   /
  A

→

      /  \|
      40  60
     \|
      60
```

было: линия из 3 узлов

ах идея такая:</p>

```
<div class="grid grid-cols-1 md:grid-cols-3">
  <div class="bg-slate-900 rounded-lg p-4">
    <p class="text-white font-bold mb-1">
    <p class="text-slate-400 text-xs">с
  </div>
  <div class="bg-slate-900 rounded-lg p-4">
    <p class="text-white font-bold mb-1">
    <p class="text-slate-400 text-xs">н
  </div>
  <div class="bg-slate-900 rounded-lg p-4">
    <p class="text-white font-bold mb-1">
    <p class="text-slate-400 text-xs">д
  </div>
```

```
</div>
<p class="text-slate-400 text-xs mt-4">Отсю
сти запросов splay-дерево не хуже (с точнос
дерево. Оно как бы само подстраивается под
```

```
</div>
```

```
<!-- 8. Операции -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
  <h3 class="text-lg font-bold text-white mb-3">
  <p class="text-slate-300 text-sm mb-4">Крас
лается в корне.</p>
  <div class="space-y-2 text-sm">
    <div class="bg-slate-900 rounded-lg p-3">
  </span> <span class="text-slate-400">— спус
маем последний узел на пути (соседний по зн
    <div class="bg-slate-900 rounded-lg p-3">
  </span> <span class="text-slate-400">— Spl
росто отрезаем ссылки.</span></div>
    <div class="bg-slate-900 rounded-lg p-3">
  </span> <span class="text-slate-400">— sp
  </div>
    <div class="bg-slate-900 rounded-lg p-3">
  </span> <span class="text-slate-400">— Spl
него не будет правого сына — туда и вешаем
```

```

        rotate(x);
    } else if ((g.left === p) === (p.left === x)) {
        rotate(p); // ZIG-ZIG: сн
        rotate(x);
    } else {
        rotate(x); // ZIG-ZAG: сн
        rotate(x);
    }
}
return x; // теперь x –
}

```

// поиск: спустились и подняли найденное наверх

```

function find(root, key) {
    let cur = root, last = null;
    while (cur) {
        last = cur;
        if (key === cur.key) break;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return last ? splay(last) : null; // splay делает
}

```

<p class="text-xs text-slate-400">Вся р
ate(x); против <span class="font-mon
т свою оценку.</p>

</div>

</details>

<!-- 10. Плюсы/минусы -->

<div class="grid grid-cols-1 md:grid-cols-2 gap

<div class="bg-emerald-950/30 rounded-xl p-

<p class="text-emerald-300 font-bold mb

<ul class="list-disc list-inside text-s

Простой код: нет высот, цветов,

Меньше памяти, чем у AVL и крас

Сам подстраивается под «горячие

split / merge получаются почти


```

        <p class="text-slate-400">Треар дер
play не хранит вообще ничего и держит балан
p>
    </div>
  </div>
</details>

</div>
</section>`,
  },
  {
    id: "prefix-sums-2d",
    title: "5. Двумерная матрица префиксных сумм",
    type: "html",
    description: "Сжатие суммы подматрицы за O(1)",
    category: "Алгоритмы матрицы",
    content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
      <h2 class="text-2xl sm:text-3xl font-bold text-
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border
        <h3 class="text-xl font-bold text-emerald-4
        <div class="bg-slate-900 p-4 rounded-lg mb-
          <p class="text-lg text-emerald-300 ital
          <p class="text-xl font-mono text-white"
            , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2
          <div class="bg-slate-800 p-6 rounded-lg
            <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
              ≈ Аналогия 1: Вырезание прямоу
            </div>
            <p class="text-slate-300 text-sm mb
              з него маленький целевой прямоугольник в це

```



```

        </div>
    </div>
</section>`,
    },
    {
        id: "top-sort",
        title: "9. Графы. Топологическая сортировка",
        type: "html",
        description: "Разложение задач по порядку выполнения",
        category: "Графы",
        content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                <p class="text-xl font-mono text-white">
перед.</p>
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        Аналогия 1: Учеба в вузе
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
                >
            </div>
            <div class="bg-slate-900 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                    Ограничения: Циклы
                </div>
            </div>
        </div>
    </div>
</section>`
    }
]

```

```

        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Рёбра, удаление которых расхреначивает",
    category: "Графы. Продвинутое",
    content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb-4">
            этот мост, экономикой обоих кусков придет
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрывая
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Если сходится нечетное число линий, значит, из
            этой вершины зашел-вышел. Если сходится четное
            число, значит, зашел-вышел. А везде должно быть
            четное число (зашел-вышел).
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "pathfinding-accel",
    title: "17. Графы. Ускорения поиска",
    type: "html",
    description: "",
    category: "Графы. Пути",
  },

```

```

        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">
                <p class="text-xl font-mono text-white">
                    (проверяем через DSU) - берем в ответ.</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border
                    order border-emerald-500 shadow-md">
                        Аналогия: Прокладка интернет
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        с самых дешевых дорог в стране". Он строит
                        единую сеть.</p>
                </div>
            </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "mst-prima",
        title: "19. Графы. Остовное дерево. Прима",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-12">
  <p class="text-lg text-emerald-300 italic">
    <p class="text-xl font-mono text-white">
      ается с соседом.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border
          order border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-4">
            изкому племени для объединения. К вечеру пл
            ства шлют гонцов к ближайшему чужому царств
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "string-kmp",
    title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
    type: "html",
    description: "",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-blue-300 italic">
          <p class="text-xl font-mono text-white">

```

```

type: "html",
description: "",
category: "Строки",
content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
щегося с i.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
            Аналогия 1: Копипаста
          </div>
          <p class="text-slate-300 text-sm mb-4">
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной копи
          </div>
        </div>
        <details class="bg-slate-900/40 rounded-lg">
          <summary class="p-4 font-bold text-slate-300">
            -b border-slate-700/50">
              Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
              <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
  if (i <= r) z[i] = min(r - i + 1, z[i - l]);

```

```
        енную сетку (ответ), он БЫСТРО (за класс Р)
      </div>
      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #800080; border-radius: 10px; box-shadow: 5px 5px 0px #800080;">
        <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #800080; border-radius: 10px; box-shadow: 5px 5px 0px #800080;">
          Аналогия 2: Машина-переводчик
        </div>
        <p class="text-slate-300 text-sm mb-0">
          ь отличный переводчик на английский (Сведения о машине В, то В – это <b>NP-Полная</b> (сосредоточена на
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
];
```

ФАЙЛ 18/87: src/data/content_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМЕР: 10.5 КБ

```
export interface Chapter {
  id: string;
  title: string;
  type: "html";
  content: string;
}

export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="text-slate-300 text-sm mb-0">
      <h3>Эйлер: Путь ≠ Цикл (ты путаешь!)</h3>
      <p>В задаче В, то В – это <b>NP-Полная</b> (сосредоточена на
    `
  }
];
```

```
<p class="text-slate-300 text-sm mb-4">
  Ты вышел из дома <strong>(нечётная)</strong>.
  талый пришёл в бар <strong>(вторая нечётная)</strong>.
  Домой вернуться не смог, потому что
</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md bg-slate-950">
  Цикл: Гамильтонов синдром (болезнь Гамильтона)
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
  <strong>Гамильтон – это болезнь.</strong>
  ng> ровно раз и вернуться домой.
  Ты обходишь все бары (вершины) ровно раз.
  Главное – зайти и выйти, не заходя дважды.
  Никто не знает, как решать быстро.
  <br><span class="text-rose-400">
    лноту).</span>
  </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md bg-slate-950">
  <summary class="p-4 font-bold text-slate-400">
    Душный режим: Формальное доказательство
  </summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
  <p class="text-blue-400">Теорема Эйлера
  <p>
```

```
    Связный граф содержит эйлеров цикл ⇔
    <br>Эйлеров путь (не цикл) ⇔ ровно 0 или 2 вершины имеют нечётную степень
  </p>
```

```
<p class="text-rose-400">Почему Гамильтон?
<p>
```

```
    Задача коммивояжёра (TSP) – это взвешенный граф, где нужно найти кратчайший цикл, проходящий через все вершины.
```



```
        </p>
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg border-emerald-500 shadow-md">
            <div class="absolute -top-3 left-4 bg-slate-800 border-rose-500 shadow-md">
                Мост: Единственная веревка
            </div>
            <p class="text-slate-300 text-sm mb-4">
                Представь, что ты альпинист. Ты спустился в пещеру.
                Из пещеры и нет других выходов — только одна веревка.
                Если верёвка (ребро v-u) оборвётся, ты погибнешь.
                Если бы из пещеры и был чёрный ход, ты бы уже вышел.
            </p>
        </div>

        <div class="bg-slate-900 p-6 rounded-lg border-emerald-500 shadow-md">
            <div class="absolute -top-3 left-4 bg-slate-900 border-rose-500 shadow-md">
                Аналогия: Кровеносный сосуд
            </div>
            <p class="text-slate-300 text-sm mb-4">
                Граф — это кровеносная система. Ребра (сосуды) — это не мост, живём.
                Если же перерезать единственную артерию, ты погибнешь.
                Алгоритм DFS ищет такие «критические» ребра.
            </p>
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border-emerald-500 shadow-md">
        <summary class="p-4 font-bold text-slate-400">
            Полный код на C++ (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300 space-y-4">
            <p>Классическая реализация — <code class="text-slate-400">
```

```
void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}
```

```
</div>
<p class="text-xs text-slate-400">Занес
</div>
</details>
</div>
</section>`,
},
{
    id: "planarity-euler-formula",
    title: "Планарность: K5, K3,3 и формула Эйлера",
    type: "html",
    content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-3xl font-bold text-white">Планар
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
        <h3 class="text-xl font-bold text-blue-400 r

        <div class="bg-slate-900 p-4 rounded-lg mb-6">
            <p class="text-lg text-blue-300 italic r
            <p class="text-3xl font-mono text-white"
            <div class="text-sm text-slate-400 spac
                <p><span class="text-blue-400 font-l
                <p><span class="text-emerald-400 fo
                <p><span class="text-rose-400 font-l`
}
```

```

        </p>
      </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border
      <summary class="p-4 font-bold text-slate-400
        order-slate-700/50">
        Доказательство непланарности K5 (Скры
      </summary>
      <div class="p-5 text-sm text-slate-300 space-y-4">
        <p class="text-blue-400">Доказательство
        <p>
          Пусть K5 планарен.  $V = 5$ ,  $E = 10$ .
          <br> $F = E - V + 2 = 10 - 5 + 2 = 7$ 
          <br>Каждая грань ограничена минимум
          <br>Сумма длин граней  $= 2E = 20$ .
          <br>Отсюда:  $2E \geq 3F \Rightarrow 20 \geq 21$  — <span>
          <br>Значит, K5 не может быть планарн
        </p>
      </div>
    </details>
  </div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    content: `<section id="graph-articulation" class="mb-6">
      <div class="flex items-center mb-6">
        <span class="bg-rose-600 text-white px-4 py-1 rounded">
        <h2 class="text-3xl font-bold text-white">Точки
      </div>

      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
          <h3 class="text-xl font-bold text-rose-400">

```

```

        <p class="text-slate-300 text-sm mb-4">
            У тебя несколько компьютеров в одной
            абелем (мостом). Сами свитчи — это точки со
        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border
    <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
        Душный мод: Код и корень DFS (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space
        <p class="text-blue-400 font-bold">Особ
        <p>
            Для стартовой вершины обхода правил
            то выше неё прыгать некуда). Поэтому для не
            у него <strong>больше 1 независимого ребенка
        </p>
        <div class="bg-slate-950 p-4 rounded-lg
            <pre class="text-xs font-mono text-
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
}

```

```

-----
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
  if (!toRemove.includes(c.id)) {
    // We prefer the newer versions (like tickets14_20
    dedup.set(c.id, c);
  }
});

// Add the new ones
if (newChapters) {
  newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
  const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
  return numA - numB;
});

```

```

    truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
    const srcLiteral = JSON.stringify(userSrc); // JSON-сериализация
    return `import sys as __sys, json as __json

__src = ${srcLiteral}
__steps = []
__final = {}
__CAP = ${DEBUG_STEP_CAP}

def __safe(v):
    try:
        r = repr(v)
    except Exception:
        r = "<...>"
    return r if len(r) <= 80 else r[:77] + "..."

def __tracer(frame, event, arg):
    if event == "line" and frame.f_code.co_filename == __src
        and not (frame.f_code.co_name.startswith("<
        loc = {}
        for k, v in frame.f_locals.items():
            if not k.startswith("__") and type(v).__name__ != "function":
                loc[k] = __safe(v)
        # PEP 709: с 3.12 компрекхеншны встроены во фрейм
        # на своей строке каждую итерацию (9 ложных «шагов» в строке)
        # Склеиваем ПОДРЯД идущие события одной строки и
        if steps and steps[-1]["line"] == frame.f_lineno`

```

```
* Новые переменные тоже считаются изменёнными.
*/
export function changedVars(
  prev: Record<string, string> | null | undefined,
  cur: Record<string, string>
): Record<string, boolean> {
  const out: Record<string, boolean> = {};
  for (const k of Object.keys(cur)) {
    out[k] = !prev || !(k in prev) || prev[k] !== cur[k];
  }
  return out;
}

/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name.slice(0, -1);

/**
* Порядок показа в панели переменных: сначала переменные
* страницы (те, что подсвечивает визуализация), остальные
*/
export function orderVars(
  cur: Record<string, string>,
  syncNames: string[]
): [string, string][] {
  const inSync = new Set(syncNames);
  const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
  const rest = Object.entries(cur)
    .filter(([k]) => !inSync.has(k))
    .sort(([a], [b]) => a.localeCompare(b));
  return [...syncEntries, ...rest];
}
```

```

    },
    {
      id: "dfs",
      labels: ["DFS", "обход в глубину", "поиск в глубину"],
      title: "DFS — обход в глубину",
      short:
        "Прёшь вперёд до упора, упёрся — откатываешься на шаг",
      formal:
        "Стек (или рекурсия). Из DFS растут: времена входа и выхода",
      complexity: "O(V + E)",
      demo: "dfs",
      simulator: "stack-dfs",
    },
    {
      id: "heap",
      labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
      title: "Куча (priority queue)",
      short:
        "Не отсортированный список, а «мешок, из которого достаёшь минимальный элемент»",
      formal:
        "Полное бинарное дерево в массиве: дети узла i лежат в узлах 2i и 2i+1",
      complexity: "вставка/извлечение O(log n), «посмотреть минимальный элемент» O(1)",
      demo: "heap",
      simulator: "heap-beam-search",
    },
    {
      id: "stack",
      labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
      title: "Стек (LIFO)",
      short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
      formal: "push / pop / top за O(1). На стеке живёт рекурсия",
      demo: "stack",
      simulator: "stack-dfs",
    },
    {
      id: "queue",
      labels: ["очередь", "очереди", "очередью", "FIFO"],
      title: "Очередь (FIFO)",

```



```

        "автомат",
        "автомата",
    ],
    title: "Обход бора в ширину (тот самый «какой-то обход бора»)",
    short:
        "В Ахо–Корасике бор обходят именно BFS: суффиксную",
        "или есть строго по уровням.",
    formal:
        "Кладём в очередь детей корня (их ссылка = корень",
        "и кладём с в очередь.",
    complexity: "O(суммарной длины словаря × алфавит)",
    demo: "trie-bfs",
    simulator: "aho-corasick",
},
{
    id: "suffix-link",
    labels: ["суффиксная ссылка", "суффиксные ссылки", "ссылка"],
    title: "Суффиксная ссылка",
    short:
        "«Куда откатиться, если следующая буква не подошла",
        "строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топология"],
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с",
        "циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л",
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{

```

```

    id: "segment-tree",
    labels: ["дерево отрезков", "дереве отрезков", "дер"],
    title: "Дерево отрезков",
    short:
        "Матрёшка из отрезков: корень отвечает за весь ма
        товых кусков.",
    formal: "Строится за  $O(n)$ , запрос и обновление —  $O($ 
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостов"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается.
    formal: "Ребро  $(u,v)$  — мост  $\iff \text{low}[v] > \text{tin}[u]$ , где
        ребро в родителя — нет).",
    complexity: " $O(V + E)$  одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "т"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и колл
    formal: "Для не-корня:  $v$  — точка сочленения  $\iff \exists$  реб
    demo: "articulation",
    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное"
        "минимальное остовное дерево", "MST",
    ],

```

```

    short:
        "Трюк «поднять все дороги на одинаковую высоту»:
formal: "w'(u,v) = w(u,v) + h(u) - h(v), где h — разность
        к путей сохраняется.",
demo: "relax",
simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компоненты"],
    title: "Компоненты сильной связности",
    short:
        "Группы вершин, где из каждой можно дойти до каждой",
formal: "Косарайю: DFS с сохранением порядка выхода",
complexity: "O(V + E)",
demo: "scc",
simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функция",
        "префикс", "префикса", "суффиксом", "суффикс", "суффиксы",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция  $\pi$  (КМП)",
    short:
        "Для каждой позиции запоминаем: какой кусок начался в этой позиции
        от кусок.",
formal:
        " $\pi[i]$  — длина наибольшего собственного префикса s[i..n],
        возвращая указатель по тексту.",
complexity: "O(n + m)",
demo: "prefix-function",
simulator: "string-kmp",
},
{
    id: "z-function",

```

```
{
  id: "negative-cycle",
  labels: ["отрицательный цикл", "отрицательного цикла"],
  title: "Отрицательный цикл",
  short:
    "Круг, пройдя по которому вы становитесь «богаче»",
  formal:
    "Форд–Беллман: если на  $V$ -й итерации ещё удаётся с",
  demo: "relax",
  simulator: "bellman-ford",
},
{
  id: "greedy",
  labels: [
    "жадный алгоритм", "жадный", "жадно", "жадная",
    "самое дешёвое ребро", "самое дешёвое ребро", "са
  ],
  title: "Жадный алгоритм",
  short:
    "На каждом шаге хватаем то, что лучше прямо сейчас",
  formal:
    "Корректность обычно доказывается «леммой о разре",
  demo: "mst",
  simulator: "mst-kruskal",
},
{
  id: "euler",
  labels: ["эйлеров цикл", "эйлеров путь", "эйлерова п
  ],
  title: "Эйлеров путь и цикл",
  short:
    "Пройти по каждому ребру ровно один раз, не отрыв
  ],
  formal:
    "Связный граф: эйлеров цикл  все степени чётные;
  ],
  simulator: "euler-path-vs-cycle",
},
{
  id: "planarity",
  labels: ["планарный", "планарность", "планарного",
```

```

        льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
complexity: "O(E log V) с бинарной кучей",
demo: "relax",
simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
title: "Алгоритм Форда-Беллмана",
short:
    "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбра
    ",
    formal:
        "После k-й итерации гарантированно верны все крат
        ось — есть отрицательный цикл.",
    complexity: "O(V · E)",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше
        "промежуточная вершина", "промежуточную вершину",
    ],
    title: "Флойд-Уоршелл: разрешаем вершину k",
    short:
        "Внешний цикл — это не «шаг», а «какие пересадки
        шить пересадку в k?",
    formal:
        "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Поряд
        ами.",
    complexity: "O(V³) времени, O(V²) памяти",
    demo: "floyd",
    simulator: "floyd",
},

```

```

    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "
    ],
    title: "Компоненты связности",
    short:
        "Граф-архипелаг: острова, между которыми нет мост
        бой.",
    formal:
        "Считаются одним проходом: пока есть непосещённая
    complexity: "O(V + E)",
    demo: "components",
    simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "гра
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Горо
    formal:
        "G = (V, E). Ребро бывает ориентированным (улица
    demo: "graph",
    simulator: "graph-dfs-bfs",
},
{
    id: "coord-compress",
    labels: ["сжатие координат", "сжатия координат", "с
    title: "Сжатие координат",
    short:
        "Координаты бывают гигантские и дробные, а нам ну
        порядковым номером.",
    formal:
        "sort + unique + binary search: значение → индекс

```

```

    formal:
      "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
    complexity: "запрос O(1) после O(nm) предподсчёта",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "reduction",
    labels: ["сведение", "сведения", "сведении", "сведён", "сведёно"],
    title: "Сведение задач",
    short:
      "«Я не умею решать A, зато умею B». Если A можно решить, то и B можно решить.",
    formal:
      "A ≤p B: есть полиномиальная функция, переводящая решение A в решение B.",
    demo: "np",
  },
  {
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки", "отрезком"],
    title: "Запрос на отрезке",
    short:
      "Спрашиваем не про один элемент, а про кусок массива. Как считать суммы на отрезке?",
    formal:
      "Суммы — префиксные суммы O(1). Минимум без изменений.",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "строкам"],
    title: "Строка как массив символов",
    short:
      "Строковые алгоритмы держатся на трёх вопросах: где встречается слово (Z-функция) и где встречаются слова словаря (бор).",
    formal:
      "Наивный поиск подстроки — O(N·M). Префикс-функция — O(N).",
  }
}

```

```

    simulator: "splay-rotations",
  },
  {
    id: "zig-zag",
    labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
    title: "Zig-Zag – x и родитель с разных сторон",
    short:
      "Узлы идут «змейкой»: p – левый сын g, а x – прав
        p и g становятся двумя детьми x.",
    formal: "Эквивалентно двойному повороту AVL-дерева",
    demo: "zig-zag",
    simulator: "splay-rotations",
  },
];

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  e.labels.map((label) => ({ label, id: e.id }))
}).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/87: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 275 | РАЗМЕР: 1.2 КБ

```
import { Chapter } from "../types";
```

```

export const graphChapters: Chapter[] = [
  {
    id: "intro",
    title: "Обзор: Кратчайшие пути и Графы",
    type: "html",
    category: "Графы",
    content: `<section id="intro-content" class="mb-12

```



```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    Аналогия 1: Позитивное планирование
  </div>
  <p class="text-slate-300 text-sm mb-2">
    (нельзя поехать и заработать). Он всегда выбирает кратчайший путь.
  <div id="slot-chapter-image-dijkstra" style="height: 100px; width: 100%; background-color: #000; position: relative; margin-top: 10px;">
    </div>

    <!-- Аналогия 2 -->
    <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px;">
      <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
        Ограничения
      </div>
      <p class="text-slate-300 text-sm mb-2">
        (не может проходить по дорожке для пешеходов), Дейкстра сломается, так как не учитывает дорожку для пешеходов.
      </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin-top: 10px;">
      <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #000; border-radius: 10px; margin-bottom: 5px;">
        Строгое доказательство (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300" style="border: 1px solid #000; border-radius: 10px; margin-top: 5px;">
        <p>Алгоритм использует приоритетную очередь (min-heap). Гарантирует корректность только для не-взвешенных графов (или с неотрицательными весами).
      </div>
    </details>

    <div id="slot-dijkstra-viz" class="mt-8"></div>
  </div>
</div>
</section>`
},
{

```

```

        г станет ЕЩЕ короче — значит мы попали во в
        </div>
    </div>

    <div id="slot-bellman-viz" class="mt-8"></div>
  </div>
</div>
</section>`
},
{
  id: "floyd",
  title: "Билет 16. Флойд-Уоршелл",
  type: "html",
  category: "Графы",
  content: `<section id="floyd-content" class="mb-12">
    <div class="flex items-center mb-6">
      <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-3xl font-bold text-white">Флойд
    </div>

    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-indigo-400">

        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-indigo-300 italic">
            <p class="text-xl font-mono text-white">
          </div>

        <div class="grid grid-cols-1 xl:grid-cols-2">
          <!-- Аналогия 1 -->
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border
            rder border-emerald-500 shadow-md">
              Аналогия 1: Автомагистрали
            </div>
            <p class="text-slate-300 text-sm mb-4">
              евле долететь из Москвы в Париж, если мы сд
```

```
<p class="text-slate-300 text-sm">Позволяет найти
-mono text-amber-300 bg-black/30 px-1 rounded">
  у.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
        Аналогия 1: Паутина (Бор)
      </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы двигаемся по веткам. Если нужного продолж
        ("нити страховки") в самый длинный из извест
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2 l
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
          Аналогия 2: Матёшка (Терминальные)
        </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
        минальные ссылки</b> (term_link) – прямые мос
      <p>
    </div>
  </div>

<details class="bg-slate-900/40 rounded-lg border b
  <summary class="p-4 font-bold text-slate-400 outl
    pen:border-slate-700/50">
    Внутренняя структура автомата и код BFS (Скры
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <p>Каждая вершина имеет таблицу переходов <code>
      ссылку <code>term_link</code> (ближайшая мен
    <div class="bg-black/50 p-4 rounded-lg font-mono
```

```

        </div>
      </details>
    </div>
  </section>`
  },
  {
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-12">
      <div class="flex items-center mb-6">
        <span class="bg-purple-600 text-white px-4 py-1">
          <h2 class="text-3xl font-bold text-white">Алгори
        </div>

        <div class="space-y-8">
          <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-purple-400">
              <p class="text-slate-300 text-sm mb-4">Когда
              ут быть большими дробными числами (долгота
              ву от 0 до N, и уже по ним строим графы.</p>
            </div>
          </div>
        </div>
      </section>`
    }
  ];

```

ФАЙЛ 23/87: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМ

```

export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
}

```

```

    ],
    "bridges-code": [
        {
            question: "После DFS у нас  $low[u] = 4$ , а  $tin[v] =$   

            options: [
                "Да, так как  $low[u] (4) > tin[v] (2)$ ",
                "Нет,  $low[u]$  должно быть меньше или равно",
                "Только если граф связный и неориентированный"
            ],
            correctIndex: 0,
            explanation: "Условие моста:  $low[u] > tin[v]$ . В н
        },
        {
            question: "Для чего нужен массив  $low[v]$ ?",
            options: [
                "Для того чтобы было больше переменных в коде",
                "Чтобы знать минимальный  $tin$  предка, до которого  

                "Чтобы хранить глубину рекурсии"
            ],
            correctIndex: 1,
            explanation: " $low[v]$  хранит минимальное время входа  

            поиска мостов и точек сочленения."
        },
        {
            question: "Какова временная сложность алгоритма п  

            options: [
                " $O(V^2)$  — ищем мосты в квадрате",
                " $O(V + E)$  — каждый элемент один раз",
                " $O(2^V)$  — полный перебор"
            ],
            correctIndex: 1,
            explanation: "DFS обходит каждую вершину и каждое
        }
    ],
    "planarity-euler-formula": [
        {
            question: "Планарный граф имеет  $V=6$ ,  $E=12$ . Возмо  

            options: [

```

```
/**
 * Связь «компилятор ↔ открытая визуализация».
 *
 * Панель компилятора в режиме пошагового отладчика тра
 * текущего шага трассы вместе с id текущей главы; поша
 * главы (через useVizStepSync) подхватывают его и прыг
 * визуализация идёт в ногу с листингом (каждая исполня
 * = очередной шаг подсветки, см. vizSync.ts).
 */

export const VizChapterContext = createContext("");

const CHANNEL = "algo:viz-step";

export interface VizStepEventDetail {
  chapterId: string;
  step: number;
}

/** Транслировать текущий шаг отладчика визуализациям т
export function emitVizStep(chapterId: string, step: num
  if (typeof window === "undefined") return;
  window.dispatchEvent(new window.CustomEvent<VizStepEve
}

/**
 * Подписка визуализации на шаги отладчика.
 * Вызывается внутри компонента демонстрации:
 *   useVizStepSync(currentStep, setCurrentStep, steps.
 */
export function useVizStepSync(currentStep: number, goT
  const chapterId = useContext(VizChapterContext);

  useEffect(() => {
    if (!chapterId) return;
    const handler = (e: Event) => {
      const detail = (e as CustomEvent<VizStepEventDeta
```

```

): number {
  let hits = 0;
  for (let t = 0; t <= idx && t < steps.length; t++) {
    const content = srcLines[steps[t].line - 1];
    if (content !== undefined && markedContents.has(content)) {
      hits++;
    }
  }
  return hits;
}

/* — Выбор демонстрации внутри страницы (вкладки ID / ID) */

const DEMO_CHANNEL = "algo:viz-demo";

export interface VizDemoEventDetail {
  chapterId: string;
  /** base-запись страницы – demoId пустой; подписанные демо */
  demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
  if (typeof window === "undefined") return;
  window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
    detail: { chapterId, demoId }
  }));
}

/** Подписка: панель компилятора следует за активной вкладкой */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
  if (typeof window === "undefined") return () => {};
  const handler = (e: Event) => {
    const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
    if (detail && detail.chapterId === chapterId) cb(detail.demoId);
  };
  window.addEventListener(DEMO_CHANNEL, handler);
  return () => window.removeEventListener(DEMO_CHANNEL, handler);
}

```

```

-----
* шаг визуализации: по ним отладчик точно переводит
* шаг демонстрации (иначе – грубо «строчный индекс =
*/
stepCodeLines?: number[];
/**
* Может ли демонстрация страницы ходить за отладчиком
* false/нет – интерактив руками (клики/наведение): ч
*/
stepDriven?: boolean;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
  "segment-trees": {
    vizTitle: "Дерево отрезков",
    stepNote: "Шаг = спуск по вершине v: либо читаем tr
    code: `n = 8 # длина массива
tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
    variables: [
      { name: "n", role: "длина исходного массива; лист
      { name: "i", role: "индекс элемента массива; его
      { name: "v", role: "текущая вершина дерева отрезк
      { name: "l, r", role: "границы отрезка, за которы
      { name: "m", role: "середина отрезка m = (l + r)
    ],
  },
  treap: {
    // у страницы нет интерактивного виджета
    variables: [],
  },
  "splay-tree": {
    vizTitle: "Splay-дерево (Zig / Zig-Zig / Zig-Zag)",
    stepNote: "Шаг = один поворот, поднимающий узел x н

```



```

for j in range(1, LOG):
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j -
stepCodeLines: [4, 10],
variables: [
    { name: "n", role: "длина массива / сторона квадр",
    { name: "k", role: "уровень таблицы: ячейка храни
    { name: "i", role: "начало блока в строке уровня
    { name: "j", role: "номер уровня при построении (
    { name: "r, c", role: "строка и столбец ячейки в
    { name: "kx, ky", role: "уровни по вертикали и гор
],
},
"prefix-sums-2d": {
    vizTitle: "Префиксные суммы (1D и 2D)",
    stepNote: "Шаг = заполнение очередного P[i] = P[i-1
    stepDriven: true,
    code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
    P.append(P[i - 1] + a[i - 1])
print("P =", P)\`,
    stepCodeLines: [4, 6],
    variables: [
        { name: "n", role: "число строк матрицы", range:
        { name: "m", role: "число столбцов матрицы", rang
        { name: "i", role: "индекс текущей строки (таблиц
        { name: "j", role: "индекс текущего столбца (стол
        { name: "S[i][j]", role: "сумма прямоугольника (1
            на каждом шаге" },
    ],
},
"dynamic-programming": {
    vizTitle: "Динамическое программирование (мемоизаци
    stepNote: "Шаг = событие внутри fibМемо(k): вызов,
```

```

        { name: "2·i+1, 2·i+2", role: "индексы левого и правого соседа"},
    ],
},
"stack-dfs": {
    vizTitle: "Стек и обход в глубину",
    stepNote: "Шаг = push новой вершины на стек или pop",
    code: `st = [] # стек = рекурсия
st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень
print("pop →", top, "стек:", st)` ,
    variables: [
        { name: "top", role: "вершина стека — то, откуда пришли"},
        { name: "v", role: "текущая вершина графа, которую посещаем"},
    ],
},
"queue-bfs": {
    vizTitle: "Очередь и обход в ширину",
    stepDriven: true,
    stepNote: "Шаг = dequeue вершины из головы очереди",
    code: `from collections import deque

q = deque(["A"])
q.append("B") # enqueue соседа
v = q.popleft() # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
    variables: [
        { name: "front", role: "голова очереди — вершина, которую посещаем"},
        { name: "v", role: "вершина, извлечённая из очереди"},
        { name: "u", role: "сосед вершины v, которого кладем в очередь"},
    ],
},
"everyday-basics": {
    vizTitle: "Бытовой тренажёр: стек, очередь, куча",
    stepNote: "Шаг = одна бытовая операция: тарелка на стол, тарелка в кучу",
    code: `from collections import deque

st, q = [], deque() # тарелки и очередь

```

```

-----
code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2], 4: [1, 3]}
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:          # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)           # выход из v!

for v in adj:
    if v not in used:
        dfs(v)
print(order[::-1])           # разворот — ответ`,
variables: [
    { name: "v", role: "текущая вершина DFS", range:
    { name: "to", role: "вершина, куда ведёт ребро из v", range:
    { name: "i", role: "перебор стартовых вершин во вложенном цикле", range:
    { name: "order", role: "список выхода из рекурсии", range:
    ],
},
"scc-kosaraju": {
    vizTitle: "Компоненты сильной связности (Косарайю)",
    stepDriven: true,
    stepNote: "Шаг = вершина из order (в обратном порядке)",
    code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [], 4: [1, 3]}
adjT = {}
for v in adj:
    adjT[v] = []
for v in adj:
    for to in adj[v]:
        adjT[to].append(v)    # транспонирование

order = [3, 2, 1, 0, 4]       # фаза 1 уже дала порядок вершин
comp = {}                     # фаза 2: вершина → № SCC

for i in range(len(order) - 1, -1, -1):
    v = order[i]               # шаг: DFS по adjT из v кра

```

```

-----
is_bridge = low[to] > tin[v]          # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
    variables: [
        { name: "v", role: "текущая вершина DFS", range:
        { name: "to", role: "сосед; ребро (v, to) проверя
        { name: "tin[v]", role: "время входа в вершину –
        { name: "low[v]", role: "минимум tin, куда можно
        { name: "timer", role: "глобальный счётчик времени
    ],
},
"euler-path-vs-cycle": {
    vizTitle: "Эйлеров путь и цикл",
    stepNote: "Шаг = один клик по следующей вершине мар
    code: `edges = [(0,1), (0,2), (1,3), (2,4), (1,2),
deg = {}
for u, v in edges:          # степени вершин
    deg[u] = deg.get(u, 0) + 1
    deg[v] = deg.get(v, 0) + 1

odd = []
for v in deg:
    if deg[v] % 2:
        odd.append(v)
print(deg, "нечётных:", len(odd))
# 0 → цикл • 2 → путь • иначе – нельзя`,
    variables: [
        { name: "path", role: "текущий маршрут – последов
        { name: "last", role: "последняя вершина пути – о
        { name: "deg(v)", role: "степень вершины: чётная
    ],
},
"planarity-euler-formula": {
    vizTitle: "Планарность: K5 и K3,3",
    stepNote: "Шаг = попытка перетащить вершину так, что
    code: `V, E = 5, 10          # K5      (K3,3: V=
F = 2 - V + E                  # формула: V - E + F = 2

not planar = E > 3 * V - 6     # 10 > 9 → K5 непланарен

```

```

        { name: "w", role: "вес ребра (u, v)", range: "по",
        { name: "dist[v]", role: "лучшее известное расстояние",
    },
},
"bellman-ford": {
    vizTitle: "Форд–Беллман",
    stepDriven: true,
    stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
    code: `n, m = 7, 10 # вершин, рёбер
dist = {"S": 0}

for i in range(1, n): # итерация i = 1 ... n-1
    u, v, w = "S", "A", 5 # шаг: очередное ребро
    if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
        dist[v] = dist[u] + w # релаксация
    if i == 1:
        print(f"итерация {i}: {dist}")
        break`
    variables: [
        { name: "n", role: "число вершин графа – ровно n-1",
        { name: "m", role: "число рёбер, по которым проходят",
        { name: "i", role: "номер итерации (после i итераций)",
        { name: "u, v, w", role: "текущее ребро: откуда, куда, вес",
        { name: "dist[v]", role: "расстояние; если обновлено, то новое",
    ],
},
floyd: {
    vizTitle: "Флойд–Уоршелл",
    stepDriven: true,
    stepNote: "Шаг = инициализация матрицы, смена промежуточного узла",
    code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
      [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, INF, 0],
      [INF, INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, INF, INF, 0]]
n = 7
for k in range(n):
    print(f"посредник k={k}")
    for i in range(n):

```

```

-----
edges.sort()                # по весу

parent = {}                 # DSU: корень вершины
for v in "ABC":
    parent[v] = v
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

mst = []
for w, u, v in edges:      # шаг: очередное ребро
    if find(u) != find(v): # разные компоненты → берём
        parent[find(u)] = find(v)
        mst.append((w, u, v))
print(mst)\,
    variables: [
        { name: "n", role: "число вершин; в остов войдёт r",
        { name: "m", role: "число рёбер, сортируем по вес",
        { name: "i", role: "ребро сортируем по весу w; ра",
        { name: "parent[v]", role: "DSU: кто представител",
        { name: "rank[v]", role: "высота дерева DSU – по k",
    ],
},
"mst-prima": {
    vizTitle: "Прим",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

start = "A"
heap = [(0, start, "start")] # (вес, вершина, откуда)

w, v, parent = heapq.heappop(heap) # шаг: берём минимум
taken = {start}                  # дерево растёт от start
print(f"берём {v} за {w} (из {parent})")\,
    variables: [
        { name: "v", role: "вершина, которую только что д",
        { name: "u", role: "ещё не взятый сосед v – канди
```

```

        pi[i] = j
print(pi)`,
    variables: [
        { name: "n", role: "длина строки s, для которой с"},
        { name: "i", role: "индекс текущего символа – счи"},
        { name: "j", role: "длина текущего наилучшего сов"},
        { name: "π[i]", role: "префикс-функция: длина наи"},
            ица" },
    ],
},
"string-z-func": {
    vizTitle: "Z-функция",
    stepDriven: true,
    stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l",
    code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0                # правый Z-блок [l, r]
z = [None] * n                  # пустые клетки: значения p
z[0] = 0

for i in range(1, n):           # шаг: вычисляем z[i]
    if i <= r:
        z[i] = min(r - i + 1, z[i - l])    # из блока
    else:
        z[i] = 0                    # свежая клетка: с нуля
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1                # досчёт в лоб
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1
print(z)`,
    variables: [
        { name: "n", role: "длина строки s", range: "n = "},
        { name: "i", role: "позиция, для которой считаем"},
        { name: "l, r", role: "правый крайний Z-блок: отр"},
        { name: "z[i]", role: "длина наибольшего общего пр"},
    ],
},
"aho-corasick": {

```

```

-----
for k in range(1, 4):
    print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
    size = 9 - (1 << k)
    half = 1 << (k - 1)
    for r in range(size):
        for c in range(size):
            st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k - 1),
                                     + half, c + half, k - 1, k - 1]))`,
    stepCodeLines: [1, 8],
    variables: [
        { name: "r, c", role: "левый верхний угол блока, "},
        { name: "k", role: "уровень: блоки 2^k x 2^k (шаг" },
        { name: "st2[(r,c,k,k)]", role: "минимум квадратн" },
    ],
},
"sparse-table#2d-query": {
    vizTitle: "Разреженная таблица 2D: запрос",
    stepNote: "Демонстрация кликабельна вручную: выбери
    code: `# st2[(r, c, kx, ky)] из вкладки «построение:
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
    variables: [
        { name: "r1, c1, r2, c2", role: "углы запрашиваем" },
        { name: "kx, ky", role: "уровень самого крупного " },
    ],
},
"prefix-sums-2d#2d": {
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручную: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):

```



```

    if (!sync.code) return head;
    return `${head}\n${sync.code}\n`;
}

/**
 * Инициализация скелета: только объявление демо-данных
 * (без for/while/def/if/print). Именно это показывается
 * умолчанию – дальше пользователь пишет свой вариант,
 * подсматривает через кнопку-«глаз».
 */
function extractInit(code: string): string {
    const keep: string[] = [];
    for (const line of code.split("\n")) {
        const t = line.trim();
        if (/^(for|while|def|class|if\s|elif\s|else|try|exc
        keep.push(line);
    }
    // убрать пустые строки с конца
    while (keep.length > 0 && keep[keep.length - 1].trim()
    return keep.join("\n");
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО иници
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, ch
    const sync = getPageSync(chapterId, demoId);

    const head = sync.vizTitle
        ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#
          опкой-«глазом»\n`
        : `# «${chapterTitle}»\n# демо на странице нет – св

    if (!sync.code) return head;
    const init = extractInit(sync.code);
    return `${head}\n${init}\n`;

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Корпоративная пре
    </div>
    <p class="text-slate-300 text-sm mb
ы рассылать 10 000 писем, он пишет одно пис
счетах сотрудников только тогда, когда сотр
женное обновление).</p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Аналогия 2: Матрешки-коробки
    </div>
    <p class="text-slate-300 text-sm mb
ше, и так далее. Если нам нужно покрасить п
Внутри всё синее!" на большую коробку. Опус
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Строгое доказательство / Код (Скр
  </summary>
  <div class="p-5 text-sm text-slate-300"
    <pre class="bg-slate-950 p-4 rounder
function push(node) {
  if (lazy[node] !== 0) {
    lazy[2 * node] += lazy[node];
    tree[2 * node] += lazy[node];
    lazy[2 * node + 1] += lazy[node];
    tree[2 * node + 1] += lazy[node];
    lazy[node] = 0;
  }
}

```

```

        </div>
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #2d3748; border-radius: 0.5rem; padding: 1.5rem;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md" style="border: 2px solid #f47920; border-radius: 0.5rem; padding: 0.25rem;">
                Аналогия 2: Ступени
            </div>
            <p class="text-slate-300 text-sm mb-4">
                Чтобы покрыть отрезок, мы берем две самые бо
            </p>
        </div>
        <details class="bg-slate-900/40 rounded-lg border border-slate-700/50" style="border: 1px solid #2d3748; border-radius: 0.5rem; padding: 1.5rem;">
            <summary class="p-4 font-bold text-slate-300" style="padding: 0.5rem 0.5rem 0.5rem 0.5rem;">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300" style="padding: 0.5rem 0.5rem 0.5rem 0.5rem;">
                <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a202c; padding: 0.5rem 0.5rem 0.5rem 0.5rem;">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
            </div>
        </details>
    </div>
</div>
</section>`
    },
    {
        id: "treap",
        title: "3. Декартово дерево (Treap)",
        type: "html",
        description: "Дерево поиска по ключу и Куча по приоритету",
        category: "Продвинутые структуры",
        content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1 rounded">Treap</span>
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">Декартово дерево</h2>
    </div>
    <div class="space-y-8">

```



```

        <div class="space-y-4 text-sm text-slate-500">
          <div>
            <strong class="text-white block">
корень?</strong>
            <p>Оно поднимет узел, на котором
ска. За счет сдвига этого листа в корень, д
/p>
          </div>

          <div>
            <strong class="text-white block">
            <p>Если агент постоянно переключ
находящимися на разных концах дерева, его
ащая дерево в вытянутый бамбук для другого.
. Постоянное свинг-переключение превратит пр
            </div>

          <div>
            <strong class="text-white block">
            <p>Хотя один поиск может стоять
емах обладают прекрасным свойством: они не
по пути. "Палочное" дерево прессуется в сб
осов.</p>
          </div>
        </div>
      </div>
    </div>
  </section>`
  },
  {
    id: "prefix-sums-2d",
    title: "5. Двумерная матрица префиксных сумм",
    type: "html",
    description: "Сжатие суммы подматрицы за  $O(1)$ ",
    category: "Алгоритмы матрицы",
    content: `

```

```

-----
export const tickets14to20: Chapter[] = [
  {
    id: "dijkstra",
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">14
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Аналогия 1
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Вася хочет заработать денег, чтобы поехать в Европу.
        <p class="text-xl font-mono text-white">Вася: Я хочу поехать в Европу, но не умею программировать.
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Позитивное планирование
          </div>
          <p class="text-slate-300 text-sm mb-4">Вася: Я хочу поехать в Европу, но не умею программировать.
            (нельзя поехать и заработать). Он всегда в долгах.
          </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Ограничения
          </div>
          <p class="text-slate-300 text-sm mb-4">Вася: Я хочу поехать в Европу, но не умею программировать.
            Он не умеет пересматривать уже "зафиксированные" планы.
          </div>
      </div>
    </div>
  </div>
`
  }
]

```

```

        <p class="text-slate-300 text-sm mb-4">
опали во временную петлю.</p>
      </div>
    </div>
    <div id="slot-bellman-viz" class="mt-8"></div>
  </div>
</div>
</section>`
},
{
  id: "floyd",
  title: "16. Графы. Поиск кратчайшего пути. Флойд",
  type: "html",
  category: "Графы. Пути",
  content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>

    <div class="bg-slate-800 p-6 rounded-lg border-1 border-slate-700">
      <div class="absolute -top-3 left-4 bg-slate-900
border-emerald-500 shadow-md">
        Суть фокуса (По шагам)
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Главный герой – <b>внешний цикл</b>
        шаг <code class="bg-slate-900 text-pink-400">
        шу себе проходить через вершину k?</i>
      </p>

```

```

        <p class="text-slate-300 text-sm mb-6">
            Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать Джонсона по пути остались теми же.
        </p>
        <ol class="text-sm text-slate-300 space-y-4">
            <li>Добавляем фиктивную вершину
            <li>Запускаем от неё медленно алгоритм
            <li>Добавляем «потенциалы» <code class="text-pink-400">
                <li>Меняем старые веса: новое равенство
                <li><b>Магия!</b> Все веса теперь равны
                <li>Теперь смело запускаем быстро
            </li>
        </ol>
    </div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-emerald-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-emerald-300 italic">
                    <p class="text-xl font-mono text-white">

```



```

<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
    * Аналогия: Заражение вирусом
</div>
<p class="text-slate-300 text-sm mb
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
</div>
</div>
</div>
</section>`
},
{
  id: "mst-boruvka",
  title: "20. Графы. Остовное дерево. Борувка",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-emerald-4
      <div class="bg-slate-900 p-4 rounded-lg mb-1
        <p class="text-lg text-emerald-300 ital
        <p class="text-xl font-mono text-white"
ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg
          <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
              Аналогия: Племена
            </div>

```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
```

```
  <!-- Аналогия 1 -->
```

```
  <div class="bg-slate-800 p-6 rounded-lg
```

```
    <div class="absolute -top-3 left-4 l  
  rder border-emerald-500 shadow-md">
```

```
    ♂ Аналогия 1: Поиск без отка
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb
```

```
    Мы идём по строке слева направо. К  
строки СЕЙЧАС закончился под моим пальцем?»
```

```
    Представь, ты ищешь в тексте слов  
х</code>. Тупой алгоритм начнет искать зано  
</code>, а это начало нашего слова! Не надо
```

```
  </p>
```

```
</div>
```

```
  <!-- Аналогия 2 -->
```

```
  <div class="bg-slate-900 p-6 rounded-lg
```

```
    <div class="absolute -top-3 left-4 l  
border-rose-500 shadow-md">
```

```
    Аналогия 2: LLM стриминг
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb
```

```
    Для LLM, которая стримит токены  
каждом шаге. Если мы частично совпали с зап  
акого места этого слова мы можем продолжить
```

```
  </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```
  <summary class="p-4 font-bold text-slate  
-b border-slate-700/50">
```

```
    Пример вычисления (Скрыто)
```

```
  </summary>
```

```
  <div class="p-5 text-sm text-slate-300
```



```

    if (i &lt;= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] &lt; n && s[z[i]] == s[i + z[i]]) z
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

    </div>

```

```

    </details>

```

```

    </div>

```

```

    </div>

```

```

</section>`

```

```

},

```

```

{

```

```

    id: "aho-corasick",

```

```

    title: "23. Алгоритм Ахо-Корасик",

```

```

    type: "html",

```

```

    category: "Строки",

```

```

    content: `

```

```

<section id="aho-corasick" class="mb-12 scroll-mt-10">

```

```

    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

        <span class="bg-indigo-600 text-white px-4 py-1 rounded">

```

```

        <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

    </div>

```

```

<div class="space-y-8">

```

```

    <div class="bg-slate-700/50 p-6 rounded-xl border-l">

```

```

        <h3 class="text-xl font-bold text-blue-400 mb-4">

```

```

        <div class="bg-slate-900 p-4 rounded-lg mb-6 text">

```

```

            <p class="text-sm text-blue-300 italic mb-2">Осн

```

```

            <p class="text-lg font-mono text-white">Бор (Tr

```

```

        </div>

```

```

        <p class="text-slate-300 text-sm">Позволяет найти

```

```

            го один проход.</p>

```

```

    </div>

```

```

<!-- Аналогии -->

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">

```

```

    <div class="bg-slate-800 p-6 rounded-lg border bo

```

```

        <div class="absolute -top-3 left-4 bg-slate-700

```

```

} </pre>
    </div>
  </details>
</div>
</section>`
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-purple-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">
        <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
            order border-emerald-500 shadow-md">
              Аналогия 1: Судoku (P vs NP)
            </div>
            <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
          </div>
          <!-- Аналогия 2 -->
          <div class="bg-slate-900 p-6 rounded-lg

```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
  <h3 class="text-xl font-bold text-slate-300">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-slate-300 italic">
        <p class="text-xl font-mono text-white">
          (V + E).</p>
        </p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Матрица = Таблица
          </div>
          <p class="text-slate-300 text-sm mb-4">
            глупо, но проверка "знает ли Вася Петю" мгновенно.
          </p>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Аналогия: Списки = Контакты
          </div>
          <p class="text-slate-300 text-sm mb-4">
            т. Оптимально для почти всех задач.</p>
        </div>
      </div>
    </div>
  </div>
</div>
```

```
{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
  <h3 class="text-xl font-bold text-indigo-400">
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия DFS -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
          ♂ DFS (Поиск в глубину) = Ж
        </div>
      </div>
    </div>
  </div>
```

```

</summary>
<div class="grid grid-cols-1 md:grid-co
    <pre class="bg-slate-950 p-3 rounde
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}
</pre>
    <pre class="bg-slate-950 p-3 rounde
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
</pre>
    </div>
</details>
</div>
</section>`,
    },
    {
        id: "graph-planar-colors",
        title: "7. Графы. Планарные. Покраска",
        type: "html",
    }

```

```

<section id="graph-components" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ество компонент.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия 1: Острова
          </div>
          <p class="text-slate-300 text-sm mb">
            карту – остались ли серые (неисследованные)
            ь через океан – столько и компонент.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">

```



```

        <div class="bg-slate-900 p-6 rounded-lg"
          <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
            Связь с Билетом 12
          </div>
          <p class="text-slate-300 text-sm mb"
            <b>SCC (Билет 10)</b> склеивает
        </br><br>
            <b>Точки сочленения (Билет 12)</b>
        ость</strong> монолита. Вырвешь точку сочле
          </p>
        </div>
      </div>

      <details class="bg-slate-900/40 rounded-lg"
        <summary class="p-4 font-bold text-slate-
        -b border-slate-700/50">
          Душный мод: Код Алгоритм Косарайю
        </summary>
        <div class="p-5 text-sm text-slate-300"
          <p class="text-emerald-400 font-bol"
          <ol class="list-decimal list-inside"
            <li>Запускаем DFS на исходном г
          </li>
            <li>Разворачиваем этот список (
            <li>Переворачиваем все ребра в
            <li>Идем по <code>order</code>:
          !</li>
            </ol>
          </div>
        </details>
      </div>
    </div>
  </section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",

```

```
-----
<section id="graph-articulation" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3"
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-rose-400 m

      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-rose-300 italic m
        <div class="text-left font-mono text-sm
          <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
        <div class="p-3 bg-slate-950 rounde
          <span class="text-rose-400 font
          <p class="text-xl font-bold tex
          <p class="text-xs text-slate-40
        </div>
      </div>
    </div>
    <div class="p-3 bg-slate-950 rounde
      <p class="text-slate-300 text-sm">
        <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong>
ег - это тупик со степенью 1).
      </p>
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap
      <div class="bg-slate-800 p-6 rounded-lg bord
        <div class="absolute -top-3 left-4 bg-s
        <div class="text-rose-500 shadow-md">
          Аналогия: Главный перекресток
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Представь город, где два района сое
```

```
if (visited[to]) {  
    low[v] = min(low[v], tin[to]);  
} else {  
    dfs(to, v);  
    low[v] = min(low[v], low[to]);  
    if (low[to] >= tin[v] && p != -1)  
        IS_CUTPOINT(v);  
    ++children;  
}  
  
}  
if (p == -1 && children > 1)  
    IS_CUTPOINT(v);  
}</pre>
```

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border

class="text-lq font-bold text-indigo-400">

<p class="text-slate-300 text-sm mb-4">

Это глубокий дуальный мост между ориент

</p>

- **Navigation:**
 - **Home:** The starting point of the application.
 - **Products:** A list of all available products.
 - **Categories:** A list of product categories.
 - **Search:** A search bar to find products by name or category.
 - **Cart:** A view showing the items added to the shopping cart.
 - **Checkout:** The final step in the purchasing process.
 - **Account:** A section for user authentication and profile management.

- Точки**

определяют физическую уязвимость свя.

```
<li><strong class="text-emerald-400">SC
```

- зависимые "конгломераты".

- Как точка сочленения рушит**

Косарайю, свернув каждую SCC в одну супер-в

(сделаем неориентированным), то любая <stro

о и есть критическая SCC! Если удалить эту

Одно слабое звено изолирует целые касты S

</section>`

1.

Jf

=====

ФАЙЛ 30/87: src/components/ArticulationPointsViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
}
```

```
interface Edge {
  u: number;
  v: number;
}
```

```
const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
  { id: 1, x: 100, y: 150, label: "1" },
  { id: 2, x: 250, y: 250, label: "2" },
  { id: 3, x: 400, y: 150, label: "3" },
  { id: 4, x: 550, y: 150, label: "4" },
  { id: 5, x: 700, y: 50, label: "5" },
  { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
```

```
const recordStep = (desc: string, u: number | null) => {
  newSteps.push({
    desc,
    visited: new Set(visited),
    ap: new Set(ap),
    currentNode: u,
    tin: [...currentTin],
    low: [...currentLow],
  });
};
```

recordStep("Начало DFS (Тарьян) для поиска точек со

```
const dfs = (v: number, p: number = -1) => {
  visited.add(v);
  currentTin[v] = currentLow[v] = timer++;
  let children = 0;

  recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);

  for (const to of adj[v]) {
    if (to === p) continue;

    if (visited.has(to)) {
      currentLow[v] = Math.min(currentLow[v], currentLow[to]);
      recordStep(
        `Обратное ребро ${v}-${to}. Обновляем low[${v}]`,
        v,
      );
    } else {
      children++;
      dfs(to, v);
      currentLow[v] = Math.min(currentLow[v], currentLow[to]);

      recordStep(
        `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
        v,
      );
    }
  }
};
```

```

    visited: new Set(),
    ap: new Set(),
    currentNode: null,
    tin: [],
    low: [],
  };

  useEffect(() => {
    let interval: NodeJS.Timeout;
    if (isPlaying && currentStepIndex < steps.length - 1) {
      interval = setInterval(() => {
        setCurrentStepIndex((prev) => prev + 1);
      }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearInterval(interval);
  }, [isPlaying, currentStepIndex, steps.length]);

  const togglePlay = () => setIsPlaying(!isPlaying);
  const reset = () => {
    setIsPlaying(false);
    setCurrentStepIndex(0);
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-
      <div className="bg-slate-800 p-4 border-b border-
        <h3 className="font-bold text-white flex items-
          <Info className="w-5 h-5 text-rose-400" />
          Моделирование поиска точек сочленения
        </h3>

        <div className="flex items-center gap-2 bg-slate-
          <button
            onClick={togglePlay}
            className="flex items-center gap-2 px-3 py-
            text-white"

```

```

        stroke={isBridge ? "#f43f5e" : "#475568"}
        strokeWidth={isBridge ? "4" : "2"}
        strokeDasharray={isBridge ? "8,4" : "0"}
      />
    );
  }}}

```

```

{/* Nodes */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id;
  const isVisited = currentStep.visited.has(node.id);
  const isAp = currentStep.ap.has(node.id);

```

```

    const fill = isCurrent
      ? "#3b82f6"
      : isAp
        ? "#e11d48"
        : isVisited
          ? "#10b981"
          : "#1e293b";
    const stroke = isCurrent
      ? "#60a5fa"
      : isAp
        ? "#f43f5e"
        : isVisited
          ? "#34d399"
          : "#334155";
    const r = isAp ? 24 : 20;

```

```

    return (
      <g key={node.id}>
        <circle
          cx={node.x}
          cy={node.y}
          r={r}
          fill={fill}
          stroke={stroke}
          strokeWidth="3"

```



```

                ? currentStep.low[node.id]
                : "-"}
            </text>
        </>
    })
  </g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-4
    Статус алгоритма
  </h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-white font-medium min-h-
      {currentStep.desc}
    </p>
  </div>

  <div className="space-y-4 flex-1 overflow-y-a
    <div className="mb-4">
      <h5 className="text-xs text-slate-500 fon
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Точка сочленения
        </div>
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Посещена
        </div>
      <div className="flex items-center gap-2 t
        <div className="w-3 h-3 rounded-full bg
          Текущая
        </div>
      <div className="flex items-center gap-2 t

```

```
    });  
  };
```

ФАЙЛ 31/87: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';  
import { useVizStepSync } from '../data/vizStepBus';  
  
interface Node { id: string; x: number; y: number; name: string; }  
interface Edge { u: string; v: string; w: number; }  
interface Step {  
  iteration: number;  
  checkingEdge: { u: string; v: string; w: number } | null;  
  distances: { [key: string]: number };  
  previous: { [key: string]: string | null };  
  log: string;  
  hasNegativeCycle?: boolean;  
  activeCodeLine: number;  
}  
  
export default function BellmanFordViz() {  
  const [hasCycle, setHasCycle] = useState(false);  
  const [steps, setSteps] = useState<Step[]>([]);  
  const [currentStepIndex, setCurrentStepIndex] = useState(0);  
  useVizStepSync(currentStepIndex, setCurrentStepIndex, setSteps);  
  const [isPlaying, setIsPlaying] = useState(false);  
  
  const nodes: Node[] = [  
    { id: 'S', x: 60, y: 150, name: 'База (S)' },  
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },  
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },  
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },  
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },  
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },  
  ];
```

```

    { line: 7, text: '      for (u, v, weight) in E: // Пройдем по всем ребрам
    { line: 8, text: '          if dist[u] + weight < dist[v]:
    { line: 9, text: '              return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"
    { line: 10, text: '      return dist' },
  ];

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  let dist: { [key: string]: number } = { S: 0, A: 999 };
  let prev: { [key: string]: string | null } = { S: null };

```

```

  simulationSteps.push({
    iteration: 0, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: '    Фура заведена. Начинаем с Базы (S). Расстояние до А: 999
    activeCodeLine: 2,
  });

```

```

  const vCount = nodes.length;
  for (let iter = 1; iter < vCount; iter++) {
    let anyUpdate = false;
    for (const edge of edges) {
      const uDist = dist[edge.u];
      const vDist = dist[edge.v];

      let updated = false;
      if (uDist !== 999 && uDist + edge.w < vDist) {
        dist = { ...dist, [edge.v]: uDist + edge.w };
        prev = { ...prev, [edge.v]: edge.u };
        updated = true;
        anyUpdate = true;
      }

```

```

    simulationSteps.push({
      iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
      distances: { ...dist }, previous: { ...prev },
      log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
        updated ? ` Релаксация успешна! dist[${edge.v}] = ${dist[edge.v]}

```

```

    }

    setSteps(simulationSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
  }, [hasCycle]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Зарядка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border" >
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center gap-3" >
        <div className="p-2.5 bg-blue-600 text-white" >
          <span className="text-2xl" > </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white" >
            <p className="text-sm text-blue-300" >Полный
          </p>
        </div>
      </div>
      <div>
        <div className="flex items-center gap-3" >
          <div className="bg-slate-900 p-1 rounded-lg border" >
            <button onClick={() => setHasCycle(false)} >
              'bg-blue-600 text-white' : 'text-slate-400'
            <button onClick={() => setHasCycle(true)} >

```

```

    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id === u);
      const vNode = nodes.find((n) => n.id === v);
      const isChecking = currentStep.checkingEdges;

      const isNeg = edge.w < 0;
      const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

      let strokeColor = '#475569';
      let marker = 'url(#arrow-bf-normal)';
      if (isChecking) { strokeColor = '#f59e0b'; }
      else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#f43f5e'; }

      return (
        <g key={idx}>
          <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={
            isChecking && isCyclePart ? 4 : 2} marker={marker} />
          <circle cx={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} stroke={strokeColor} strokeWidth={
            isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} />
          <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} text={edge.w} fill={strokeColor} fontSize="11" fontWeight="bold" />
        </g>
      );
    })
  }

  {nodes.map((node) => {
    const dist = currentStep.distances[node.id];
    const isCheckingNode = currentStep.checkingNodes;

    return (
      <g key={node.id} className="transition-all duration-200">
        <circle cx={node.x} cy={node.y} r={isCheckingNode ? 10 : 5} fill={isCheckingNode ? '#7dd3fc' : '#64748b'} stroke={strokeColor} strokeWidth={1} />
        <text x={node.x} y={node.y + 5} fill={strokeColor} />
        <text x={node.x} y={node.y - 28} fill={strokeColor} />
      </g>
    );
  })
}

```

```

    { /* Таблица расстояний */ }
    <div className="w-full lg:w-1/2 bg-rose-950/10" >
      <div>
        <h4 className="text-lg font-bold text-rose-950" >
          <div className="grid grid-cols-4 gap-2 text-rose-950" >
            {nodes.map(n => {
              const d = currentStep.distances[n.id];
              const isTgt = currentStep.checkingEdge === n.id;
              return (
                <div key={n.id} className={`p-2 rounded-lg border-slate-700/60 bg-slate-800/40`} >
                  <div className="text-xs text-slate-400" >
                    <div className={`font-mono font-bold`} >
                      {d}
                    </div>
                  </div>
                </div>
              );
            })}
          </div>
        </div>
      </div>
    </div>

    { /* Код алгоритма */ }
    <div className="w-full lg:w-1/2 bg-slate-900/60" >
      <div>
        <h4 className="text-lg font-bold text-slate-400" >
          <div className="bg-slate-950/80 rounded-lg p-4" >
            {codeLines.map(item => {
              <div key={item.line} className={`py-1.5`} >
                {item.e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'}
                <span className="text-slate-600 w-5" >
                  <span className="whitespace-pre flex" >
                    {item.code}
                  </span>
                </div>
              </div>
            })}
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

```
    logs: string;
}

const generateBfsSteps = (): Step[] => {
    const steps: Step[] = [];
    const queue: number[] = [];
    const visited = new Set<number>();

    // Init
    steps.push({
        activeLine: 1,
        queue: [],
        visited: [],
        currentNode: null,
        logs: "Инициализация: начинаем с корня (0).",
    });

    queue.push(0);
    visited.add(0);
    steps.push({
        activeLine: 2,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: null,
        logs: "Добавляем стартовую вершину 0 в очередь и по",
    });

    while (queue.length > 0) {
        steps.push({
            activeLine: 4,
            queue: [...queue],
            visited: Array.from(visited),
            currentNode: null,
            logs: `Очередь не пуста, элементов: ${queue.length}`,
        });

        const curr = queue.shift()!;
        steps.push({
```

```

        });
    }
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);
  useVizStepSync(stepIdx, setStepIdx, STEPS.length - 1)

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length
  }, []));

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);

  useEffect(() => {
    if (!isPlaying) return;
    if (stepIdx >= STEPS.length - 1) {
      setIsPlaying(false);
      return;
    }
  }, [stepIdx, isPlaying]);
}

```



```

    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-2 gap-4" >
    {/** Визуал графа */}
    <div className="bg-slate-800 border border-slate-600 border-radius-50px]" >
      <svg className="w-full h-full min-h-[350px]" >
        {/** Рёбра */}
        {EDGES.map((e, idx) => {
          const n1 = NODES.find(n => n.id === e.from)
          const n2 = NODES.find(n => n.id === e.to)
          const isVisited = step.visited.includes(n1.id)
          return (
            <line
              key={idx}
              x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
              className={`transition-all duration-500 ${
                isVisited ? 'stroke-slate-600' : 'stroke-blue-400'
              }`}
            />
          )
        })}
      </svg>
      {/** Вершины */}
      {NODES.map(n => {
        const isCurrent = step.currentNode === n.id
        const isInQueue = step.queue.includes(n.id)
        const isVisited = step.visited.includes(n.id)

        let fill = '#1e293b'; // slate-800
        let stroke = '#475569'; // slate-600
        let scale = 1;

        void scale;

        if (isCurrent) {
          fill = '#3b82f6'; // blue-500
          stroke = '#60a5fa'; // blue-400
          scale = 1.3;
        }
      })}
    </div>
  </div>

```

```

<div className="bg-slate-900 border border-slate-800" style={{ padding: 10px }}>
  <div className="bg-slate-800 text-slate-400 p-4" style={{ border: 1px solid #334d5b, border-radius: 5px, margin: 10px 0 10px 10px }}>
    <span>bfs(graph, start)</span>
    <span className="text-blue-400">Mar {step}</span>
  </div>
  <div className="p-4">
    {
      { line: 1, code: 'queue = Queue()' },
      { line: 2, code: 'queue.push(start); visited.add(start)' },
      { line: 3, code: '' },
      { line: 4, code: 'while not queue.isEmpty():' },
      { line: 5, code: '    node = queue.pop()' },
      { line: 6, code: '    for neighbor in graph[node]:' },
      { line: 7, code: '        if neighbor not in visited:' },
      { line: 8, code: '            visited.add(neighbor)' },
      { line: 9, code: '            queue.push(neighbor)' },
    }.map((cl) => {
      const isActive = step.activeLine === cl.line
      return (
        <div key={cl.line} className={`px-2 py-2 border-l-2 border-blue-400 -ml-0.5` : `text-slate-400`}>
          <span className="opacity-40 w-6 inline-block"> </span>
          <span className="whitespace-pre">{cl.code}</span>
        </div>
      )
    })
  </div>
</div>

<div className="bg-slate-800/80 border border-slate-700 p-4" style={{ border: 1px solid #334d5b, border-radius: 5px, margin: 10px 0 10px 10px }}>
  <div className="text-sm font-bold text-emerald-400">
    <span>Состояние Очереди (Queue):</span>
    <span className="text-xs text-slate-500">{step.queue.length}</span>
  </div>
  <div className="flex items-center gap-2 mt-2">
    {step.queue.length === 0 ? (

```

```
    },
    'bellman-ford': {
      src: bellmanImg,
      alt: 'Фура по болоту',
      caption: ' Тяжёлая, но ей пофиг на ямы!',
      borderColor: 'border-rose-500/30',
      captionColor: 'text-rose-400',
    },
    floyd: {
      src: floydImg,
      alt: 'Все ко Всем Флойд',
      caption: ' Все связаны со всеми!',
      borderColor: 'border-emerald-500/30',
      captionColor: 'text-emerald-400',
    },
  };

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-rose-500`} >
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold`} >{info.caption}</p>
    </div>
  );
}
```

ФАЙЛ 34/87: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React from "react";
```

```

        :bg-slate-700 enabled:hover:text-white disabled:opacity-50
      >
      <span className="hidden sm:inline">Вперёд</span>
      <ChevronRight className="w-4 h-4" />
    </button>
  </div>
);
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-slate-700 transition-all"
      >
        <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
          <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
        </div>
        <div className="text-sm font-bold text-slate-100">
          {prev.title}
        </div>
      </button>
    ) : (
      <div className="hidden sm:block" />
    )}
    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-slate-700 transition-all sm:col-start-2"
      >
        <div className="flex items-center justify-end text-indigo-400 mb-1">
          Следующая тема <ChevronRight className="w-3.5 h-3.5" />
        </div>
        <div className="text-sm font-bold text-slate-100">
          {next.title}
        </div>
      </button>
    )}
  </nav>
);
}

```

```

const [anchor, setAnchor] = useState<HintAnchor | null>(null);
const hideTimer = useRef<number | null>(null);
const [saving, setSaving] = useState<"idle" | "work">("idle");

useTermHints(contentRef, [chapter.id]);

const cancelHide = useCallback(() => {
  if (hideTimer.current) {
    window.clearTimeout(hideTimer.current);
    hideTimer.current = null;
  }
}, []);

const scheduleHide = useCallback(() => {
  cancelHide();
  hideTimer.current = window.setTimeout(() => setAnchor(anchor), 2500);
}, [cancelHide]);

useEffect(() => {
  setAnchor(null);
  setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
  setSaving("work");
  try {
    await downloadChapterHtml(chapter);
    setSaving("done");
    window.setTimeout(() => setSaving("idle"), 2500);
  } catch {
    setSaving("idle");
  }
}, [chapter]);

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
  },
  []
);

```

```

<button
  onClick={saveHtml}
  disabled={saving === "work"}
  title="Сохранить эту тему одним автономным файлом"
  className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-emerald-500"
>
  {saving === "work" ? (
    <Loader2 className="w-3.5 h-3.5 animate-spin" />
  ) : saving === "done" ? (
    <Check className="w-3.5 h-3.5 text-emerald-500" />
  ) : (
    <Download className="w-3.5 h-3.5" />
  )}
  {saving === "done" ? "Файл сохранён" : "Скачать"}
</button>
<button
  onClick={() => window.print()}
  title="Распечатать или сохранить в PDF средствами браузера"
  className="flex items-center gap-1.5 text-x-300 hover:text-white hover:border-indigo-500"
>
  <Printer className="w-3.5 h-3.5" /> Печать
</button>
<span className="text-[11px] text-slate-500">
  </div>

<div
  ref={contentRef}
  className="prose prose-invert max-w-none"
  onMouseOver={handlePointerOver}
  onMouseOut={handlePointerOut}
  onClick={handleClick}
  onFocus={handleFocus}
  dangerouslySetInnerHTML={{ __html: chapter.content }}
/>

<p className="mt-6 flex items-start gap-2 text-slate-500">

```

```

        <viz.Component />
      </VizChapterContext.Provider>
    </section>
  )}

  <ChapterNav prev={prev} next={next} index={index} />
</article>

<TermPopover
  anchor={anchor}
  onClose={() => setAnchor(null)}
  onOpenSimulator={(id) => {
    setAnchor(null);
    onOpenSimulator(id);
  }}
  onCardEnter={cancelHide}
  onCardLeave={scheduleHide}
/>
</>
);
};

```

ФАЙЛ 36/87: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Undo, Play, Pause, SkipForward, RefreshCw } from 'react-native';

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [

```

```

    { id: 3, label: "D", x: 180, y: 300 },
    { id: 4, label: "E", x: 100, y: 370 },
    { id: 5, label: "F", x: 260, y: 370 },
    { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 6, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: [1,6], backEdge: null,
    log: "Из B идём в G. tin[G]=3, low[G]=3.",
    activeCodeLine: [4,5,7,10,11]
  }
];

```



```

tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: [4,5], backEdge: null,
log: "E→F. tin[F]=7, low[F]=7.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: [5,3],
log: "Из F видим D (посещён). Обратное ребро (F,D).
activeCodeLine: [7,8,9]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо
activeCodeLine: [11,13,16,17]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо
activeCodeLine: [11,13,16,17]
},
{
currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) –
activeCodeLine: [13,14,15,16,17]
},
{
currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6
activeEdge: null, backEdge: null,
log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,

```

```

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <div className="flex items-center gap-1 bg-slate-900 p-1.5">
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
            disabled={dfsIdx === 0}
            className="p-1.5 hover:bg-slate-700 rounded"
            <Undo className="h-3.5 w-3.5" />
          </button>
          <button onClick={() => setDfsAuto(!dfsAuto)}
            className={`px-2.5 py-1 rounded text-[10px] ${
              dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
            }`}
            <{dfsAuto ? "<>" : "<"}><Pause className="h-3 w-3" />
          </button>
          <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length-1); }}
            disabled={dfsIdx === DFS_STEPS.length-1}
            className="p-1.5 hover:bg-slate-700 rounded"
            <SkipForward className="h-3.5 w-3.5" />
          </button>
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
            className="p-1.5 hover:bg-slate-700 rounded"
            <RefreshCw className="h-3.5 w-3.5" />
          </button>
        </div>
      </div>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-5 gap-4">
      <div className="md:col-span-3 bg-slate-950 rounded p-4">
        <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)]"
          <div>
            <svg className="w-full h-[280px] z-10 relative">
              {GRAPH_EDGES.map(e => {
                const u = GRAPH_NODES.find(n => n.id === e.u);
                const v = GRAPH_NODES.find(n => n.id === e.v);
                const bridge = step.bridges.includes(e.key);
              })}
            </svg>
          </div>
        </div>
      </div>
    </div>
  </div>
)

```

```

        "transition-all duration-300" />
        <text x={n.x} y={n.y-20} textAnchor="left">
        { l:{step.low[n.id]|| "?"}}</text>
      </>
    ))
  </g>;
  )))
</svg>
<div className="flex items-center justify-between">
  <span>Step {dfsIdx+1}/{DFS_STEPS.length}</span>
  <div className="flex-1 mx-2 bg-slate-950 h-12 rounded-lg">
    <div className="bg-indigo-600 h-full transition-all duration-300">
    </div>
  </div>
</div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
  {/* Code panel */}
  <div className="bg-slate-950 rounded-lg border border-slate-800">
    <div className="bg-slate-900 px-3 py-1.5 text-slate-400 font-mono">
      <pre>
        {CODE_LINES.map((cl, i) => {
          const hl = step.activeCodeLine.includes(cl.line)
          return <div key={i}
            className={`px-1 transition-all duration-300
              ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-600"}
            `}>
          <span className="text-slate-600 mr-1">{cl.line}
        })}
      </pre>
    </div>
  </div>
</div>

```

ФАЙЛ 37/87: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
  ];
```

```

const prev: Record<string, string | null> = { A: null };

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ` Дейкстра готов к доставке. Начинаем из ПИ.`,
  activeCodeLine: 2,
});

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }

  if (!u) break;

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ` Выбираем вершину ${u} с мин. непосещённой`,
    activeCodeLine: 5,
  });

  const neighbors = edges.filter(e => e.u === u);
  for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
      currentNode: u, checkingEdge: { u, v },
      distances: { ...dist }, visited: { ...visited },
      log: ` Смотрим соседа ${v}. Текущее расстояние: ${dist[u]} + ${edge.w} = ${dist[v]}`,
      activeCodeLine: 6,
    });
  }
}

```

```

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border
    { /* Шапка симулятора */ }
    <div className="flex flex-wrap items-center justify
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white
          <span className="text-2xl"> </span>
        </div>
      <div>
        <h3 className="text-2xl font-bold text-white
          <p className="text-sm text-indigo-300">Полн
        </div>
      </div>

    <div className="flex items-center gap-2">
      <button
        onClick={() => { setIsPlaying(false); setCu
        className="flex items-center gap-1 bg-slate
        ion-colors"
      >
        Сброс
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1 px-4 py
          isPlaying ? 'bg-amber-600 hover:bg-amber-!
        }`}
      >
        {isPlaying ? ' Пауза' : '▶ Пуск'}
      </button>
      <button
        onClick={() => { setIsPlaying(false); if (c
        disabled={currentStepIndex >= steps.length

```

```

    let marker = 'url(#arrow-dh-normal)';
    if (isChecking) marker = 'url(#arrow-dh-a
    else if (isOptimal) marker = 'url(#arrow-

    return (
      <g key={idx}>
        <line
          x1={uNode.x} y1={uNode.y}
          x2={vNode.x} y2={vNode.y}
          stroke={isChecking ? '#f59e0b' : is
          strokeWidth={isChecking ? 5 : isOpt
          markerEnd={marker}
          className="transition-all duration-
          strokeDasharray={isChecking ? '4,4'
        />
        <circle
          cx={({uNode.x + vNode.x} / 2} cy={({u
          r="12" fill="#1e293b"
          stroke={isChecking ? '#f59e0b' : '#
        />
        <text
          x={({uNode.x + vNode.x} / 2} y={({uNo
          fill={isChecking ? '#f59e0b' : '#f8
          fontSize="11" fontWeight="bold" tex
        >
          {edge.w}
        </text>
      </g>
    );
  }}}

  {/* Вершины */}
  {nodes.map((node) => {
    const isCurrent = currentStep.currentNode
    const isVisited = currentStep.visited[node.i
    const dist = currentStep.distances[node.i

```

```

        isCurrentNode ? 'bg-emerald-900/60' :
        'text-slate-300'
      }`}>
      <div className="flex items-center gap-2">
        <span className={`px-2 py-0.5 rounded-sm text-slate-300`} `}>
          {u}
        </span>
        <span className="text-slate-400">→</span>
      </div>
      <div className="flex flex-wrap gap-2">
        {adjList[u].map((edge, idx) => {
          const isCheckingThis = currentStep === idx
          return (
            <span key={idx} className={`px-2 py-0.5 rounded-sm
              isCheckingThis ? 'bg-amber-500 text-slate-300' :
              'text-slate-300'`} `}>
              {edge.v}
            <span>{edge.v}</span><span className="font-size-2">
              <span className={isCheckingThis ? 'bg-amber-500' : 'bg-emerald-900/60'}>
                {edge.w}</span>
              </span>
            </span>
          )
        })}
      </div>
    </div>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10 p-6">
    <div>
      <h4 className="text-lg font-bold text-rose-50">Таблица расстояний
      </h4>
      <div className="overflow-x-auto">

```



```

    });

    if (n in memoMap) {
        generatedSteps.push({
            id: stepId++,
            n,
            action: 'memo_hit',
            desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано`,
            codeLine: 2,
            memoState: { ...memoMap }
        });
        return memoMap[n];
    }

    if (n <= 2) {
        generatedSteps.push({
            id: stepId++,
            n,
            action: 'base_case',
            desc: `Базовый случай: n <= 2 (n=${n}). Возвр`,
            codeLine: 3,
            memoState: { ...memoMap }
        });
        return 1;
    }

    generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вычисляем рекурсивно: fibМемо(${n - 1})`,
        codeLine: 4,
        memoState: { ...memoMap }
    });

    const res1 = simulateFib(n - 1);
    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

```

```

    return () => clearTimeout(timer);
  }, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
  <div className="bg-slate-900 rounded-2xl border border-slate-800" >
    { /* Header & Controls */ }
    <div className="flex flex-col lg:flex-row lg:items-center" >
      <div>
        <div className="flex items-center gap-3 mb-2" >
          <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
            <RefreshCw className="w-6 h-6" />
          </div>
          <h3 className="text-2xl font-extrabold text-white" >
            DP
          </h3>
          <p className="text-sm text-slate-400" >
            Смотри, как таблица DP кэширует результаты
          </p>
        </div>

        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center gap-2 bg-slate-800 p-2" >
            <span>N:</span>
            <select
              value={targetN}
              onChange={(e) => setTargetN(Number(e.target.value))}
              disabled={isPlaying}
              className="bg-transparent text-white font-mono" >
              <option value={4}>N = 4</option>
              <option value={5}>N = 5</option>
              <option value={6}>N = 6</option>
              <option value={7}>N = 7</option>
            </select>
          </div>
        </div>
      </div>
    </div>
  )

```

```

    { /* DP Table View */ }
    <div className="mb-8 bg-slate-950 p-4 md:p-6 rounded"
      <h4 className="text-xs font-bold uppercase tracking-wider"
        <div className="flex flex-wrap items-center gap-2"
          {Array.from({ length: targetN }, (_, i) => i)}
          const isCached = num in memoState || num <= 2;
          const val = num <= 2 ? 1 : memoState[num];
          const isCurrent = currentStep?.n === num;

          return (
            <div
              key={num}
              className={`flex flex-col items-center justify-center gap-2
                ${isCurrent
                  ? 'bg-emerald-950 border-emerald-500'
                  : isCached
                  ? 'bg-slate-900 border-emerald-500/50'
                  : 'bg-slate-900/50 border-slate-800'}
              `}
            >
              <span className="text-xs text-slate-400"
                <span className={`text-xl md:text-2xl font-medium`}
                  {isCached ? val : '0'}
                </span>
                {num <= 2 && <span className="text-[9px] font-mono"
                  </div>
            </div>
          );
        })))
    </div>

    {currentStep && (
      <div className="mt-6 pt-4 border-t border-slate-800"
        <div className="text-sm font-medium text-slate-400"
          <span className="inline-block w-2 h-2 rounded-full"
            <span>{currentStep.desc}</span>
          </div>
          <div className="text-xs font-mono bg-slate-900"

```

```

</h5>
<p className="text-sm text-slate-400">
  В классической рекурсии для  $N=\{targetN\}$  по
  вычисленное значение в таблице, мы момента.
</p>
<div className="p-4 bg-slate-900 rounded-xl
  text-slate-300">
  <span>Всего шагов симуляции: {steps.length}</span>
  <span className="text-emerald-400 font-bo
</div>
</div>
)}

{activeTab === 'code' && (
  <div className="bg-slate-950 rounded-2xl border
    <div className="bg-slate-900 px-6 py-3 border
      <span className="text-xs font-bold font-m
      <span className="text-xs text-emerald-400
    </div>
    <pre className="p-6 font-mono text-sm space
      {DP_CODE_LINES.map((line, idx) => {
        const lineNum = idx + 1;
        const isActive = currentStep?.codeLine
        return (
          <div
            key={lineNum}
            className={`flex items-center px-4
              isActive
                ? 'bg-emerald-600/20 border-l-4
                : 'text-slate-300 hover:bg-slate
            }`}
          >
            <span className="w-8 text-xs text-s
            <span>{line}</span>
          </div>
        );
      })}
    </pre>

```

```

    { id: 3, label: "D", x: 280, y: 250 },
    { id: 4, label: "E", x: 100, y: 250 }
  ];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

export function EulerSimulator() {
  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Кликни на вершину");
  const [c1Done, setC1Done] = useState(false);

  const resetC1 = () => {
    setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
  };

  const clickC1 = (vId: number) => {
    if (c1Done && c1Path.length > 0) return;
    if (c1Path.length === 0) {
      setC1Path([vId]);
      setC1Msg("Старт! Кликай соседние вершины, чтобы п
      return;
    }
    const last = c1Path[c1Path.length - 1];
    const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
    if (!edge) {
      setC1Msg("Нет ребра между этими вершинами! Выбери
      return;
    }
    const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
    if (c1VisitedEdges.includes(key)) {
      setC1Msg("Это ребро уже пройдено! Эйлер не прощае
      setC1Done(true); return;
    }
  }
}

```

```

const key = `${Math.min(e.u,e.v)}-${Math.max(e.u,e.v)}`;
const used = clVisitedEdges.includes(key);
return <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
  stroke={used ? "#6366f1" : "#475569"}
  strokeWidth={used ? 4 : 2}
  className="transition-all duration-300" />
  )})
  {C1_NODES.map(n => {
    const isLast = clPath[clPath.length-1] === n.id;
    const visited = clPath.includes(n.id);
    return <g key={n.id} className="cursor-pointer"
      {isLast && <circle cx={n.x} cy={n.y} r={11}
        <circle cx={n.x} cy={n.y} r={11}
          fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
          stroke={isLast ? "#fff" : visited ? "#6366f1" : "#6366f1"}
          strokeWidth={2.5}
          className="transition-all duration-200" />
          <text x={n.x} y={n.y+4} textAnchor="middle">{n.label}</text>
        </g>
      )})
    </svg>
    <div className="absolute top-2 left-2 bg-slate-500 text-white p-2 rounded"
      >Решено: {clVisitedEdges.length}/{C1_EDGES.length}</div>
  </div>

  <div className={`p-3 rounded-xl border text-sm ${
    clDone && clVisitedEdges.length === C1_EDGES.length
      ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
      : clDone
      ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
      : "bg-slate-900/50 border-slate-700 text-slate-50"}
    >
    {clMsg}
  </div>
</div>
);

```

```
    'Потому что он написан только для положительных чисел',
    'Он жадный и считает, что каждый следующий шаг дает минимальное увеличение суммы закрытых вершин.',
    'Отрицательные ребра создают гравитационный коллапс',
    'Дейкстра просто не любил отрицательные балансы на складе',
],
correctAnswer: 1,
explanation: 'Именно! Дейкстра уверен: если ты приедешь в город, то тебе не понадобится ничего, кроме велосипеда, для чего нужен алгоритм Беллмана-Форда.',
},
{
  id: 3,
  question: 'Какова ключевая фишка алгоритма Борувки',
  options: [
    'Он использует меньше памяти благодаря коммунити-меморизации',
    'Он умеет работать с отрицательными весами без циклов',
    'Он позволяет выполнять шаги слияния колхозов (коммуны)',
    'Он был разработан в Токио и поэтому самый быстрый',
  ],
  correctAnswer: 2,
  explanation: 'Абсолютно верно! В Борувке каждая коммуна должна выбрать своего лидера, чтобы делить вычисления на GPU или многопроцессорных кластерах.',
},
{
  id: 4,
  question: 'В чем главная суть Динамического программирования',
  options: [
    'Писать код очень быстро, динамично нажимая на клавиши',
    'Запоминать (кешировать) результаты уже решенных подзадач',
    'Использовать динамическую типизацию как в Python',
    'Постоянно менять требования к проекту в процессе разработки',
  ],
  correctAnswer: 1,
  explanation: 'Точно! Главный принцип ДП – мемоизация (кеширование).',
},
{
  id: 5,
  question: 'Какая структура данных идеально помогает'
```



```

    });

    const handleRestart = () => {
      setCurrentQIndex(0);
      setSelectedOption(null);
      setIsAnswered(false);
      setScore(0);
      setShowResults(false);
    };

    if (showResults) {
      return (
        <div className="bg-slate-900/90 border border-slate-
          my-12">
          <div className="w-20 h-20 bg-gradient-to-tr from
            -6 shadow-xl shadow-indigo-500/30">
            <Award className="w-10 h-10 text-white" />
          </div>
          <h3 className="text-3xl font-bold text-white mb-
            <p className="text-slate-400 text-sm mb-6">Ты пр

          <div className="bg-slate-950 border border-slate
            <p className="text-slate-400 text-sm uppercase
            <p className="text-5xl font-black text-indigo
            <p className="text-slate-300 text-sm">
              {score === quizQuestions.length
                ? ' Идеально! Ты настоящий сеньор-раскра
                : score >= 3
                ? ' Отличный результат! Главные концепции
                : ' Стоит повторить главы пособия и еще р
            </p>
          </div>

          <button
            onClick={handleRestart}
            className="inline-flex items-center gap-2 px-4
              d rounded-xl shadow-lg shadow-indigo-600/30
          >

```

```

        } else {
            optionStyle = "bg-slate-800/40 border-s
        }
    }
}

return (
    <div
        key={idx}
        onClick={() => handleSelectOption(idx)}
        className={"flex items-start gap-4 p-4"}
    >
        <div className="mt-0.5 shrink-0">
            {isAnswered && isCorrect ? (
                <CheckCircle2 className="w-6 h-6 text-
            ) : isAnswered && isSelected && !isCo
                <XCircle className="w-6 h-6 text-ro
            ) : (
                <div className={"w-6 h-6 rounded-fu
order-indigo-500 bg-indigo-500 text-white"
                    {String.fromCharCode(65 + idx)}
                </div>
            )}
        </div>
        <div className="flex-1 text-sm md:text-l
            {option}
        </div>
    </div>
);
}}
</div>
</div>

```

```

{isAnswered && (
    <div className="bg-slate-950 border border-slate
        <div className="flex items-center gap-2 mb-2
            <AlertCircle className="w-4 h-4" />
            <span>Объяснение:</span>
        </div>
    )}

```

```

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  useVizStepSync(currentStepIndex, setCurrentStepIndex,
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];

  const initialMatrix = [
    [0, 4, 999, 2, 999, 999, 999],
    [999, 0, 3, 999, 999, 3, 999],
    [999, 999, 0, 999, 2, 999, 999],
    [999, 999, 999, 0, 4, 2, 999],
    [999, 999, 999, 999, 0, 999, 1],
    [999, 999, 999, 999, 999, 0, 5],
    [999, 1, 999, 999, 999, 999, 0]
  ];

  const adjList: { [key: number]: { v: number; w: numbe
    0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
    1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
    2: [{ v: 4, w: 2 }],
    3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
    4: [{ v: 6, w: 1 }],
    5: [{ v: 6, w: 5 }],
    6: [{ v: 1, w: 1 }],
  };

```

```

        k, i, j, matrix: dist.map(r => [...r]), u
        log: `  Улучшение пути [${i}]->[${j}] чере
        ∞'}.`,
        activeCodeLine: 7,
      });
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
  log: `  Все пары отработаны. Алгоритм Флойда завер
  activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => setCurrentStepIndex((p)
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl bor
    <div className="flex flex-wrap items-center justifi
      <div className="flex items-center gap-3">

```

```

const u = parseInt(uStr);
return adjList[u].map((edge) => {
  const uNode = nodesSvg.find((n) => n.id === u);
  const vNode = nodesSvg.find((n) => n.id === v);
  const isTarget = currentStep.i === u && currentStep.j === v;
  const isVia = (currentStep.i === u && currentStep.j !== v) || (currentStep.i !== u && currentStep.j === v);
  let strokeColor = '#475569'; let marker = 'none';
  if (isTarget) { strokeColor = '#10b981'; marker = 'circle'; }
  else if (isVia) { strokeColor = '#818cf8'; marker = 'none'; }
  return (
    <g key={`-${u}-${v}`}>
      <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={4} markerEnd={marker} strokeDasharray={isTarget ? [5, 5] : [1, 0]}>
      <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)} r={isTarget ? 4 : 2} stroke={strokeColor} strokeWidth={4}>
      <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2) + 8} fill={strokeColor} fontSize="10" fontWeight="bold" text={`${u} -> ${v}`}>
    </g>
  );
});
}}}
{nodesSvg.map((node) => {
  const isK = currentStep.k === node.id;
  const isI = currentStep.i === node.id;
  const isJ = currentStep.j === node.id;
  let fill = '#1e293b'; let stroke = '#64748f';
  if (isK) { fill = '#4f46e5'; stroke = '#a6a6a6'; }
  else if (isI) { fill = '#b45309'; stroke = '#a6a6a6'; }
  else if (isJ) { fill = '#047857'; stroke = '#a6a6a6'; }
  return (
    <g key={node.id} className="transition-colors">
      <circle cx={node.x} cy={node.y} r={isTarget ? 4 : 2} className="transition-colors duration-200">
      <text x={node.x} y={node.y + 5} fill={strokeColor} font-size="10">
      <text x={node.x} y={node.y + 32} fill={strokeColor} font-size="10">
    </text>
  </g>
  );
});

```



```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};
```

```
type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
  queueOrStack?: string[];
  visitedNodes?: string[];
};
```

```
function generateDFSSteps(start: string) {
  const steps: Action[] = [];
  const vis = new Set<string>();
```

```

        q], visitedNodes: [...Array.from(vis)] }));
    q.shift();

    for (const u of adj[v]) {
      if (!vis.has(u)) {
        vis.add(u);
        q.push(u);
        steps.push({ type: 'visit', node: u, de
          tack: [...q], visitedNodes: [...Array.from(
            }
        }
    }

    steps.push({ type: 'backtrack', node: '', desc: `04
      om(vis)] }));

    return steps;
  }

```

```

export function GraphTraversalViz() {
  const [mode, setMode] = useState<"dfs" | "bfs">("df
  const [autoPlay, setAutoPlay] = useState(false);
  const [stepIdx, setStepIdx] = useState(0);

  const steps = useMemo(() => mode === 'dfs' ? genera
  useVizStepSync(stepIdx, setStepIdx, steps.length -

  useEffect(() => {
    setStepIdx(0);
    setAutoPlay(false);
  }, [mode]);

  useEffect(() => {
    if (autoPlay) {
      const t = setInterval(() => {
        setStepIdx(p => {
          if (p >= steps.length - 1) {

```



```

        const v = nodes.find(n => n.id === vId)
        // Check if edge is part of the current path
        const edgeTraversed = isVisited(v, path)

        return (
          <line key={i} x1={u.x} x2={v.x} y1={u.y} y2={v.y}
            className={`stroke-${
              500 : 'stroke-emerald-500'
            }`}
          />
        )
      )
    }

    /** Nodes */
    {nodes.map(n => {
      const visited = isVisited(n, path)
      const current = isCurrent(n, path)

      let fillColor = "fill-slate-500"
      let strokeColor = "stroke-slate-500"
      let textColor = "fill-slate-500"
      let radius = 18;

      if (visited) {
        fillColor = mode === 'dark' ? 'fill-amber-500' : 'fill-emerald-500'
        strokeColor = mode === 'dark' ? 'stroke-amber-500' : 'stroke-emerald-500'
        textColor = "fill-white"
      }

      if (current && step.type !== 'edge') {
        strokeColor = "stroke-amber-500"
        textColor = "fill-amber-500"
        radius = 22;
      }

      return (
        <g key={n.id} className={`node-${n.id}`}
          <circle cx={n.x} cy={n.y}
            className={`stroke-${strokeColor} fill-${fillColor}`}
            r={radius}
          />
          <text x={n.x} y={n.y}
            className={`text-${textColor}`}
            dy={15}
          />
        />
      )
    })
  )
}

```

```

                {(!step.queueOrStack || step.queue
                  <div className="text-slate-500"
                ))
            }
        </div>
    </div>
</div>

<div className="flex flex-col sm:flex-row gap-2"
  {/* Controls */}
  <div className="flex bg-slate-900 border border-slate-800
    <button onClick={() => {setAutoPlay(
      "p-2 text-slate-400 hover:text-white hover:bg-slate-800"
      <Undo strokeWidth={3} size={16} />
    </button>
    <button onClick={() => setAutoPlay(
      justify-center min-w-[80px] ${mode === 'done' ? 'text-slate-400' : 'text-white'}
    >
      {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
    </button>
    <button onClick={() => {setAutoPlay(
      = steps.length - 1} className="p-2 text-slate-400 hover:bg-transparent">
      <SkipForward strokeWidth={3} size={16} />
    </button>
    <button onClick={() => {setAutoPlay(
      g-slate-800 rounded-lg ml-2 border-l border-slate-800
      <RefreshCw strokeWidth={3} size={16} />
    </button>
  </div>

  {/* Description Log */}
  <div className="flex-1 bg-slate-900/50 border border-slate-800 p-2"
    ow-hidden">
    <div className="text-[10px] text-slate-400">
      {steps.length}</div>
    <div className="text-sm text-slate-400">
      {step.desc}
    </div>
  </div>

```

```
function siftDown(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  const n = a.length;
  let idx = 0;
  while (true) {
    let largest = idx;
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < n && a[l].priority > a[largest].priority) largest = l;
    if (r < n && a[r].priority > a[largest].priority) largest = r;
    if (largest !== idx) {
      [a[largest], a[idx]] = [a[idx], a[largest]];
      idx = largest;
    } else break;
  }
  return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
  return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
  const level = Math.floor(Math.log2(index + 1));
  const posInLevel = index - (Math.pow(2, level) - 1);
  const count = Math.pow(2, level);
  const x = ((posInLevel + 0.5) / count) * 100;
  const y = 10 + level * 24;
  return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
  { name: 'Иван (Инфаркт)', symptom: ' Болят сердце', priority: 100 },
  { name: 'Маша (Перелом)', symptom: ' Открытый перелом ноги', priority: 80 },
  { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', priority: 60 },
  { name: 'Семён (Температура)', symptom: ' Жар 39.5', priority: 40 }
];
```

```

        setTimeout(() => setShake(false), 400);
        return;
    }
    setBusy(true);

    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const finalName = name || preset.name.split(' ')[0];
    const finalEmoji = preset.emoji;
    const finalSymptom = preset.symptom;

    const newPatient: Patient = {
        id: `p${uid++}`,
        name: finalName,
        emoji: finalEmoji,
        symptom: finalSymptom,
        priority,
        animState: 'in',
    };

    const nextHeap = heapInsert(heap, newPatient).map(x => x);
    // Mark specifically this new patient as 'in' in the heap
    const target = nextHeap.find(x => x.priority === priority);
    const marked = nextHeap.map(x => x.id === (target?.id));

    setHeap(marked);
    addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

    setTimeout(() => {
        setHeap(nextHeap);
        setBusy(false);
    }, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const randomShift = Math.floor(Math.random() * 10);
    const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
    insertPatient(preset.name, prio);
};
```

```

    });

    // Patient gets healed in the bed
    setTimeout(() => {
      setHealing(prev => prev ? { ...prev, animStat: 'healed' } : null);
      addLog(` Успех! Пациент ${topPatient.name} поправился.`);
      setTimeout(() => {
        setHealing(null);
        setBusy(false);
      }, 600);
    }, 800);

    }, 350);
  }, 650);
}, [busy, heap]);

const reset = () => {
  setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 'idle' })));
  setLog([' База данных скорой помощи перезагружена.']);
  setHealing(null);
  setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 'idle' } ))));
};

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
  const res: React.ReactElement[] = [];
  const p = nodePos(idx);
  const l = 2 * idx + 1;
  const r = 2 * idx + 2;
  if (l < heap.length) {
    const lp = nodePos(l);
    res.push(
      <line key={`el${idx}`}
        x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
        x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
});

```

```

        className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-emerald-500
        opacity-40 disabled:cursor-not-allowed text-white font-mono text-sm"
        value={scale} transition-all cursor-pointer focus:outline-none"
    >
        <span> Привезти по Скорой (INSERT)</span>
    </button>

    {/* Добавить кастомного */}
    <form onSubmit={handleCustomInsert} className="flex flex-col items-center min-h-100"
    >
        <input
            type="text"
            required
            value={customName}
            onChange={e => setCustomName(e.target.value)}
            placeholder="Имя пациента"
            className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:text-slate-400"
        />
        <div className="flex flex-col items-center min-h-100"
        >
            <span className="text-[9px] font-mono text-slate-400"
            >
                <input
                    type="range"
                    min="10"
                    max="99"
                    value={customPriority}
                    onChange={e => setCustomPriority(Number(e.target.value))}
                    className="w-20 accent-emerald-500 cursor-pointer"
                />
            </div>
            <button
                type="submit"
                disabled={busy || !customName.trim() || !customPriority}
                className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 text-white font-mono text-sm px-4 rounded-xl shadow-lg active:scale-95 transition-all"
            >
                Везти
            </button>
        </form>
    
```

```

const isRoot = idx === 0;
return (
  <div
    key={patient.id}
    className={`
      absolute flex flex-col items-center
      -translate-x-1/2 -translate-y-1/2 t
      ${patient.animState === 'in' ? 'ani
      ${patient.animState === 'winner' ?
      ${patient.animState === 'out' ? 'an
      ${isRoot ? 'bg-rose-950/80 border-r
    `}
    style={{
      left: `${pos.x}%`,
      top: `${pos.y}%`,
      width: isRoot ? 60 : 52,
      height: isRoot ? 60 : 52,
    }}
  >
    <span className="text-2xl leading-non
    <span className="text-[10px] font-mono
      {patient.priority}%
    </span>

    {/* Tooltip */}
    <div className="absolute bottom-full
      <div className="bg-slate-950 border
0 shadow-xl">
      <div className="font-bold text-wh
      <div className="text-emerald-400"
      </div>
    </div>
  </div>
);
}}
</div>

<div className="w-full h-3 bg-gradient-to-r f

```

```
      <div className="w-full h-3 bg-gradient-to-r f
    </div>

  </div>

  {/* ЛОГ ДЕЙСТВИЙ */}
  <div className="bg-slate-900 border border-slate-
    <div className="text-[10px] font-mono text-slate
      {log.map((entry, idx) => (
        <div
          key={idx}
          className={`text-xs font-mono flex items-ce
        >
          {idx === 0 ? <span className="text-emerald-5
          <span>{entry}</span>
        </div>
      )}}
    </div>
  </div>
);
};
```

ФАЙЛ 44/87: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { createContext, useContext, useEffect, u

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими
 * заметить, что именно переехало, и анимация читалась
 */
```



```

    return (
      <div className="w-full">
        <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
          {children}
        </svg>
        {caption && (
          <div className="text-[10px] text-slate-400 text-center">
            {caption}
          </div>
        )}
        <div className="text-[9px] text-slate-600 text-center">
          {paused ? "пауза – курсор на карточке" : "наведение"}
        </div>
      </div>
    );
  };
};

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
      {label !== undefined && (
        <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
          {label}
        </text>
      )}
    </g>
  );
};

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
}> = ({ a, b }) => (
  <g>
    <line x1={a[0]} y1={a[1]} x2={b[0]} y2={b[1]}>
  </g>
);

```

```

        ? "Разрешаем пересадку через k: 3 + 4 = 7"
        : "7 < 9 → D[i][j] = 7"
    }
    >
    <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
    <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
    <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

    <text x={130} y={112} textAnchor="middle" fontSize=
        9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
        3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
        4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
    </Svg>
    );
};

```

/** Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    [
      ["A", "B"], ["B", "C"]],
    [
      ["D", "E"]],
    [
      ["F", "G"], ["G", "H"], ["F", "H"]],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];

```

```

    <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
    {step >= 2 && {
      <text
        x={({a[0] + b[0]) / 2}
        y={({a[1] + b[1]) / 2 - 4}
        textAnchor="middle"
        fontSize={10}
        fontWeight="700"
        fill={C.warn}
      >
        {w}
      </text>
    }}
  </g>
);
}})
{Object.entries(pos).map(([k, [x, y]]) => (
  <Node key={k} x={x} y={y} label={k} fill={C.idle} />
))}
</Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"

```

```
const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, string][];
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "r" ? "#f1c40f" : "#f1c40f";

  return (
    <Svg caption={captions[step]}>
      {f.edges.map(([a, b], i) => (
        <line
          key={i}
          x1={f.pos[a][0]}
          y1={f.pos[a][1]}
          x2={f.pos[b][0]}
          y2={f.pos[b][1]}
          stroke={C.edge}
          strokeWidth={1.8}
          style={{ transition: MOVE }}
        />
      ))}
      {Object.entries(f.pos).map(([id, [x, y]]) => (
        <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
          <circle r={13} fill={color(id)} stroke={C.idl} />
          <text textAnchor="middle" dominantBaseline="bottom">
            {id}
          </text>
        </g>
      ))}
    </Svg>
  );
};
```

/** Zig – один поворот, когда родитель уже корень. */

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step,
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      )}}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k}
          fill={wave.includes(k) ? C.active : visited.h
          stroke={wave.includes(k) ? "#a5b4fc" : undefi
        )}}
      </Svg>
    );
  };

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("

```

```

        x + 1}</text>
      })
    </g>
  );
}
<text x={224} y={62} textAnchor="middle" fontSize=
<text x={224} y={74} textAnchor="middle" fontSize=
</Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
      <Edge a={[60, 70]} b={[200, 70]} color={improved
      <text x={130} y={54} textAnchor="middle" fontSize=
      <Node x={60} y={70} r={20} fill={C.active} label=
      <Node x={200} y={70} r={20} fill={improved ? C.do
      <text x={60} y={106} textAnchor="middle" fontSize=
      <text x={200} y={106} textAnchor="middle" fontSiz
      <text x={130} y={22} textAnchor="middle" fontSize=
    </Svg>
  );
};

```

```

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D
  ];
  const cut = step === 1;
  return (

```

```

    });
    const edges = [
      { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
      { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
    ];
    const chosen = ["A-D", "A-B", "B-C", "D-E"];
    const step = useLoop(chosen.length + 1, 800);
    const taken = new Set(chosen.slice(0, step));
    return (
      <Svg caption={`Жадно берём самые лёгкие рёбра без ц
        {edges.map((e, i) => {
          const on = taken.has(`${e.u}-${e.v}`);
          return (
            <g key={i}>
              <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
              <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
                textAnchor="middle" fontSize={9} fontWeigh
            </g>
          );
        })}
        {Object.entries(pos).map(([k, [x, y]]) => (<Node
      </Svg>
    );
  };

```

```

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
    )}
  );
}

```

```

return (
  <Svg caption={popping ? "pop: забираем ВЕРХНИЮ таре."
    <rect x={90} y={22} width={80} height={96} rx={6}
    {items.map((v, i) => (
      <g key={v} style={{ transition: "all .3s" }}>
        <rect x={94} y={100 - i * 26} width={72} height={26}
          fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
        <text x={130} y={111 - i * 26} textAnchor="middle" />
      </g>
    ))}
    <text x={130} y={16} textAnchor="middle" fontSize={12} />
  </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к концу"
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6"
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
      <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={26}
            fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
          <text x={53 + i * 54} y={71} textAnchor="middle" />
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};

```



```

const pref = a.reduce<number[]>((acc, v) => [...acc,
const step = useLoop(4, 1200);
const l = 1, r = 3;
const show = step >= 1;
return (
  <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -
    о префиксные суммы P">
    <text x={4} y={18} fontSize={8} fill={C.dim}>a</t
    {a.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 39} y={24} width={34} height
          fill={show && i >= l && i <= r ? C.active :
        <text x={37 + i * 39} y={36} textAnchor="midd
          >
        </g>
      )
    )
  <text x={4} y={72} fontSize={8} fill={C.dim}>P</t
  {pref.map((v, i) => (
    <g key={i}>
      <rect x={20 + i * 33} y={78} width={29} height
        fill={show && i === l ? C.bad : show && i =
        stroke={C.idleStroke} style={{ transition:
      <text x={34 + i * 33} y={90} textAnchor="midd
        >
      </g>
    )
  )
  </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы дл
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>

```

```

    <text x={196} y={84} textAnchor="middle" dominantBaseline="middle" fill={C.dim}>
      ?"></text>
    <text x={130} y={124} textAnchor="middle" fontSize={16} fill={C.dim}>
    </Svg>
  );
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс = суффикс` : ""}>
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} height={20}
            fill={step > 0 && (idx < len || (idx > i - len)) ? C.warn : C.idle}
            style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="middle" font-size={16}>
            {ch}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle" font-size={16}>
            {pi[idx]}</text>
          <rect x={18 + idx * 33} y={70} width={28} height={20}
            fill={idx <= i && step > 0 ? C.warn : C.idle} />
        </g>
      ))}
      <text x={4} y={40} font-size={8} fill={C.dim}>s</text>
      <text x={4} y={70} font-size={8} fill={C.dim}>π</text>
      <text x={130} y={106} textAnchor="middle" font-size={16}>
        π[i] – длина самого длинного «префикс = суффикс»</text>
    </Svg>
  );
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);

```

```
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBa
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"
  | "segment-tree" | "bridge" | "articulation" | "mst"
  | "prefix-function" | "dp" | "floyd" | "components" |
  | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  trie: TrieDemo,
  "trie-bfs": TrieBfsDemo,
  "suffix-link": SuffixLinkDemo,
  toposort: ToposortDemo,
  relax: RelaxDemo,
  "prefix-sum": PrefixSumDemo,
  "sparse-table": SparseTableDemo,
  "segment-tree": SegmentTreeDemo,
  bridge: BridgeDemo,
  articulation: ArticulationDemo,
  mst: MstDemo,
  amortized: AmortizedDemo,
  np: NpDemo,
  scc: SccDemo,
  "prefix-function": PrefixFunctionDemo,
  dp: DpDemo,
  floyd: FloydDemo,
  components: ComponentsDemo,
  graph: GraphBasicsDemo,
```

```

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
  <div className="fixed inset-0 z-[110] flex items-en
    <div className="absolute inset-0 bg-slate-950/80
    <div className="relative w-full sm:max-w-5xl max-l
      ded-2xl shadow-2xl flex flex-col">
        <div className="flex items-center gap-3 px-4 py
          <div className="min-w-0 flex-1">
            <h3 className="font-bold text-white text-sm
              {viz.hint && <p className="text-[11px] text
            </div>
            <button
              onClick={onClose}
              className="p-2 rounded-lg bg-slate-800 hove
              aria-label="Заккрыть симулятор"
            >
              <X className="w-5 h-5" />
            </button>
          </div>
        </div>

```

```
export const TermPopover: React.FC<Props> = ({ anchor, onClose }) => {
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: number}>(null);

  useEffect(() => {
    if (!anchor) {
      setPos(null);
      return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height - GAP)), left: left });
  }, [anchor]);

  useEffect(() => {
    if (!anchor) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    window.addEventListener("scroll", onKey, true);
    return () => {
      window.removeEventListener("keydown", onKey);
      window.removeEventListener("scroll", onKey, true);
    };
  }, [anchor, onClose]);

  if (!anchor) return null;
  const entry = GLOSSARY_BY_ID.get(anchor.termId);
```

```
        </p>
      )}

      {entry.complexity && {
        <div className="text-[11px] font-mono text-
          Сложность: {entry.complexity}
        </div>
      )}

      {viz && simulatorId && {
        <button
          onClick={() => onOpenSimulator(simulatorId)}
          className="w-full flex items-center justify-
            unded-lg py-2 transition-colors"
        >
          <Play className="w-3.5 h-3.5" /> Показать
        </button>
      )}
    </div>
  </div>,
  document.body
);
};
```

ФАЙЛ 51/87: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync } from '../data/vizStepBus';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
```

```

        return "stroke-slate-700 stroke-dasharray-4";
    }

    if (e.id === 'AB') {
        if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 4) return "stroke-emerald-500";
    }
    if (e.id === 'BC') {
        if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
        if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
        if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // отсюда
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
    if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
    }
    if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
    if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
    if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
    if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
    if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
    if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

```



```

case 0: return (
    <div>
        <b>Исходный граф.</b> Имеется ребро
        Если мы запустим быстрый алгоритм Д
    </div>
);
case 1: return (
    <div>
        <b>Шаг 1: Точка отсчета.</b><br/>
        Добавляем фиктивную вершину S. Соед
ми</span>.
        Это наша база для расчета потенциал
    </div>
);
case 2: return (
    <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
        Запускаем медленный, но мощный алгор
ных вершин:
        <span className="text-pink-300 ml-1
    </div>
);
case 3: return (
    <div>
        <b>Шаг 3: Магическая формула.</b><b>
        Всем ребрам графа назначаем новый в
        <div className="text-center font-mo
        Это повышает или понижает вес ребра
    </div>
);
case 4: return (
    <div>
        <b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
        Старый вес: <span className="text-r
        Вычисляем:  $-2 + 0 - (-2) =$  <span cl
    </div>
);
case 5: return (

```

```

    start-reverse">
        <path d="M 0 0 L 10 5 L
    </marker>
    <marker id="arrow-slate-dim
uto-start-reverse">
        <path d="M 0 0 L 10 5 L
    </marker>
    <marker id="arrow-amber" vie
start-reverse">
        <path d="M 0 0 L 10 5 L
    </marker>
    <marker id="arrow-pink" vie
tart-reverse">
        <path d="M 0 0 L 10 5 L
    </marker>
    <marker id="arrow-emerald"
o-start-reverse">
        <path d="M 0 0 L 10 5 L
    </marker>
</defs>

{/* Edges */}
{edges.map((e, i) => {
    const u = nodes.find(n => n
    const v = nodes.find(n => n

    const isS = e.u === 'S';
    if (isS && (step === 0 || s

    const midX = (u.x + v.x) / 2;
    const midY = (u.y + v.y) / 2;

    const edgeColorClass = getE
    const textHigh = isEdgeText
    const textEmer = isEdgeText

    return {
        <g key={i}>

```

```

</text>

        {pot !== null && {
          <g transform={`
fade-in zoom-in duration-500">
            <rect x="-1
k-500 stroke-1" />
            <text x="0"
font-bold fill-pink-400">
              {pot}
            </text>
          </g>
        })
      </g>
    )
  }}}}
</svg>
</div>

```

```

<div className="w-full md:w-[320px] flex
  {/* Log Box */}
  <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
  {getDesc()}
  </div>

```

```

  {/* Controls */}
  <div className="flex bg-slate-900 b
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
      <RotateCcw strokeWidth={3}
    </button>
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
      <Undo strokeWidth={3} size=
    </button>

```

```

interface GNode {
  id: number;
  x: number;
  y: number;
  label: string;
}

interface GEdge {
  u: number;
  v: number;
}

const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
}

```

```

    recordStep(
      "init",
      "Инициализация алгоритма Косарайю. Начинаем первый
      null,
      false,
      -1,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, (
    initialEdges.forEach((e) => adj[e.u].push(e.v)));

    // DFS 1: post-order list
    const dfs1 = (u: number) => {
      visited.add(u);
      recordStep(
        "dfs1",
        `DFS 1: зашли в вершину ${u}, помечаем посещение
        u,
        false,
        -1,
      );

      for (const to of adj[u]) {
        if (!visited.has(to)) {
          dfs1(to);
        }
      }

      order.push(u);
      recordStep(
        "dfs1",
        `DFS 1: вершина ${u} полностью обработана. Запи
        u,
        false,
        -1,
      );
    };
  };

```

```

        if (!visited.has(to)) {
            dfs2(to, currentScc);
        }
    }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
    const u = order[i];
    if (!visited.has(u)) {
        sccId++;
        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека`,
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в`,
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
    null,
    true,
    sccId,
);

```

```

    });

    return (
      <div className="bg-slate-900 rounded-xl border border-
        <div className="bg-slate-800 p-4 border-b border-
          <h3 className="font-bold text-white flex items-
            <Layers className="w-5 h-5 text-emerald-400"
              Визуализатор Алгоритма Косарайю (Поиск SCC)
            </h3>

          <div className="flex items-center gap-2 bg-slate-
            <button
              onClick={togglePlay}
              className="flex items-center gap-2 px-3 py-
                d-500 text-white"
            >
              {isPlaying ? (
                <Pause className="w-4 h-4 ml-1" />
              ) : (
                <Play className="w-4 h-4 ml-1" />
              )}
              {isPlaying ? "Пауза " : "Старт "}
            </button>

            <button
              onClick={reset}
              className="flex items-center justify-center
                ors"
            >
              <RotateCcw className="w-4 h-4" />
            </button>
          </div>
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-12
          {/* Playfield SVG */}
          <div className="lg:col-span-8 bg-[#0B1120] rela
            <svg className="w-full h-full max-w-[650px]"

```

```

{initialEdges.map((edge, idx) => {
  const u = initialNodes.find((n) => n.id === edge.u);
  const v = initialNodes.find((n) => n.id === edge.v);

  // If transposed, draw reversed coordinates
  const x1 = step.transposed ? v.x : u.x;
  const y1 = step.transposed ? v.y : u.y;
  const x2 = step.transposed ? u.x : v.x;
  const y2 = step.transposed ? u.y : v.y;

  const isSccEdge =
    step.sccMap[edge.u] &&
    step.sccMap[edge.v] &&
    step.sccMap[edge.u] === step.sccMap[edge.v];
  let stroke = "#475569";
  let markerId = "arrow";

  if (step.transposed) {
    stroke = isSccEdge ? "#818cf8" : "#4f4682";
    markerId = "arrow-transposed";
  } else {
    if (
      step.currentNode === edge.u ||
      step.currentNode === edge.v
    ) {
      stroke = "#10b981";
      markerId = "arrow-active";
    }
  }
}

// Adjust slightly for bidirectional link
const isBidi =
  (edge.u === 3 && edge.v === 4) ||
  (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
  <line

```



```

        />
        <text
            x={node.x}
            y={node.y}
            dy=".3em"
            textAnchor="middle"
            fill="white"
            fontSize="13px"
            fontWeight="bold"
            className="select-none"
        >
            {node.label}
        </text>

        {sccId && (
            <text
                x={node.x}
                y={node.y - 28}
                textAnchor="middle"
                fill="#10b981"
                fontSize="10px"
                fontWeight="bold"
            >
                SCC #{sccId}
            </text>
        )}
    </g>
  );
  }}}
</svg>

{step.transposed && (
  <div className="absolute top-4 left-4 bg-in
    1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>
  )}
</div>

```

```

    <div>
      <span className="text-[10px] text-slate-500">
        ЛОГ ОПЕРАЦИЙ:
      </span>
      <div className="space-y-1.5 max-h-[150px]">
        {steps
          .slice(0, currentStepIdx + 1)
          .reverse()
          .slice(0, 5)
          .map((s, idx) => {
            <div
              key={idx}
              className={`text-[11px] p-1.5 rounded`
            >
              {s.desc}
            </div>
          )}}
      </div>
    </div>
  </div>

  <div className="mt-4 pt-3 border-t border-slate-200">
    <span>
      Шаг {currentStepIdx + 1} из {steps.length}
    </span>
    <div className="flex gap-1.5">
      <button
        disabled={currentStepIdx === 0}
        onClick={() => setCurrentStepIdx((prev) => prev - 1)}
        className="bg-slate-900 border border-slate-700 text-white px-2 py-1 rounded"
      >
        Назад
      </button>
      <button
        disabled={currentStepIdx >= steps.length - 1}
        onClick={() => setCurrentStepIdx((prev) => prev + 1)}
        className="bg-slate-900 border border-slate-700 text-white px-2 py-1 rounded"
      >
        Далее
      </button>
    </div>
  </div>

```

```

    title: string;
    description: string;
    type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
  { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
  { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
  { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
  { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
  { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
  const [vertices, setVertices] = useState<Vertex[]>(in
  const [edges, setEdges] = useState<Edge[]>(initialEdge

```



```

        title: 'Шаг 6: Берем ребро E-F (вес 7)',
        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 12: Include E-F
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро E-F в осто́ве',
        description: 'Вершина F окрашена в фиолетовый ц
        type: 'success'
    }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Берем ребро D-G (вес 8)',
        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.'
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{

```

```

        description: 'Оба конца E и H уже фиолетовые. С
        type: 'warning'
    }, ...prev]);
},
// Step 20: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро E-H.',
        type: 'error'
    }, ...prev]);
},
// Step 21: Inspect H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Берем ребро H-I (вес 12)',
        description: 'Соединяет фиолетовую H и последнюю
        type: 'info'
    }, ...prev]);
},
// Step 22: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Успех: MST построено Краскалом!',
        description: 'Все 9 вершин окрашены в единый фи
            + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
        type: 'success'
    }, ...prev]);
},
];

// --- PRIM STEPS ---
const primSteps = [
    // Step 1: Start at E

```

```

        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в кучу',
        type: 'success'
    }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к вершине A',
        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-E(6)
        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {

```

```
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (E) тянется к F.',
    type: 'info'
  }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
    eap' } : e));
  setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (10)']);
  setLogs(prev => [{
    title: 'Успех: Вершина F захвачена',
    description: 'Ребро F-I(13) добавлено в кучу. Ресурс перемещен к F.',
    type: 'success'
  }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
    description: 'Плесень расширяется вниз к G.',
    type: 'info'
  }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
    eap' } : e));
  setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H (10)']);
  setLogs(prev => [{
    title: 'Успех: Вершина G захвачена',
```



```

        setLogs(prev => [{
            title: 'Успех: Вершина H захвачена',
            description: 'В кучу добавлено H-I(12).',
            type: 'success'
        }, ...prev]));
    },
    // Step 20: Pop E-H (11) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
            title: 'Шаг 11: Куча выдает E-H (вес 11)',
            description: 'Обе вершины E и H уже в плесени.',
            type: 'warning'
        }, ...prev]));
    },
    // Step 21: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setHeapQueue(['H-I (12)', 'F-I (13)']));
        setLogs(prev => [{
            title: 'Отказ: Игнорируем E-H',
            description: 'Выбрасываем из кучи.',
            type: 'error'
        }, ...prev]));
    },
    // Step 22: Pop H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Шаг 12: Куча выдает H-I (вес 12)',
            description: 'Плесень тянется к последней чистой',
            type: 'info'
        }, ...prev]));
    },
    // Step 23: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {

```

```

// Component 1: {A, B, C} (colored red)
// Component 2: {D, E, F, G, H, I} (colored gold)
setVertices(prev => prev.map(v =>
  [0, 1, 2].includes(v.id) ? { ...v, color: 'red' }
));
setEdges(prev => prev.map(e => {
  if (['0-1', '1-2'].includes(e.id)) {
    return { ...e, status: 'included', color: 'red' }
  }
  if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
    return { ...e, status: 'included', color: 'blue' }
  }
  return e;
}));
setLogs(prev => [{
  title: 'Слияние: Деревни объединились в колхозы',
  description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
  type: 'success'
}, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
() => {
  // Now both kolkhozes look at all outgoing roads
  // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
  // A-D (5), B-E (6), C-F (9).
  // Cheapest is A-D (5).
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
    status: 'included', color: 'red'
  } : e));
  setLogs(prev => [{
    title: 'Вторая пятилетка: Колхозы выбирают мост',
    description: 'Теперь Красный и Синий колхозы объединились в один колхоз. Самый дешевый мост A-D с весом 5. Остальные мосты B-E(6) и C-F(9) остаются в наметке.',
    type: 'warning'
  }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
  setVertices(prev => prev.map(v => ({ ...v, color: 'blue' })));
  setEdges(prev => prev.map(e => ({ ...e, status: 'included', color: 'blue' })));
  setLogs(prev => [{
    title: 'Завершение: Все деревни объединены в один колхоз',
    description: 'Все дороги построены. Колхозы объединены в один колхоз.',
    type: 'success'
  }, ...prev]);
}

```

```

    setHeapQueue([]);
    if (algo === 'kruskal') {
        setLogs([
            {
                title: 'Введение в раскраску (Краскал)',
                description: 'Представь, что все 9 вершин графа
                    уже покрашены, и тебе нужно выбрать
                    ребро (Heap) и захватываем соседей!',
                type: 'info',
            },
        ]);
    } else if (algo === 'prim') {
        setLogs([
            {
                title: 'Введение в Плесень (Прима)',
                description: 'Логика «Плесени»: выбираем стар
                    едь (Heap) и захватываем соседей!',
                type: 'info',
            },
        ]);
    } else {
        setLogs([
            {
                title: 'Введение в Коллективизацию (Борувка)',
                description: 'Логика «Борувки»: Каждая из 9 д
                    ет хочет объединиться в колхоз.',
                type: 'info',
            },
        ]);
    }
};

```

```

const getColorClass = (color: string, id: number) => {
    switch (color) {
        case 'red': return 'bg-red-500 border-red-300 tex
        case 'blue': return 'bg-blue-500 border-blue-300
        case 'purple': return 'bg-purple-600 border-purple
        case 'mold': return 'bg-emerald-500 border-emerald
        default:
            if (activeAlgo === 'prim' && id === 4 && stepIn

```

```

    { /* Algorithm Tabs */ }
    <div className="flex flex-wrap items-center gap-1" />
    <div className="flex items-center bg-slate-400 p-2" />
    <button
      onClick={() => handleReset('kruskal')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'kruskal'
          ? 'bg-indigo-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >
      <GitGraph className="w-3.5 h-3.5" />
      <span>Краскал (Билет 18)</span>
    </button>
    <button
      onClick={() => handleReset('prim')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'prim'
          ? 'bg-emerald-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >
      <Layers className="w-3.5 h-3.5" />
      <span>Прима (Билет 19)</span>
    </button>
    <button
      onClick={() => handleReset('boruvka')}
      className={
        "flex items-center gap-1.5 px-3 py-2"
      +
        (activeAlgo === 'boruvka'
          ? 'bg-rose-600 text-white shadow-md'
          : 'text-slate-400 hover:text-slate-300')
      >

```

```
<div className="lg:col-span-2 bg-slate-950 rounded-2xl p-4 active min-h-[480px] overflow-hidden">
```

```
  { /* Legend */ }
```

```
  <div className="absolute top-4 left-4 bg-slate-950 p-4 text-center gap-3 text-xs">
```

```
    <div className="flex items-center gap-1.5">
```

```
      <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
    </div>
```

```
    {activeAlgo === 'kruskal' ? (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
      </>
```

```
    ) : activeAlgo === 'prim' ? (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-1 bg-sky-600">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
      </>
```

```
    ) : (
```

```
      <>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-3 h-3 rounded-full bg-slate-950">
```

```
        </div>
```

```

        fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
        fontSize="4.5"
        fontFamily="monospace"
        fontWeight="bold"
        textAnchor="middle"
        dominantBaseline="middle"
        className="select-none filter drop
dy="-1.5"
    >
        {edge.weight}
    </text>
</g>
    );
  }}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-f
  {vertices.map(vertex => {
    <div
      key={vertex.id}
      style={{ top: vertex.y + "%", left: v
      className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
    >
      <span>{vertex.label}</span>
      {vertex.id === 4 && activeAlgo === 'p
        <span className="text-[8px] block -
      }}
    </div>
  )}}
</div>

```

```

{stepIndex >= currentSteps.length && (
  <div className="absolute inset-x-6 bottom
shadow-xl z-20">

```

```

<div className="flex-1 p-4 overflow-y-auto"
  {logs.map((log, idx) => {
    <div
      key={idx}
      className={
        "p-4 rounded-xl border transition-a
        {log.type === 'success'
          ? 'bg-emerald-950/40 border-emera
          : log.type === 'warning'
          ? 'bg-amber-950/40 border-amber-5
          : log.type === 'error'
          ? 'bg-rose-950/40 border-rose-500
          : 'bg-slate-900/60 border-slate-7
        }
      }
    >
      <div className="flex items-center gap
        {log.type === 'success' && <CheckCi
        {log.type === 'warning' && <HelpCir
        {log.type === 'error' && <XCircle c
        {log.type === 'info' && <Play class
        <h5 className="font-bold text-sm le
      </div>
      <p className="text-xs text-slate-300
    </div>
  }}}
</div>
</div>
</div>
</div>

```

```

{/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
  <div className="flex flex-col sm:flex-row items
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-40
      <h4 className="text-xl font-bold text-white
    </div>

```

```

        <Award className="w-3.5 h-3.5 text-indigo-500">
    </p>
    <p className="text-2xl font-black text-indigo-500">
    <p className="text-slate-400 text-xs mt-1">
</div>
<div className="bg-slate-950 p-4 rounded-2xl">
    <p className="text-slate-400 text-xs font-medium">
ожении:</p>
    <p className="text-xs text-slate-300 leading-relaxed">
ди и красим вершины в один цвет, бракуя циклы.
    </div>
</div>
<div className="lg:col-span-2">
    <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
            <Code className="w-3.5 h-3.5 text-indigo-500">
            </p>
            <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))`}
```



```
def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst`}
```

</pre>

</div>

</div>

</div>

}}

{activeCheatTab === 'boruvka' && {

<div className="grid grid-cols-1 lg:grid-cols-

<div className="lg:col-span-1 space-y-4">

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Clock className="w-3.5 h-3.5 text-ro

</p>

<p className="text-2xl font-black text-r

<p className="text-slate-400 text-xs mt

p>

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Award className="w-3.5 h-3.5 text-ro

</p>

<p className="text-2xl font-black text-r

```
    </div>
  );
};
```

ФАЙЛ 54/87: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
  {
    img: dijkstraImg,
    alt: 'Добрый робот Дейкстра',
    title: '    Добрый (Дейкстра)',
    desc: 'Быстрый, жадный, но работает только с',
    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
    rest: 'весами. Подсунь отрицательное ребро – сломает',
    complexity: 'O((V+E) log V)',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: '    Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый – зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/60',
```

}

ФАЙЛ 55/87: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";
```

```
interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы — показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}
```

```
/**
```

```
 * Мобильное содержание: липкая панель снизу-сверху + в
```

```
 * На десктопе не рендерится (там обычный сайдбар).
```

```
 */
```

```
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
  const [open, setOpen] = useState(false);
```

```
  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : "";
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);
```

```
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escape" ?
    window.addEventListener("keydown", onKey);
```

```
        <div className="flex-1 min-h-0">
          <Sidebar
            selectedChapterId={selectedChapterId}
            setSelectedChapterId={setSelectedChapterId}
            onNavigate={() => setOpen(false)}
          />
        </div>
      </div>
    </div>
  )}
</>
);
};
```

ФАЙЛ 56/87: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator';
  setActiveTab: (tab: 'guide' | 'simulator') => void;
  /** Открыта ли боковая панель компилятора. */
  compilerOpen: boolean;
  onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab, compilerOpen, onToggleCompiler }) => {
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
```

```

        <BookOpen className="w-4 h-4" /> Учебное по
</button>
<button
  id="tab-simulator-btn"
  onClick={() => setActiveTab('simulator')}
  className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
    activeTab === 'simulator'
      ? 'bg-indigo-600 text-white shadow-lg s
      : 'text-slate-400 hover:text-white'
  }`}
>
  <Cpu className="w-4 h-4" /> Тренажёры
</button>
<button
  id="tab-compiler-btn"
  onClick={onToggleCompiler}
  aria-pressed={compilerOpen}
  title="Боковая панель Python-компилятора (на
    страницей"
  className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
    compilerOpen
      ? 'bg-emerald-600 text-white shadow-lg
      : 'text-slate-400 hover:text-white'
  }`}
>
  <Terminal className="w-4 h-4" /> Python
</button>
<a
  id="download-pdf-btn"
  href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
target="_blank"
rel="noreferrer"
  className="flex items-center justify-center
over:bg-emerald-600/20 border border-emeral
title="Скачать полный PDF сборник всех стра

```

```

return (
  <div className="space-y-4">
    <h4 className="text-base font-bold text-white">K5
    <p className="text-xs text-slate-400">Просто смотри

  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    { /* K5 */ }
    <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
      <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
      <svg className="w-full h-full absolute inset-0">
        {(() => {
          const pts = [
            {id:0,x:160,y:40},
            {id:1,x:270,y:100},
            {id:2,x:230,y:230},
            {id:3,x:90,y:230},
            {id:4,x:50,y:100}
          ];
          const edges = [
            [0,1],[0,2],[0,3],[0,4],
            [1,2],[1,3],[1,4],
            [2,3],[2,4],
            [3,4]
          ];
          function findPt(id:number) { return pts.find(p => p.id === id); }
          function intersect(a:number,b:number,c:number,d:number) {
            if (a===c||a===d||b===c||b===d) return true;
            const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
            function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
              return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
            }
            return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
              && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
          }
          const crosses: number[] = [];
          for(let i=0;i<edges.length;i++) {
            for(let j=i+1;j<edges.length;j++) {

```

```

    ];
    const edges = [
      [0,3],[0,4],[0,5],
      [1,3],[1,4],[1,5],
      [2,3],[2,4],[2,5]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
      if (a===c||a===d||b===c||b===d) return
      const p1=findPt(a),p2=findPt(b),p3=find
      if (!p1||!p2||!p3||!p4) return false;
      function ccw(ax:number,ay:number,bx:num
        return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
      }
      return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
        && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses:number[]=[];
    for(let i=0;i<edges.length;i++) for(let j
      if (intersect(edges[i][0],edges[i][1],e
        if (!crosses.includes(i)) crosses.push
        if (!crosses.includes(j)) crosses.push
      }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
      const p1=findPt(edges[i][0]),p2=findPt(
      const xing=crosses.includes(i);
      lines.push(<line key={`l${i}`} x1={p1.x
        stroke={xing?"#f43f5e":"#4f46e5"} stro
      }
    const labels = ["\u04141","\u04142","\u04
    for(const p of pts) {
      circles.push(<g key={`c${p.id}`}>
        <circle cx={p.x} cy={p.y} r={8} fill=
        <text x={p.x} y={p.y+3} textAnchor="m
      </g>);
    }
  }

```

```

    Play,
    Plus,
    RotateCcw,
    SkipBack,
    SkipForward,
    Square,
    Terminal,
    Unlink,
    Variable,
    X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { emitVizStep, onVizDemo, vizHitsAtTrace, vizSte
import {
    baseVarName,
    buildDebugRunner,
    changedVars,
    DEBUG_STEP_CAP,
    orderVars,
    type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo

interface PythonCompilerProps {
    /** Страница, с которой синхронизируется редактор. */
    chapterId: string;
    chapterTitle: string;
    /** Ширина панели на десктопе (px) – управляется роди
    width?: number;
    onWidthChange?: (w: number) => void;
    /** Перейти к странице пособия (посмотреть визуализац
    onOpenGuide: () => void;
    /** Скрыть панель. */
    onClose: () => void;

```



```
interface Snippet {
  label: string;
  tip: string;
  code: string;
}

const SNIPPETS: { group: string; items: Snippet[] }[] =
  [
    {
      group: "Сортировка",
      items: [
        {
          label: "arr.sort() / sorted()",
          tip: "In-place сортировка по возрастанию и новая",
          code: 'arr = [5, 2, 8, 1]\narr.sort()
                ) # копия по убыванию\n',
        },
        {
          label: "sorted(key=...)",
          tip: "Сортировка по ключу: рёбра по весу, пары",
          code: 'edges = [(3, "A", "B"), (1, "B", "C"), (
                    es)\n',
        },
      ],
    },
    {
      group: "Циклы",
      items: [
        {
          label: "for i in range(n)",
          tip: "Вложенные циклы по i и j – как обход матри",
          code: "n, m = 3, 4\nfor i in range(n):",
        },
        {
          label: "while ...",
          tip: "Классический while-счётчик – как n-1 итер",
          code: "i = 1\nwhile i < 5:\n    print(f\"итераци",
        },
      ],
    },
  ]
```

```

    },
    {
      group: "Вывод",
      items: [
        {
          label: "f-строка",
          tip: "Подставить значения переменных шага в строку",
          code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}]")',
        },
      ],
    },
  ],
};

```

```

/** Переживает скрытие панели: код пользователя не теряется
let savedEditor: { code: string; template: string; chapterId: string };

```

```

export function PythonCompiler({ chapterId, chapterTitle }) {
  /** Активная вкладка демонстрации страницы (у sparse-страницы)
  const [demoId, setDemoId] = useState<string | null>(null);
  const sync = useMemo(() => getPageSync(chapterId, demoId));
  const syncVarBases = useMemo(() => sync.variables.map((v) => v.bases));
  const syncVarSet = useMemo(() => new Set(syncVarBases));
  /** Полный эталонный код – подсматривается «глазом», не редактируется
  const template = useMemo(() => buildSyncTemplate(chapterId, sync));
  /** Дефолт редактора: шапка + только инициализированные переменные
  const initTemplate = useMemo(() => buildInitTemplate(chapterId));
  /** Помеченные строки эталона по содержимому: каждое слово – это код
  const markedContents = useMemo(() => {
    const marks = sync.stepCodeLines;
    if (!marks || marks.length === 0) return undefined;
    const lines = template.split("\n");
    return new Set(marks.map((l) => (lines[l + 2] ?? "")));
  }, [sync, template]);
  /** Режим «подсмотреть эталон»: показывает полный код
  const [showRef, setShowRef] = useState(false);

  const [code, setCode] = useState(() => {
    if (!savedEditor) return initTemplate;

```

```
// Смена страницы (или вкладки демо): нетронутый редактор
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
  const prev = prevTplRef.current;
  if (initTemplate === prev.base && template === prev.full) {
    prevTplRef.current = { base: initTemplate, full: template };
    setPlaying(false);
    setDebug(null); // трасса ссылается на строки старого кода
    setShowRef(false);
    if (code === prev.base || code === prev.full || code === prev.full) {
      setCode(initTemplate);
      setDesyncedFrom(null);
    } else {
      setDesyncedFrom(chapterId);
    }
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
  prevTplRef.current = { base: initTemplate, full: template };
  setCode(initTemplate);
  setDesyncedFrom(null);
  setStripTab("vars");
  setPlaying(false);
  setDebug(null);
  setShowRef(false);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(showRef ? template : code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
    setRuntimeMessage("Не получилось скопировать – буфер переполнен");
  }
}, [code, showRef, template]);
```

```

        const idx = Math.max(0, Math.min(d.result.steps
        if (vizActiveNow(d.src)) emitVizStep(chapterId,
            лизация следует за шагом
        return { ...d, idx });
    });
},
[chapterId, markedContents, stepCapable, vizActiveNow
]);

const stepBy = useCallback(
    (delta: number) => {
        setDebug((d) => {
            if (!d) return d;
            const idx = Math.max(0, Math.min(d.result.steps
            if (vizActiveNow(d.src)) emitVizStep(chapterId,
            return { ...d, idx });
        });
    },
    [chapterId, markedContents, stepCapable, vizActiveNow
]);

const stepTo = useCallback((idx: number) => goDebug(idx));

const stopDebug = useCallback(() => {
    setPlaying(false);
    setDebug(null);
}, []);

/** Интервал автопрохода шагов (мс). */
const AUTOPLAY_MS = 650;

// Автопроход: пока включен, шагает stepBy каждые AUTOPLAY_MS
useEffect(() => {
    if (!playing || !debug) return;
    if (debug.idx >= debug.result.steps.length - 1) {
        setPlaying(false);
        return;
    }
},

```

```

    } catch (error) {
      setOutput(`Ошибка:\n${errorText(error)}`);
      setRuntimeMessage("Не удалось протрассировать – пр
    } finally {
      setRunning(false);
    }
  }, [code, stdin, running, runtimeReady, chapterId, ma

// Стрелки ← → в режиме отладки листают ШАГИ (а не стр
useEffect(() => {
  if (!debug) return;
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLInputElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target))
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(1);
    }
    if (e.key === "ArrowLeft") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(-1);
    }
    if (e.key === " ") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying((v) => !v);
    }
    if (e.key === "Escape") stopDebug();
  };
  // capture-фаза: чтобы глобальный обработчик навигац
  window.addEventListener("keydown", onKey, true);
  return () => window.removeEventListener("keydown", onKey);
}, [debug, stepBy, stopDebug]);

```

```

        requestAnimationFrame(() => {
            ta.focus();
            ta.selectionStart = ta.selectionEnd = start + p;
        });
        return next;
    });
}, []));

const hasViz = sync.variables.length > 0;

return (
    <aside
        aria-label="Python-компилятор"
        className="fixed z-40 inset-x-0 bottom-0 h-[58dvh]
            px] flex flex-col bg-slate-900 border-t lg:border-0"
        style={width && typeof window !== "undefined" && width}
    >
        {/* Ручка растягивания панели за левый край (только для десктопа) */}
        {onWidthChange && (
            <div
                role="separator"
                aria-orientation="vertical"
                aria-label="Потяните, чтобы изменить ширину панели"
                title="Потяните, чтобы изменить ширину панели"
                className="hidden lg:block absolute top-0 bottom-0 left-0"
                onPointerDown={(e) => {
                    const el = e.currentTarget;
                    el.setPointerCapture(e.pointerId);
                    const startX = e.clientX;
                    const startW = width ?? 560;
                    const onMove = (ev: PointerEvent) => {
                        const next = Math.round(startW + (startX - ev.clientX));
                        onWidthChange(Math.max(380, Math.min(next, 1000)));
                    };
                    const onUp = () => {
                        el.removeEventListener("pointermove", onMove);
                        el.removeEventListener("pointerup", onUp);
                    };
                }}
            >

```

```

        type="button"
        onClick={onClose}
        className="p-1.5 rounded-lg text-slate-400"
        aria-label="Заккрыть компилятор"
      >
        <X className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip
      side="bottom"
      content={
        showRef
          ? "Скрыть эталонный код и вернуться к с
          : "Подсмотреть эталонный код целиком (т
        }
      >
        <button
          type="button"
          onClick={() => setShowRef((v) => !v)}
          className={`p-1.5 rounded-lg transition-co
            hite hover:bg-slate-800`}
          aria-label={showRef ? "Скрыть эталонный к
        >
          {showRef ? <EyeOff className="w-4 h-4" />
        </button>
      </Tooltip>
    <Tooltip
      side="bottom"
      content="Запуск = пошаговый отладчик: код в
      трация слева следует за шагами. Листайте ←
    >
    <button
      type="button"
      onClick={() => void debugRun()}
      disabled={running}
      className="ml-1 inline-flex items-center
      -60 text-white text-xs font-bold transition
    >

```

```

        tabIndex={0}
        className={`cursor-help inline-flex items-center justify-center gap-1
          hasViz ? "text-emerald-400" : "text-slate-400"
        }`}
      >
        <Link2 className="w-3 h-3" />
        {hasViz ? "синхронизировано" : "нет виз"}
      </span>
    </Tooltip>
  </div>
</div>

{stripTab === "vars" ? (
  <div className="px-3 pb-2.5 space-y-2 max-h-300">
    <div className="flex items-center gap-2 min-w-300">
      <Tooltip side="bottom" content="Открыть эскиз">
        <button
          type="button"
          onClick={onOpenGuide}
          className="inline-flex items-center gap-1
            700 text-slate-300 hover:text-white hover:bg-slate-500"
        >
          <span className="truncate">{chapterTitle}</span>
        </button>
      </Tooltip>
    </div>
    {desyncedFrom !== null && (
      <Tooltip
        side="bottom"
        content="В редакторе ваш код, а страница
        т заменён)."
      >
        <button
          type="button"
          onClick={resync}
          className="flex items-center gap-1.5
            0/40 text-amber-400 hover:bg-amber-500/20 t
        >

```



```

        <span className="truncate max-w-50">
          </span>
        </Tooltip>
      )}
    </div>
  )}
</div>
) : (
  <div className="px-3 pb-2.5 max-h-36 overflow-hidden">
    {SNIPPETS.map(({gItem}) => {
      <div key={gItem.group}>
        <div className="text-[9px] font-bold uppercase">
          <div className="flex flex-wrap gap-1.5">
            {gItem.items.map(({s}) => {
              <Tooltip key={s.label} side="bottom">
                <button
                  type="button"
                  onClick={() => insertSnippet(s.label)}
                  className="inline-flex items-center gap-1.5 text-slate-600 hover:text-white"
                  title={s.description}
                >
                  <Plus className="w-3 h-3 text-slate-600"/>
                  {s.label}
                </button>
              </Tooltip>
            )}}
          </div>
        </div>
      </div>
    )}}
  <p className="text-[10px] text-slate-600">Here you can find a list of snippets that you can use in your code. You can click on the snippet to see its details and use it in your code.
</div>
)}
</div>

```

```

/* — Редактор + ввод
<div className="flex-1 min-h-0 flex flex-col">
  {debug ? (

```

```

        <ChevronRight className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip content="В конец трассы">
      <button type="button" onClick={() => {
        tep || debug.idx >= debug.result.steps.length
        0 disabled:opacity-40 transition-colors">
        <SkipForward className="w-4 h-4" />
      </button>
    </Tooltip>
    <span className="ml-1.5 text-[10px] text-
      {curDebug?.step ? (
        <>
          var {debug.idx + 1}/{debug.result.steps
            {curDebug.step.func !== "<module>" &&
              {vizActiveNow(debug.src) && (
                <span className="text-indigo-300">
                  {" · "}{curHits <= 1 ? "инициализация" : ""}
                  {vizLink === "partial" && curHits > 1 ? "поиск" : ""}
                </span>
              )}
            </>
          ) : (
            "шагов нет – программа не выполнила ни одного"
          )}
        </span>
      {debug.result.truncated && (
        <Tooltip content={`Трасса обрезана: лимит достигнут.
        шие данные.`}>
          <span tabIndex={0} className="cursor-default rounded-full px-1.5 py-0.5">
            ЛИМИТ
          </span>
        </Tooltip>
      )}
    <Tooltip
      content={
        !stepCapable

```

```
00/30 transition-colors">
    <Square className="w-3.5 h-3.5" />
  </button>
</Tooltip>
</div>
```

```
{/* листинг с подсветкой текущей строки */}
<div ref={listRef} className="flex-1 min-h-[300px]">
  {debug.src.split("\n").map((line, i) => {
    const ln = i + 1;
    const active = ln === curDebug?.step?.line;
    return (
      <div
        key={i}
        data-line={ln}
        className={`flex whitespace-pre px-3 py-2
          active ? "bg-emerald-500/15 text-emerald-300" : ""
        }`}
      >
        <span
          className={`w-7 shrink-0 select-none text-right
            active ? "text-emerald-300" : "text-emerald-500"
          }`}
        >
          {active ? "▶" : ln}
        </span>
        <span>{line || " "}</span>
      </div>
    );
  })}
</div>
```

```
{/* панель переменных — «Variables», как в VS Code */}
<div className="shrink-0 max-h-[34%] overflow-hidden" >
  <div className="flex items-center gap-1.5" >
    <Variable className="w-3 h-3 text-indigo-300" />
    Переменные на этом шаге
    <span className="normal-case tracking-normal" />
  </div>
</div>
```

```

        <span className="text-[10.5px] font-bold" style="font-family: monospace;">
            Эталонный код этого демо • только чтение
        </span>
        <div className="ml-auto flex items-center gap-2" style="font-family: monospace;">
            <button
                type="button"
                onClick={() => {
                    setCode(template);
                    setShowRef(false);
                }}
                className="rounded-md bg-indigo-600 hover:bg-indigo-700 text-white px-4 py-2"
            >
                вставить в редактор
            </button>
            <button
                type="button"
                onClick={() => void copyCode()}
                className="rounded-md bg-slate-800 hover:bg-slate-900 text-white px-4 py-2"
            >
                копировать
            </button>
        </div>
    </div>
    <pre className="flex-1 min-h-0 overflow-auto font-family: monospace" style="font-family: monospace;">
        {template}
    </pre>
</div>
) : (
    <textarea
        ref={editorRef}
        value={code}
        onChange={(event) => setCode(event.target.value)}
        onKeyDown={(event) => {
            if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
                event.preventDefault();
                void debugRun();
            }
        }}
    />

```

```
        </span>
      </button>
      {outputOpen && {
        <pre className="flex-1 min-h-[70px] overflow-
          leading-5 text-slate-300">
          {output}
        </pre>
      }}
      <div className="px-3 py-1.5 border-t border-sla
    </div>
  </aside>
);
}
```

ФАЙЛ 59/87: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
  id: string;
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
  { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue-500 to-violet-500', bubbleText: 'Матана!' },
  { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet-500 to-violet-700', bubbleText: 'Вкусную порцию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber-500 to-amber-700', bubbleText: 'Мясо!' }
];
```

```

const preset = PRESETS[counter % PRESETS.length];
counter++;

const newGuest: Customer = {
  id: Date.now().toString(),
  ...preset,
  animState: 'in',
};

setQueue(prev => [...prev, newGuest]);
addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

setTimeout(() => {
  setQueue(prev => prev.map(c => c.id === newGuest.id ? { ...c, animState: 'out' } : c), 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
  if (!isServing) return;
  if (queue.length === 0) {
    setShake(true);
    addLog('⚠ DEQUEUE: В очереди никого нет! Повар скучает');
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsServing(true);
  const headGuest = queue[0];

  addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

  // Step 1: Head guest gets eating animation
  setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'eating' } : c));

  setTimeout(() => {
    // Step 2: Guest leaves with happy face, queue shifts
    setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'out' } : { ...prev[idx - 1], animState: 'eating' }));
  }, 450);
}, [queue, isServing]);

```

```
        Новые гости встают строго <b>в конец</b> оч  
        Здесь нет обхода и блата – только строгий п  
    </p>  
</div>  
</div>
```

```
{/* УПРАВЛЕНИЕ */}  
<div className="flex items-center gap-3 flex-wrap"  
  <button  
    onClick={enqueue}  
    disabled={isServing}  
    className="flex-1 min-w-[150px] bg-gradient-to  
      d:opacity-40 disabled:cursor-not-allowed te  
      ive:scale-95 transition-all cursor-pointer  
  >  
    <span> Встать в очередь (ENQUEUE)</span>  
  </button>  
  
  <button  
    onClick={dequeue}  
    disabled={isServing || queue.length === 0}  
    className="flex-1 min-w-[150px] bg-gradient-to  
      opacity-40 disabled:cursor-not-allowed text  
      :scale-95 transition-all cursor-pointer fle  
  >  
    <span> Налить суп первому (DEQUEUE)</span>  
  </button>  
  
  <button  
    onClick={reset}  
    className="px-5 py-3.5 rounded-2xl bg-slate-8  
      r-pointer border border-slate-700"  
  >  
    Сброс  
  </button>  
</div>  
  
{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
```



```

<div className="flex-1 flex flex-col justify-
  {servedHistory.length === 0 ? (
    <div className="flex-1 flex flex-col item
      <span className="text-5xl mb-2"> </span>
      <p className="text-xs font-bold font-mo
      <p className="text-[10px] mt-1">Здесь 6
    </div>
  ) : (
    <div className="flex flex-col gap-1.5 w-f
      {servedHistory.map((item, idx) => (
        <div
          key={idx}
          className="w-full py-1.5 px-3 round
0 flex items-center gap-2 text-[11px] text-
        >
          <span>{item}</span>
        </div>
      )}}
    </div>
  )}
</div>

{/* Пол */}
<div className="w-full h-3 bg-gradient-to-r f
</div>
</div>

```

```

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
  <div className="text-[10px] font-mono text-slate
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-ce
      >
        {idx === 0 ? <span className="text-indigo-5
        <span>{entry}</span>
      </div>
    )}
  )}

```

```

    буль!'
  });
  const [isTalking, setIsTalking] = useState<boolean>(false);
  const [isBlinking, setIsBlinking] = useState<boolean>(false);

  // Регулярное моргание рыбы
  useEffect(() => {
    const blinkInterval = setInterval(() => {
      setIsBlinking(true);
      setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
  }, []);

  // Эффект разговора при смене текста
  useEffect(() => {
    setIsTalking(true);
    const timer = setTimeout(() => setIsTalking(false), 1000);
    return () => clearTimeout(timer);
  }, [speechText]);

  // Построение начального бора (Trie) без fail-ссылок
  const buildInitialTrie = (wordList: string[]) => {
    const newNodes: { [key: number]: TrieNode } = {
      0: { id: 0, char: 'ROOT', parent: 0, children: {} }
    };
    let count = 0;

    wordList.forEach((word) => {
      let curr = 0;
      for (let i = 0; i < word.length; i++) {
        const c = word[i].toUpperCase();
        if (newNodes[curr].children[c] === undefined) {
          count++;
          newNodes[curr].children[c] = count;
          newNodes[count] = {
            id: count,
            char: c,
            parent: curr,
            children: {}
          };
        }
        curr = newNodes[curr].children[c];
      }
    });
    return newNodes;
  };

  const initialTrie = buildInitialTrie(words);
  const root = initialTrie[0];
  const rootChar = 'ROOT';
  const rootParent = 0;
  const rootChildren = initialTrie[0].children;
  const rootId = 0;

  const [currentNode, setCurrentNode] = useState<TrieNode>(root);
  const [currentChar, setCurrentChar] = useState<string>('');
  const [currentParent, setCurrentParent] = useState<number>(0);
  const [currentId, setCurrentId] = useState<number>(0);
  const [currentChildren, setCurrentChildren] = useState<{ [key: string]: number }>({});
  const [currentCharIndex, setCurrentCharIndex] = useState<number>(0);
  const [currentCharLength, setCurrentCharLength] = useState<number>(0);
  const [currentCharSet, setCurrentCharSet] = useState<Set<string>>(<

```

```

    }
  };

  layoutDFS(0, 0);

  setNodes(newNodes);
  setSimulationStep(0);
  setBfsQueue([]);
  setCurrentNode(null);
  setSpeechText(
    `Отлично! Я построил базовое префиксное дерево (БПД)
    S, чтобы расставить fail-ссылки!`
  );
};

useEffect(() => {
  buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
  e.preventDefault();
  const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
  if (!cleanWord) return;
  if (words.includes(cleanWord)) {
    setSpeechText(`Слово «${cleanWord}» уже есть в слове!`);
    return;
  }
  const updated = [...words, cleanWord];
  setWords(updated);
  setNewWord('');
  buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе мир рухнет!');
    return;
  }
  const updated = words.filter(word => word !== wordToRemove);
  setWords(updated);
  setNewWord('');
  buildInitialTrie(updated);
};

```

```

const updatedNodes = { ...nodes };
const rNode = updatedNodes[r];
const childrenIds = Object.values(rNode.children);

let logs = `Рассматриваем вершину #${r} (${rNode.char}`;

for (const [char, u] of Object.entries(rNode.children)) {
  newQueue.push(u);
  let v = rNode.fail;
  while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
    v = updatedNodes[v].fail;
  }
  const failTarget = updatedNodes[v].children[char];
  updatedNodes[u].fail = failTarget;

  // Рассчитываем терминальную ссылку (term_link)
  if (updatedNodes[failTarget].is_terminal) {
    updatedNodes[u].term_link = failTarget;
  } else {
    updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
  }

  logs += `Для #${u} ('${char}') fail = #${failTarget}`;
}

if (childrenIds.length === 0) {
  logs += 'У этой вершины нет детей, просто переходим к следующему';
}

setNodes(updatedNodes);
setBfsQueue(newQueue);
setCurrentNode(r);
setSpeechText(logs);
};

return (
  <div className="bg-slate-800/90 rounded-2xl p-6 border"
    {/* Шапка виджета */}

```

```
<ellipse cx="100" cy="100" rx="70" ry="45"
<path d="M 40 85 Q 100 55 160 85 Q 100 115
```

```
{/* Хвост */}
```

```
<path
  d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
  fill="#be123c"
  className="origin-[35px_100px] animate-
/>
```

```
{/* Плавники */}
```

```
<path d="M 90 55 L 70 30 L 110 55 Z" fill="
```

```
<path d="M 90 145 L 70 170 L 110 145 Z" f
```

```
<path
  d="M 120 115 L 100 135 L 130 130 Z"
  fill="#f87171"
  className="origin-[120px_115px] animate
```

```
</>
```

```
{/* Глаз (с морганием) */}
```

```
<circle cx="145" cy="85" r="12" fill="#fff
```

```
{isBlinking ? (
```

```
  <line x1="133" y1="85" x2="157" y2="85"
```

```
) : (
```

```
<>
```

```
  <circle cx="148" cy="85" r="6" fill="
```

```
  <circle cx="151" cy="82" r="2" fill="
```

```
</>
```

```
))
```

```
{/* Рот (анимация разговора) */}
```

```
{isTalking ? (
```

```
  <path d="M 165 105 Q 155 120 165 125 Q
```

```
) : (
```

```
  <path d="M 165 110 Q 155 115 168 118" s
```

```
))
```

```
{/* Улыбка / жабры */}
```

```

        <button
          onClick={startBFS}
          className="bg-emerald-600 hover:bg-emerald-700 shadow-lg shadow-emerald-900/40 transition-colors"
        >
          <Play className="w-4 h-4" /> Запустить
        </button>
      )}
      {simulationStep === 1 && (
        <button
          onClick={stepBFS}
          className="bg-blue-600 hover:bg-blue-700 shadow-lg shadow-blue-900/40 transition-colors"
        >
          <span>Шаг BFS</span>
          <span className="bg-blue-800 px-1.5 py-0.5" />
        </button>
      )}
      {simulationStep === 2 && (
        <button
          onClick={() => buildInitialTrie(words)}
          className="bg-indigo-600 hover:bg-indigo-700 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
        >
          <RotateCcw className="w-4 h-4" /> Перезагрузить
        </button>
      )}
    </div>
  </div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800 p-4" >
  <h5 className="text-white font-bold mb-3 flex items-center" >
    <span>Дерево и Суффиксные (fail) ссылки</span>
  </h5>

```

```

        markerEnd="url(#arrowhead)"
    />
    <text
        x={{parent.x + node.x} / 2 - 10}
        y={{parent.y + node.y} / 2}
        fill="#94a3b8"
        fontSize="12"
        fontWeight="bold"
    >
        {node.char}
    </text>
</g>
});
}}

{/* Draw Fail Links */}
{simulationStep > 0 && Object.values(nodes)
    .filter(node => node.id !== 0 || node.fail !== 0)
    .map(node => {
        // Wait, all nodes get fail link. Let's
        // draw all! But maybe wait, we can just draw
        // the first one if step >= 1 and computed it
        // To avoid too many lines, let's draw
        const target = nodes[node.fail];
        if (!target || !node.x || !node.y || !target.x || !target.y) return;

        // Simple bezier curve for fail link
        const isCurrent = currentNode === node;

        return (
            <path
                key={`fail-${node.id}`}
                d={`M ${node.x} ${node.y - 10} Q ${
                    node.x + 10, node.y - 10, node.x + 16
                }`}
                fill="none"
                stroke={isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)'}
                strokeWidth={isCurrent ? 2 : 1.5}
                strokeDasharray="4 4"
                markerEnd="url(#fail-arrow)"
            />
        );
    })
}
```

```

        fill="white"
        fontSize={isRoot ? "10" : "12"}
        fontWeight="bold"
        textAnchor="middle"
      >
        {isRoot ? 'R' : node.char}
      </text>
      {!isRoot && {
        <text
          x={node.x + 12}
          y={node.y - 12}
          fill="#94a3b8"
          fontSize="9"
        >
          {node.id}
        </text>
      }}
    </g>
  );
  }}}
</svg>
);
}}({})
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
  <div className="bg-slate-900/50 p-5 rounded-xl">
    <div>
      <h5 className="text-white font-bold mb-3">
        <span> </span> Словарь (Паттерны)
      </h5>
      <div className="flex flex-wrap gap-2 mb-4">
        {words.map((w) => {
          <span
            key={w}
            className="bg-slate-800 border border-slate-700 padding-1 group"

```



```

        <span> </span> Таблица переходов и fail-
    </span>
    <span className="text-xs text-slate-400 font-
        Всего вершин: {Object.keys(nodes).length}
    </span>
</h5>

<div className="max-h-[220px] overflow-y-auto
    <table className="w-full text-left border-c
        <thead>
            <tr className="border-b border-slate-700
                <th className="py-2 px-3">ID Вершины</th>
                <th className="py-2 px-3">Символ</th>
                <th className="py-2 px-3">Родитель</th>
                <th className="py-2 px-3">fail-ссылка</th>
                <th className="py-2 px-3">term_link</th>
                <th className="py-2 px-3">Дети</th>
                <th className="py-2 px-3">Слова (dict
            </tr>
        </thead>
        <tbody className="divide-y divide-slate-800
            {Object.values(nodes).map((node) => {
                const isCurrent = currentNode === node
                const isInQueue = bfsQueue.includes(n

            return (
                <tr
                    key={node.id}
                    className={`transition-colors ${
                        isCurrent
                            ? 'bg-blue-900/40 font-bold t
                            : isInQueue
                            ? 'bg-slate-800/80'
                            : 'hover:bg-slate-800/40'
                    }`}
                >
                    <td className="py-2.5 px-3 flex i
                        <span

```

```

    }}
  </td>
  <td className="py-2.5 px-3 text-slate-600">
    {Object.keys(node.children).length > 0 ? '0' : Object.entries(node.children)
      .map(([c, id]) => `${c}→#${id}`)
      .join(', ')}
  </td>
  <td className="py-2.5 px-3">
    {node.words.length === 0 ? (
      <span className="text-slate-600">
        {node.words.join(', ')}
      </span>
    ) : (
      <span className="bg-emerald-900 text-white">
        {node.words.join(', ')}
      </span>
    )}
  </td>
</tr>
</tbody>
</table>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 61/87: src/components/SegmentTreeVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

```

```

    "build(v, l, r) {",                // 1
    "  if (l === r) {",                // 2
    "    tree[v] = A[l]; return",      // 3
    "  }",                             // 4
    "  mid = (l + r) >> 1",           // 5
    "  build(2v, l, mid)",             // 6
    "  build(2v+1, mid+1, r)",         // 7
    "  tree[v] = tree[2v]+tree[2v+1]", // 8
    "}",                               // 9
  ];
void CODE;

function go(v: number, l: number, r: number) {
  if (l === r) {
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
      , highlightNodes: [v], arrayHighlight: [l, r] });
    tree[v] = arr[l];
    steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
      l, r] });
    return;
  }
  const m = (l + r) >> 1;
  steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
    { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
  go(2 * v, l, m);
  steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r}`,
    lightNodes: [v], arrayHighlight: [l, r] });
  go(2 * v + 1, m + 1, r);
  tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1]`,
    tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень: ${arr[0]}`,
  Highlight: [0, n - 1] });
return steps;
}

```

```

const tree: Record<number, number> = {};
for (let i = 0; i < n; i++) tree[n + i] = arr[i];
for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

const steps: Step[] = [];
let id = 0;
let l = qL + n;
let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}, codeLine: 3, treeState: tree, highlightNodes: [l, r]` });

while (l < r) {
  const highlights: number[] = [];
  if (l & 1) {
    res += tree[l] ?? 0;
    steps.push({ id: id++, desc: `l=${l} нечётный → l++`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res });
    highlights.push(l);
    l++;
  }
  if (r & 1) {
    r--;
    res += tree[r] ?? 0;
    steps.push({ id: id++, desc: `r=${r + 1} нечётный → r--`, codeLine: 6, treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR], result: res });
    highlights.push(r);
  }
  l >>= 1;
  r >>= 1;
  if (l < r) {
    steps.push({ id: id++, desc: `Поднимаемся: l = qL + 2^k, r = qR + 2^k`, codeLine: 7, treeState: tree, highlightNodes: [l, r], arrayHighlight: [qL, qR], result: res });
  }
}

steps.push({ id: id++, desc: `Запрос sum([qL..qR])`, codeLine: 8, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });

```

```

    "    res = 0;",
    "    l = ql + n; r = qr + n + 1;",
    "    // -----",
    "    while (l < r) {",
    "        if (l & 1) res += tree[l++];",
    "        // -----",
    "        if (r & 1) res += tree[--r];",
    "        l >>= 1; r >>= 1;",
    "    }",
    "    return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
    const [idx, setIdx] = useState(0);
    const [playing, setPlaying] = useState(false);
    const [speed, setSpeed] = useState(700);

    useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

    useEffect(() => {
        if (!playing || idx >= steps.length - 1) {
            if (idx < steps.length - 1) {
                const t = setTimeout(() => setIdx(i => i + 1), speed);
                return () => clearTimeout(t);
            }
        }
    }, [playing, idx, steps, speed]);

    const step = steps[idx] || steps[0];
    if (!step) return null;

    const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, slate: { btn: 'bg-slate-600 hover:bg-slate-500 border-slate-500 bg-slate-600', childRing: 'ring-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' } };
    const [color] = color.split(' ');

    return { step, idx, setIdx, playing, setPlaying, speed, clr };
}

```

```

        </div>
    );
}, [step, clr]);

return (
  <div className={`rounded-2xl border overflow-hidden
    ? 'border-emerald-500/30' : 'border-amber-500/30'
    }/* Header */>
    <div className={`px-4 py-2.5 flex items-center justify-between
      -emerald-950/50' : 'bg-amber-950/50'} border-b border-emerald-500/30`>
      <h4 className="font-bold text-sm text-white flex-1">
        <span className="text-[10px] font-mono text-slate-500">
      </div>

      {/* Array strip */}
      <div className="px-3 py-2 bg-slate-950 border-b border-emerald-500/30">
        {arr.map((v, i) => {
          const inRange = step.arrayHighlight && i >= step.start;
          return (
            <div key={i} className={`flex flex-col items-center justify-between
              go' ? 'bg-indigo-950 border-indigo-500' : 'bg-emerald-950 border-emerald-500'
              er-500'} -translate-y-0.5` : 'bg-slate-900 border-slate-500'}`>
              <span className="text-[7px] text-slate-500">
              <span className="text-[11px] font-bold font-sans text-white">
            </div>
          );
        })}
      </div>

      {/* Tree */}
      <div className="flex-1 p-2 overflow-x-auto bg-slate-950">
        <div className="scale-[0.85] origin-top min-w-max">
      </div>

      {/* Code */}
      <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
        x">
        {code.map((line, i) => {

```

```

        button>
      </div>
      <select value={speed} onChange={e => setSpeed(N
        text-[9px] font-bold rounded px-1.5 py-1 outl
        <option value={1200}>Медл.</option>
        <option value={700}>Норма</option>
        <option value={350}>Быстро</option>
        <option value={120}>Турбо</option>
      </select>
    </div>

    {/* Progress bar */}
    <div className="h-1 bg-slate-950"><div className=
      + 1) / total) * 100 : 0}%` }} /></div>
  </div>
  );
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12
  const [customInput, setCustomInput] = useState('5, 8,
  const [qL, setQL] = useState(1);
  const [qR, setQR] = useState(4);
  const [view, setView] = useState<'build' | 'queries'

  // Clamp query range when array changes
  useEffect(() => {
    setQL(prev => Math.min(prev, arr.length - 1));
    setQR(prev => Math.min(prev, arr.length - 1));
  }, [arr]);

  const handleApply = (e: React.FormEvent) => {
    e.preventDefault();
    const parsed = customInput.split(',').map(s => pars
    if (parsed.length >= 2 && parsed.length <= 16) {

```

```

        <button type="submit" className="bg-indigo-500 text-white py-2 px-4">Применить</button>
      </div>
      <div className="flex flex-wrap gap-1.5 pt-1">
        {arr.map((v, i) => (
          <span key={i} className="bg-slate-900 border border-indigo-500 padding-2">
            <span className="text-slate-500">A[{i}</span>
            </span>
          )
        )}
        <span className="text-xs text-slate-500 font-weight-normal">
      </div>
    </form>
  )
}

```

```

  { /* Query range */
    <div className="bg-slate-950 p-4 rounded-xl border border-indigo-500">
      <label className="text-[10px] font-bold uppercase">
        <div className="flex items-center gap-2">
          <div className="flex items-center gap-1">
            <span className="text-xs text-slate-400">Left</span>
            <input
              type="number"
              min={0}
              max={arr.length - 1}
              value={qL}
              onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
              className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
            />
          </div>
          <div className="flex items-center gap-1">
            <span className="text-xs text-slate-400">Right</span>
            <input
              type="number"
              min={0}
              max={arr.length - 1}
              value={qR}
              onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
              className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
            />
          </div>
        </div>
      </label>
    </div>
  )
}

```



```

        color="indigo"
        arr={arr}
      />
    })

{view === 'queries' && {
  <div className="grid grid-cols-1 xl:grid-cols-2">
    <SimPanel
      key={`qtd-${arr.join(' - ')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Сверху`}
      icon="↑"
      steps={queryTDSteps}
      code={QUERY_TD_CODE}
      color="emerald"
      arr={arr}
    />
    <SimPanel
      key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Снизу`}
      icon="↓"
      steps={queryBUSteps}
      code={QUERY_BU_CODE}
      color="amber"
      arr={arr}
    />
  </div>
})

{view === 'theory' && {
  <div className="space-y-5">
    {/* Why it splits this way */}
    <div className="bg-slate-950 p-5 rounded-xl border">
      <h4 className="text-base font-bold text-white">
        Почему разбиение именно так для N={arr.length}?
      </h4>
      <p className="text-sm text-slate-300 leading-relaxed">
        Каждый узел берёт <code className="bg-slate-900 px-1">
        и получает <code className="bg-slate-900 px-1">
        <code className="bg-slate-900 px-1 rounded font-mono text-emerald-400">

```



```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft } from 'lucide-react';

interface Plate {
  id: string;
  name: string;
  icon: string;
}

interface Person {
  id: string;
  name: string;
  icon: string;
}

interface Hypothesis {
  id: string;
  text: string;
  probability: number;
}

export const Simulator: React.FC = () => {
  const [activeTab, setActiveTab] = useState<'stack' | 'queue' | 'hypothesis'>('stack');

  // Stack state
  const [stack, setStack] = useState<Plate[]>([
    { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
    { id: '3', name: 'Блюдце от торта', icon: '🍰' },
  ]);

  const [popMessage, setPopMessage] = useState<string | null>(null);

  // Queue state
  const [queue, setQueue] = useState<Person[]>([
    { id: '1', name: 'Студент Вася', icon: '👤' },
  ]),
```

```

    }
    const topPlate = stack[stack.length - 1];
    setStack(prev => prev.slice(0, prev.length - 1));
    setPopMessage(`  Вымыта верхняя тарелка (LIFO): ${topPlate}`);
  });

const resetStack = () => {
  setStack([
    { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
    { id: '3', name: 'Блюде от торта', icon: '🍰' },
  ]);
  setPopMessage("  Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
  const presets = [
    { name: 'Любитель борща', icon: '🥘' },
    { name: 'Курьер доставки', icon: '📦' },
    { name: 'Усталый админ', icon: '🧑‍💻' },
    { name: 'Турист с рюкзаком', icon: '🎒' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPerson = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setQueue(prev => [...prev, newPerson]);
  setDequeueMessage(`  Встал в конец очереди: ${newPerson.name} ${newPerson.icon}`);
};

const servePerson = () => {
  if (queue.length === 0) {
    setDequeueMessage("⚠  Очередь пуста! Суп ждет нов");
    return;
  }

```



```

        activeTab === 'heap'
        ? 'bg-emerald-600/20 border-emerald-500 te
        : 'bg-slate-800/60 border-slate-700/60 te
    }` }
>
<div className="flex items-center gap-3">
  <Award className={`w-5 h-5 ${activeTab ===
  <div className="text-left">
    <div className="font-bold text-sm text-wh
    <div className="text-xs opacity-80">Приор
  </div>
</div>
<span className="text-xs bg-emerald-500/20 te
  Beam Search
</span>
</button>
</div>

{/* STACK TAB */}
{activeTab === 'stack' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
    tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция стопки тарелок</span>
          <span className="text-xs font-mono bg-b
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Новые тарелки ставятся наверх. Мыть мож
        </p>
      </div>
    <div className="flex flex-wrap items-center
      <button
        onClick={addPlate}
        className="flex-1 md:flex-none flex ite
        rounded-xl text-sm font-bold shadow-lg tra
      >

```

```

<div className="w-full max-w-sm flex flex-direction-reverse justify-between" style={
  <div className="absolute top-0 text-xs font-mono" style={
    full flex items-center gap-1">
      <span>ВЕРШИНА СТЕКА (TOP)</span>
      <ArrowDown className="w-3.5 h-3.5 animate-pulse" style={
    </div>

    {/* Reversed map so top of stack is visible */}
    {stack.slice().reverse().map((plate, index) => {
      const isTop = index === 0;
      return (
        <div
          key={plate.id}
          className={`w-full flex items-center justify-between border-1
            isTop
              ? 'bg-gradient-to-r from-blue-500 to-blue-600'
              : 'bg-slate-900/90 border-slate-700'}
          >
          <div className="flex items-center justify-between" style={
            <span className="text-2xl">{plate.name}</span>
            <span className={`font-bold text-white`} style={
              {plate.name}
            </span>
          </div>
          <div>
            <span className={`text-xs font-mono`} style={
              isTop ? 'bg-blue-500 text-white' : 'bg-slate-900 text-slate-500'
            >{isTop ? 'Готово к помывке' : 'Помыт'}</span>
          </div>
        </div>
      );
    })}
  </div>
)}
<div className="mt-6 text-xs text-slate-500">
  ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •

```



```

        className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
      >
        <RotateCcw className="w-5 h-5" />
      </button>
    </div>
  </div>

  {dequeueMessage && {
    <div className="bg-indigo-950/60 border bor
    p-3 animate-fade-in">
      <span className="inline-block w-2 h-2 roun
      {dequeueMessage}
    </div>
  }}

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2
  overflow-x-auto">
    {queue.length === 0 ? {
      <div className="text-center py-12 text-sl
      <div className="text-5xl mb-3"> </div>
      <p className="text-base font-bold">Очер
      <p className="text-xs mt-1">Добавьте ро
    </div>
    } : {
      <div className="w-full flex items-center
      <div className="flex flex-col items-cen
    300">
      <span className="text-3xl"> </span>
      <span className="text-xs font-bold for
    </div>

    <div className="text-indigo-500 flex it
      <ArrowRight className="w-6 h-6 animat
    </div>

    {queue.map((person, index) => {

```

```
        ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
    </div>
  </div>
</div>
})

{/* HEAP TAB */}
{activeTab === 'heap' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Приоритетная очередь / Куча</sp
          <span className="text-xs font-mono bg-e
x-Heap</span>
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Здесь не важно, кто пришел первым. Важн
        </p>
      </div>
      <div className="flex items-center gap-3 w-f
        <button
          onClick={popHeap}
          className="flex-1 md:flex-none flex ite
py-2.5 rounded-xl text-sm font-bold shadow-
        >
          Взять кандидата с макс. вероятностью
        </button>
        <button
          onClick={resetHeap}
          title="Сбросить"
          className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
      </div>
    </div>
```

```

        <Plus className="w-4 h-4" /> Добавить в
    </button>
</div>
</form>

{heapMessage && {
  <div className="bg-emerald-950/60 border bo
    gap-3 animate-fade-in">
    <span className="inline-block w-2 h-2 roun
      {heapMessage}
    </div>
  }}
}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
  >
  {sortedHeap.length === 0 ? {
    <div className="text-center py-12 text-sl
      <div className="text-5xl mb-3"> </div>
      <p className="text-base font-bold">Куча
      <p className="text-xs mt-1">Добавьте ги
    </div>
  } : {
    <div className="w-full max-w-2xl space-y-
      <div className="flex items-center justifi
        <span>КАНДИДАТ / ГИПОТЕЗА</span>
        <span>ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)</span>
      </div>

    {sortedHeap.map(({item, index}) => {
      const isTop = index === 0;
      const percent = (item.probability * 100).toFixed(2);
      return {
        <div
          key={item.id}
          className={`flex items-center justify-between
            isTop
              ? 'bg-gradient-to-r from-emerald-500 to-emerald-200'
            : 'bg-slate-900'
          }
        </div>
      }
    )}
  }
}

```

```
        });
      }}}
    </div>
  })
  <div className="mt-6 text-xs text-slate-500" >
    НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
  </div>
</div>
</div>
  })
</div>
};
};
```

ФАЙЛ 64/87: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";
```

```
interface Props {
  /** Открыть тренажёр в модальном окне. */
  onOpen: (vizId: string) => void;
  /** Перейти к теме, к которой привязан тренажёр. */
  onGoChapter: (chapterId: string) => void;
}
```

```
/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно поты
 */
```

```

    />
    {query && (
      <button
        onClick={() => setQuery("")}
        className="absolute right-2 top-1/2 -translate-x-1/2
        aria-label="Очистить поиск"
      >
        <X className="w-4 h-4" />
      </button>
    )}
  </div>

  {items.length === 0 && <p className="text-slate-500">
    Нет результатов
  </p>}

  <div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-3">
    {items.map(({ id, entry, chapterTitle }) => (
      <div
        key={id}
        className="flex flex-col bg-slate-900 border border-slate-800
        >
        <h3 className="font-bold text-white text-sm">
          {entry.title}
          {entry.hint && <p className="text-[12px] text-slate-400">
            {entry.hint}
          </p>
          <div className="flex-1" />
        </h3>
        <div className="flex items-center gap-2 mt-2">
          <button
            onClick={() => onOpen(id)}
            className="flex items-center gap-1.5 text-slate-400
            transition-colors"
            >
            <Play className="w-3.5 h-3.5" /> Закрыть
          </button>
          {chapterTitle && (
            <button
              onClick={() => onGoChapter(id)}
              title={chapterTitle}
              className="flex items-center gap-1.5 text-slate-400
              transition-colors min-w-[100px] hover:text-white"
            >
              <X className="w-3.5 h-3.5" />
              {chapterTitle}
            </button>
          )}
        </div>
      </div>
    )}
  </div>

```

```

const clone = (t: TNode | null): TNode | null =>
  t ? { key: t.key, left: clone(t.left), right: clone(t
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
  {
    name: "Линия влево",
    hintCase: "Zig-Zig",
    target: 1,
    build: () => {
      const n1 = leaf(1);
      const n2: TNode = { key: 2, left: n1, right: leaf
      const n4: TNode = { key: 4, left: n2, right: leaf
      return { key: 6, left: n4, right: leaf(7) };
    },
  },
  {
    name: "Змейка",
    hintCase: "Zig-Zag",
    target: 3,
    build: () => {
      const n3 = leaf(3);
      const n2: TNode = { key: 2, left: leaf(1), right:
      const n4: TNode = { key: 4, left: n2, right: leaf
      return { key: 6, left: n4, right: leaf(7) };
    },
  },
  {
    name: "Один уровень",
    hintCase: "Zig",
    target: 2,
    build: () => ({ key: 4, left: { key: 2, left: leaf(
  },
  {
    name: "Смешанный",
    hintCase: "Zig-Zag, затем Zig-Zig",
    target: 5,

```

```

    if (!gg) return x;
    if (gg.left === p) gg.left = x;
    else gg.right = x;
    return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
    const path = findPath(root, key);
    if (path.length < 2) return null;
    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const g = path.length >= 3 ? path[path.length - 3] : null;

    if (!g) return { node: x.key, kind: "Zig" };

    const xLeft = p.left === x;
    const pLeft = g.left === p;
    if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
    return { node: x.key, kind: "Zig-Zag (сначала нижняя)" };
}

/** ----- раскладка -----

interface Placed {
    key: number;
    x: number;
    y: number;
    depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
    const nodes: Placed[] = [];
    const edges: [number, number][] = [];
    let order = 0;

    const walk = (n: TNode | null, depth: number) => {

```

```

        setTree(PRESETS[idx].build());
        setHistory([]);
        setMoves([]);
        setShowAnswer(false);
    },
    [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)));
const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)));

const clickNode = (key: number) => {
    if (solved) return;
    const p = findPath(tree, key);
    if (p.length < 2) return; // корень крутить не с кем

    const want = canonicalNext(tree, target);
    setHistory((h) => [...h, clone(tree) as TNode]);
    setMoves((m) => [...m, { node: key, correct: want?.length === p.length }]);
    setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
    if (history.length === 0) return;
    setTree(history[history.length - 1]);
    setHistory((h) => h.slice(0, -1));
    setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return (
    <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
        <div className="flex items-start justify-between p-4">
            <div>
                <h4 className="text-sm sm:text-base font-bold">

```



```

    const nb = nodes.find((n) => n.key === b);
    if (!na || !nb) return null;
    const onPath = path.has(a) && path.has(b);
    return {
      <line
        key={i}
        x1={na.x}
        y1={na.y}
        x2={nb.x}
        y2={nb.y}
        stroke={onPath ? "#6366f1" : "#475569"}
        strokeWidth={onPath ? 3 : 1.8}
        style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
      />
    );
  }}}

{nodes.map((n) => {
  const isTarget = n.key === target;
  const clickable = !solved && n.depth > 0;
  return {
    <g
      key={n.key}
      onClick={() => clickable && clickNode(n)}
      style={{
        transform: `translate(${n.x}px, ${n.y}px)`,
        transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
        cursor: clickable ? "pointer" : "default",
      }}
    >
      {isTarget && {
        <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
      }}
      <circle
        r={16}
        fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475569"}
        stroke={isTarget ? "#a5b4fc" : "#475569"}
        strokeWidth={2}
      />
    </g>
  }
})}
```

```

    <span className="text-[11px] text-slate-500 ml-1" style={{
      поворотов: {moves.length} • минимум: {minimal}
    }} />
  </span>
</div>

{showAnswer && !solved && expected && (
  <div className="mt-3 text-[12px] bg-amber-950/40" style={{
    Сейчас splay крутил бы узел{" "}
    <span className="font-mono font-bold">{expected}
  }} />
)}

{solved && (
  <div
    className={`mt-3 rounded-lg border px-3 py-2.5`}
    wrong === 0
      ? "bg-emerald-950/40 border-emerald-700/60"
      : "bg-rose-950/30 border-rose-800/60 text-rose-50"
  >
    <div className="flex items-center gap-2 font-mono" style={{
      {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : null}
      {wrong === 0
        ? `Узел ${target} в корне за ${moves.length} ходов`
        : `Узел ${target} в корне за ${moves.length} ходов`
      }
    }} />
    </div>
    {wrong > 0 && (
      <ul className="list-disc pl-5 space-y-0.5">
        {moves.map(
          (m, i) =>
            !m.correct && (
              <li key={i}>
                ход {i + 1}: крутили <span className="font-mono">{m.expected}
              </li>
            )
        )}
      </ul>
    )}
  )}
</div>

```

```

pos: Pos;
edges: [NodeId, NodeId][];
kind: "start" | "aim" | "result";
title: string;
text: string;
/** Пара, которую крутим на этом шаге: кадр-«прицел».
focus?: [NodeId, NodeId];
/** Кто переехал на этом кадре – рисуем призрак и стрелу.
moved?: NodeId[];
ms: number;
}

interface Case {
key: "zig" | "zig-zig" | "zig-zag";
label: string;
when: string;
rotations: string;
/** Номера узлов: роль → число. */
keys: Partial<Record<NodeId, number>>;
/** Обход слева направо – одинаковый на всех кадрах.
inorder: string[];
/** Что означают поддеревья A..D. */
ranges: string;
frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {

```

```
pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZZ_S1 = {
pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZIGZIG: Case = {
key: "zig-zig",
label: "Zig-Zig",
when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
rotations: "2 поворота, первым – верхний",
keys: { x: 20, p: 40, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
{
...ZZ_S0,
kind: "start",
title: "Было: прямая линия 60 → 40 → 20",
text: "Три узла выстроились в «бамбук»: каждый следующий узел  
зглаживает.",
ms: RESULT_MS,
},
{
...ZZ_S0,
kind: "aim",
title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
text: "Главная хитрость этого случая: первым крутит  
focus: ["g", "p"],
ms: AIM_MS,
},
{
...ZZ_S1,
kind: "result",
title: "Поворот 1: 40 поднялось над 60",
```

```

key: "zig-zag",
label: "Zig-Zag",
when: "x и p – дети с РАЗНЫХ сторон (змейка)",
rotations: "2 поворота, первым – нижний",
keys: { x: 40, p: 20, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
    {
        ...ZG_S0,
        kind: "start",
        title: "Было: 60 → влево на 20 → вправо на 40",
        text: "Узлы идут «змейкой»: сначала шаг влево, по",
        ms: RESULT_MS,
    },
    {
        ...ZG_S0,
        kind: "aim",
        title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
        text: "Дед (60) стоит на месте и в этом повороте",
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        ...ZG_S1,
        kind: "result",
        title: "Поворот 1: змейка выпрямилась",
        text: "40 заняло место 20 и забрало его к себе ле",
        moved: ["x", "p", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZG_S1,
        kind: "aim",
        title: "Прицел 2: ребро 60 – 40",
        text: "Остался одиночный поворот вокруг деда – и",
        focus: ["g", "x"],
    }
]

```

```

<svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
  <defs>
    <marker id="splay-arrow" viewBox="0 0 10 10" re
      <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
    </marker>
  </defs>

```

{/ призраки: где узел стоял до поворота */}*

```

{ghosts.map((id) => {
  const from = prev[id]!;
  const to = pos[id]!;
  const r = isSubtree(id) ? 15 : 20;
  const dx = to[0] - from[0];
  const dy = to[1] - from[1];
  const len = Math.hypot(dx, dy) || 1;
  const pad = r + 8;
  return (
    <g key={`ghost-${id}`} opacity={0.55}>
      <circle cx={from[0]} cy={from[1]} r={r} fill="#a5b4fc" />
      {len > pad * 2 + 6 && (
        <line
          x1={from[0] + (dx / len) * pad}
          y1={from[1] + (dy / len) * pad}
          x2={to[0] - (dx / len) * pad}
          y2={to[1] - (dy / len) * pad}
          stroke="#a5b4fc"
          strokeWidth={1.5}
          strokeDasharray="4 3"
          markerEnd="url(#splay-arrow)"
        />
      )}
    </g>
  );
})}

```

```

{edges.map(([a, b], i) => {
  const pa = pos[a];
  const pb = pos[b];

```

```

        fontSize={sub ? 12 : 15}
        fontWeight="700"
        fill={sub ? "#94a3b8" : "#fff"}
      >
        {sub ? id : keys[id]}
      </text>
    {!sub && (
      <text textAnchor="middle" y={-r - 7} font
        {id}
      </text>
    )}
  </g>
);
}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const [caseIdx, setCaseIdx] = useState(0);
  const [frame, setFrame] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [slow, setSlow] = useState(false);

  const active = CASES[caseIdx];
  const total = active.frames.length;
  const idx = Math.min(frame, total - 1);
  const current = active.frames[idx];
  const prevPos = active.frames[Math.max(idx - 1, 0)].p
  const duration = useMemo(() => Math.round(current.ms
  const last = idx === total - 1;

  useEffect(() => {
    setFrame(0);
  }, [caseIdx]);

  useEffect(() => {
    if (!playing) return;

```

```

    </span>
    <span className="text-sm font-bold text-white" style={{fontStyle: 'italic'}}>
      Шаг {idx + 1} из {total}. {current.title}
    </span>
  </div>
  <p className="text-[13px] text-slate-400 leading-5">
    </div>

  <div className="bg-slate-900 rounded-xl border border-slate-800" style={{borderRadius: 10px}}>
    <Tree frame={current} prev={prevPos} keys={activeKeys} />

    {/* обход слева направо – одинаковый на всех кадрах */}
    <div className="mt-2 pt-2 border-t border-slate-800" style={{borderTop: '1px solid #33415d'}}>
      <div className="flex items-center gap-1.5 flex-wrap">
        <span className="text-slate-500 mr-1">Обход</span>
        {active.inorder.map((t, i) => {
          <span
            key={i}
            className={`px-1.5 py-0.5 rounded font-mono text-[10px]
              /\d/.test(t) ? "bg-emerald-500/15 text-emerald-400" : "text-slate-500"
            }`}
            >
            {t}
          </span>
        ))}
        <span className="text-emerald-400/80 ml-1">Итого: {active.inorder.length}</span>
      </div>
      <p className="text-[11px] text-slate-500 mt-1">
        </div>
      </div>

      {/* индикатор автопрокрутки */}
      <div className="h-1 w-full bg-slate-800 rounded-full" style={{height: 10px}}>
        <div
          key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
          className="h-full bg-indigo-500"
          style={{
            width: playing ? 100 : 0,
            transition: playing ? 'width 0.5s ease' : 'width 0.5s ease',
          }}
        </div>
      </div>
    </div>
  </div>

```



```

        title="Автопрокрутка ещё медленнее"
        className={`flex items-center gap-1.5 px-3 py-3
          slow
            ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
            : "bg-slate-800 border-slate-700 text-slate-500"
          }`}
      >
        <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5" : "1"}
      </button>
    <button
      onClick={() => {
        setFrame(0);
        setPlaying(false);
      }}
      className="flex items-center gap-1.5 px-3 py-3 text-xs font-bold transition-colors"
    >
      <RotateCcw className="w-3.5 h-3.5" /> Сброс
    </button>

    <div className="flex items-center gap-3 ml-auto"
      {[["x", "p", "g"] as const].map((id) =>
        active.keys[id] === undefined ? null : (
          <span key={id} className="flex items-center gap-1"
            <span className="w-3 h-3 rounded-full"
              {id === "x" ? "bg-emerald-500" : "bg-slate-500"}
            </span>
            <span>{id}</span>
          </span>
        )
      )}
    </div>
  </div>

  <p className="mt-2 text-[11px] text-slate-500">
    Листай кнопкой «Дальше» в своём темпе – кадры не
  </p>
</div>
);
};

```

```
    return y;
  };
```

```
// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
  const y = x.right;
  if (!y) return x;
  x.right = y.left;
  y.left = x;
  return y;
};
```

```
// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
  if (!node) return null;
  return {
    key: node.key,
    left: cloneTree(node.left),
    right: cloneTree(node.right),
    x: node.x,
    y: node.y,
  };
};
```

```
export const SplayTreeViz: React.FC = () => {
  // Let's create an elegant initial unbalanced or semi-balanced tree
  // Values: 10, 20, 30, 40, 50, 60
  const createInitialTree = (): SplayNode => {
    let r: SplayNode | null = null;
    const initialKeys = [40, 20, 50, 10, 30, 60];
    initialKeys.forEach((key) => {
      r = insertBST(r, key);
    });
    return r!;
  };
};
```

```
const [tree, setTree] = useState<SplayNode>(createInitialTree());
const [inputValue, setInputValue] = useState<string>('');
```

```

        steps.push(`Поиск ${key}: элемент отсутствует в д
        steps.push(
            `Поиск споткнулся на узле [${lastNode.key}]. И
        );
    } else {
        steps.push(
            `Поиск ${key}: элемент найден! Запускаем подъем
        );
    }
}

```

// Now let's implement standard Splay logic recursively

```

const splayAction = (
    node: SplayNode | null,
    k: number,
): SplayNode | null => {
    if (!node || node.key === k) return node;

    if (k < node.key) {
        if (!node.left) return node;

        if (k < node.left.key) {
            // Zig-Zig (Left-Left)
            node.left.left = splayAction(node.left.left,
            node = rightRotate(node);
            steps.push(
                `Вращение Zig-Zig (Правый поворот родителя
            );
        } else if (k > node.left.key) {
            // Zig-Zag (Left-Right)
            node.left.right = splayAction(node.left.right
            if (node.left.right) {
                node.left = leftRotate(node.left);
                steps.push(`Компонент Zig-Zag: левый поворо
            }
        }
    }

    if (!node.left) return node;
    steps.push(

```

```

        setTree(newRoot);
    }
    setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
    e.preventDefault();
    const val = parseInt(inputValue);
    if (isNaN(val)) return;

    // Standard BST insert, then splay!
    let newTree = cloneTree(tree);
    newTree = insertBST(newTree, val);
    const { newRoot, steps } = splayStepByStep(newTree,
    if (newRoot) {
        setTree(newRoot);
    }
    setLog([`Вставили вершину [${val}].`, ...steps]);
    setInputValue("");
    setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
    e.preventDefault();
    const val = parseInt(inputValue);
    if (isNaN(val)) return;
    handleSplay(val);
    setInputValue("");
};

const resetTree = () => {
    setTree(createInitialTree());
    setHighlightedNode(null);
    setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive

```

```

    if (node.right && node.right.x && node.right.y) {
      lines.push(
        <line
          key={`l-${node.key}-${node.right.key}`}
          x1={node.x}
          y1={node.y}
          x2={node.right.x}
          y2={node.right.y}
          stroke="#475569"
          strokeWidth="2"
        />,
      );
      lines = lines.concat(renderLines(node.right));
    }
    return lines;
  };

const renderNodes = (node: SplayNode | null): React.R
  if (!node) return [];
  let circles: React.ReactNode[] = [];
  const isHighlighted = highlightedNode === node.key;

  circles.push(
    <g
      key={`n-${node.key}`}
      className="cursor-pointer group"
      onClick={() => handleSplay(node.key)}
      data-title={`Кликните, чтобы выполнить Splay(${
    >
    <circle
      cx={node.x}
      cy={node.y}
      r="18"
      fill={isHighlighted ? "#4f46e5" : "#1e293b"}
      stroke={isHighlighted ? "#818cf8" : "#64748b"}
      strokeWidth={isHighlighted ? "3" : "2"}
      className="transition-all duration-300 group-l
    />
  
```

```

        viewBox="0 0 600 320"
      >
        {renderLines(drawTree)}
        {renderNodes(drawTree)}
      </svg>

    {/* Quick interactive controls */}
    <div className="absolute bottom-4 left-4 right-4" >
      <form onSubmit={handleFind} className="flex" >
        <input
          type="number"
          placeholder="Поиск..."
          value={inputValue}
          onChange={(e) => setInputValue(e.target.value)}
          className="w-24 bg-slate-950 text-white"
        />
        <button
          type="submit"
          onClick={handleFind}
          className="bg-indigo-600 hover:bg-indigo-500"
        >
          Поиск / Splay
        </button>
        <button
          type="button"
          onClick={(e) => {
            e.preventDefault();
            const val = parseInt(inputValue);
            if (!isNaN(val)) {
              let newTree = cloneTree(tree);
              newTree = insertBST(newTree, val);
              const { newRoot, steps } = splayStep(newTree, val);
              if (newRoot) setTree(newRoot);
              setLog([`Вставили [${val}].`, ...steps]);
              setInputValue("");
              setHighlightedNode(val);
            }
          }}
        >
          Вставить
        </button>
      </form>
    </div>
  )}
  </div>
</div>

```

```

<div className="border-t border-slate-800 pt-3" style={{border: 1px solid #444; padding-top: 10px;}}
  <p>
    <span className="font-semibold text-slate-400">
      у самого корня. Делаем 1 вращение.
    </span>
  </p>
  <p>
    <span className="font-semibold text-slate-400">
      оба левые или оба правые. Вращаем деда, по
    </span>
  </p>
  <p>
    <span className="font-semibold text-slate-400">
      колено. Вращаем узел в разные стороны.
    </span>
  </p>
</div>
</div>
</div>

```

```

{/* Answers Section inside the visualizer widget */}
<div className="bg-slate-950/80 p-5 border-t border-slate-800" style={{background: 'linear-gradient(to top, #000 49%, #333 49% 51%, #333 51% 53%, #000 53%); border-top: 1px solid #444; padding: 5px 0 0 0;}}
  <h4 className="text-indigo-400 font-bold text-slate-300">
    <HelpCircle className="w-4 h-4 text-indigo-400"/>
    Разбор экзаменационных вопросов (Без нитья):
  </h4>

```

```

<div className="grid grid-cols-1 md:grid-cols-3" style={{display: 'grid; grid-template-columns: 1fr; min-height: 300px;}}
  <div className="bg-slate-900/60 p-4 rounded-l" style={{background: 'linear-gradient(to top, #000 49%, #333 49% 51%, #333 51% 53%, #000 53%); border-left: 1px solid #444; padding: 4px 0 0 0;}}
    <div>
      <p className="font-bold text-slate-300 mb-2">
        1. Отсутствующий элемент
      </p>
      <p className="text-slate-400 leading-relaxed">
        Допустим, мы ищем{" "}
        <code className="text-indigo-400 font-sans">{" "}</code>
        его нет. Алгоритм спустится к листу, на
        неудачей {например, {" "}}
        <code className="text-indigo-400">[20]</code>
        <code className="text-indigo-400">[30]</code>
        поднимет в корень{" "}

```



```

    return;
  }

  const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
  counter++;

  const newPlate: Plate = {
    id: Date.now().toString(),
    ...preset,
    isDirty: true,
    animState: 'in',
  };

  setDirtyStack(prev => [...prev, newPlate]);
  addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.id}`);

  setTimeout(() => {
    setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : null));
  }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    addLog('⚠ POP: Нечего мыть! Все тарелки чистые.');
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);

  // Step 1: Start washing animation (sponge and soap)
  setDirtyStack(prev => prev.map(p => p.id !== topPlate.id ? p : null));

```

```

{ /* МНЕМОНИКА / ПРАВИЛО */ }
<div className="bg-blue-950/40 border border-blue-950" >
  <div className="text-3xl" > </div>
  <div>
    <h3 className="font-black text-blue-400 text-2xl" >
      <p className="text-xs text-slate-300 mt-1 leading-4" >
        Ты складываешь грязные тарелки друг на друга.
        А когда приходит время мыть, ты берёшь именно самую верхнюю.
        Попытаешься вытащить нижнюю – всё рухнет!
      </p>
    </div>
  </div>

{ /* УПРАВЛЕНИЕ */ }
<div className="flex items-center gap-3 flex-wrap" >
  <button
    onClick={pushPlate}
    disabled={isWashing}
    className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
      opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
      rounded-2xl px-3 py-2.5 transition-all cursor-pointer flex items-center justify-center"
  >
    <span> Положить грязную (PUSH)</span>
  </button>

  <button
    onClick={popPlate}
    disabled={isWashing || dirtyStack.length === 0}
    className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
      opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
      rounded-2xl px-3 py-2.5 transition-all cursor-pointer flex items-center justify-center"
  >
    <span> Помыть верхнюю (POP)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white text-sm font-medium"
  >
    <span> Сбросить (RESET)</span>
  </button>
</div>

```



```

        key={plate.id}
        className="w-full h-8 rounded-full flex items-center justify-center gap-2 opacity-100"
      >
        <span className="text-sm"> {plate.name}</span>
        <span className="text-[10px] text-end"> {plate.id}</span>
      </div>
    </div>
  </div>

  {/* Столешница сушилки */}
  <div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800">
  </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4">
  <div className="text-[10px] font-mono text-slate-200">
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-center justify-between gap-2`}
      >
        {idx === 0 ? <span className="text-blue-500"> Действие: </span>
        <span>{entry}</span>
      </div>
    ))}
  </div>
</div>
);
};

```

```

        if (j === pattern.length) {
            steps.push({ i, j, pi: [...pi], found: true });
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string) {
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=$` });

        if (i <= r) {
            steps.push({ i, l, r, z: [...z], desc: `Внимание!` });
            z[i] = Math.min(r - i + 1, z[i - l]);
            steps.push({ i, l, r, z: [...z], desc: `Предел. Копипаста сработала.` });
        }

        steps.push({ i, l, r, zTarget: z[i], zSource: i });
        while (i + z[i] < n && s[z[i]] === s[i + z[i]]) {
            steps.push({ i, l, r, zTarget: z[i], zSource: i });
            падают! Окно растёт.` });
            z[i]++;
        }

        if (i + z[i] < n) {
            steps.push({ i, l, r, zTarget: z[i], zSource: i });
        }
    }
}

```

```
// Reset when inputs or mode change
useEffect(() => {
  setStepIdx(0);
  setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
  if (autoPlay) {
    timer.current = setInterval(() => {
      setStepIdx(prev => {
        if (prev >= steps.length - 1) { set
        return prev + 1;
      });
    }, 1200);
  }
  return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
```

```

// Z Logic
if (mode == "z") {
    if (index >= step.l && index <= step.r) {
        bgColor = "bg-emerald-900";
        borderColor = "border-emerald-900";
    }
    if (step.zTarget == index) {
        borderColor = "border-amber-500";
        bgColor = step.match == "z" ? "bg-amber-500" : "bg-emerald-900/40");
    }
}

return (
    <div key={index} className={
        /* L/R markers for Z */
        {mode == "z" && step.l <= index && index <= step.r ? "border-l-2 border-r-2" : ""}
        <div className="absolute left-0 right-0 top-0 bottom-0 border border-dashed border-emerald-900" />
    )}
    {mode == "z" && step.r <= index && index <= step.l ? "border-l-2 border-r-2" : ""}
    <div className="absolute left-0 right-0 top-0 bottom-0 border border-dashed border-emerald-900" />
    )}

    /* Index */
    <span className={`text-${mode == "z" ? "emerald-900" : "gray-400"} font-mono font-bold`} />

    /* Char Box */
    <div className={`w-8 h-8 flex items-center justify-center text-${mode == "z" ? "emerald-900" : "gray-400"} font-mono text-lg transition-colors duration-200`} />
    {char}
    </div>

    /* Value array box */
    <div className="text-[10px] font-mono font-bold" />
    {mode == "kmp" ? step.kmp : step.zTarget}
    </div>

```

```

        { /* Found flash effect
        {step.found && mode ===
            <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
        }}
        {step.found && mode ===
            <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
        }}
    </div>
    )
    }}}
</div>
</div>

<div className="flex flex-col sm:flex-row g
    { /* Controls */
    <div className="flex bg-slate-900 borde
        <button onClick={prevStep} disabled
rounded-lg disabled:opacity-30 disabled:hov
            <Undo strokeWidth={3} size={16}
        </button>
        <button onClick={playPause} classNam
x items-center justify-center min-w-[80px]"
            {autoplay ? <><Pause size={16} <
        </button>
        <button onClick={nextStep} disabled
r:bg-slate-800 rounded-lg disabled:opacity-
            <SkipForward strokeWidth={3} si
        </button>
        <button onClick={reset} className="
border-slate-800">
            <RefreshCw strokeWidth={3} size
        </button>
    </div>

    { /* Description Log */
    <div className="flex-1 bg-slate-900/50

```



```
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
    }
}
}
```

</pre>

</div>

</div>

);

}

ФАЙЛ 70/87: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
```

```
interface TooltipProps {
```

```
    /** Содержимое подсказки (можно JSX). */
```

```
    content: React.ReactNode;
```

```
    /** Элемент, на который наводят курсор / фокус. */
```

```
    children: React.ReactNode;
```

```
    /** С какой стороны показывать. По умолчанию – сверху
```

```
    side?: "top" | "bottom";
```

```
    className?: string;
```

```
}
```

```
/**
```

```
 * Всплывающая подсказка (как title="", только красивая
```

```
 * Появляется по наведению курсора и по фокусу с клавиатуры
```

```
 */
```

```
export const Tooltip: React.FC<TooltipProps> = ({ content,
```

```
import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import {
  Play,
  Pause,
  RotateCcw,
  Clock,
} from "lucide-react";

interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}

interface TEdge {
  u: number;
  v: number;
}

const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математический анализ)" },
  { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
  { id: 2, x: 280, y: 230, label: "2", name: "Линейный (Линейная алгебра)" },
  { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы и структуры данных)" },
  { id: 4, x: 640, y: 150, label: "4", name: "Машинное (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
  { u: 0, v: 1 },
  { u: 0, v: 2 },
  { u: 1, v: 3 },
  { u: 2, v: 3 },
  { u: 3, v: 4 },
];
```

```
// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
  visited.add(u);
  dfsStack.push(u);
  recordStep(
    `DFS: зашли в вершину ${u} (${dagNodes[u].name})`,
    u,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs(to);
    } else if (!fullyProcessed.has(to)) {
      // Explaining potential cycle detected (not in topological order)
      recordStep(
        `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`,
        u,
      );
    }
  }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in reverse
recordStep(
  `DFS: закончили обработку '${dagNodes[u].name}'`,
  u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    dfs(i);
  }
}
```

```

    setIsPlaying(false);
    setCurrentStepIdx(0);
  };

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)
    .join(" → ");
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучен
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        500 text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
      >

```

```

const isActive =
  (step.currentNode === edge.u || step.currentNode === edge.v) &&
  step.visited.has(edge.v);

return (
  <line
    key={`edge-${idx}`}
    x1={u.x}
    y1={u.y}
    x2={v.x}
    y2={v.y}
    stroke={isActive ? "#818cf8" : "#334155"}
    strokeWidth={isActive ? "3" : "2"}
    markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow"})`}
    className="transition-all duration-300"
  />
);
}}}

```

```

{/* Nodes */}
{dagNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited = step.visited.has(node.id);
  const isProcessed = step.fullyProcessed.has(node.id);

  let fill = "#1e293b";
  let stroke = "#334155";
  let textColor = "text-slate-400";

  if (isCurrent) {
    fill = "#312e81";
    stroke = "#818cf8";
    textColor = "text-white font-bold";
  } else if (isProcessed) {
    fill = "#022c22";
    stroke = "#059669";
  }

```

```

        >
        {node.name.split(" ").slice(1).join(" ")}
      </text>
    </g>
  );
}
</svg>
</div>

```

```

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 text-slate-100 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

```

```

<div className="bg-slate-900 border border-slate-800 p-4">
  <p className="text-xs text-indigo-300 min-h-[100px]">
    {step.desc}
  </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-auto">
  {/* Topological List output */}
  <div>
    <span className="text-[10px] text-slate-500">
      РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
    </span>
    <div className="bg-slate-900/60 p-2.5 rounded-md pr-1 text-slate-300">
      {step.topoList.length === 0 ? (
        <span className="text-slate-600 text-sm">
          ожидаем обработку узлов...
        </span>
      ) : (
        getTopoListString()
      )}
    </div>
  </div>

```

```
        className="bg-slate-900 border border-slate-800 text-white"
      >
        Вперед
      </button>
    </div>
  </div>
</div>
</div>
</div>
);
};
```

ФАЙЛ 72/87: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
```

```

    hint?: string;
    Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, ч
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
  "heap-beam-search": {
    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы – куча сама держит лучшую
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветится

```



```

        </>
        <ChapterImage vizType="dijkstra" />
        <DijkstraViz />
    </>
    },
},
"bellman-ford": {
    title: "Форд–Беллман",
    Component: () => (
        </>
        <ChapterImage vizType="bellman-ford" />
        <BellmanFordViz />
    </>
    ),
},
floyd: {
    title: "Флойд–Уоршелл",
    Component: () => (
        </>
        <ChapterImage vizType="floyd" />
        <FloydViz />
    </>
    ),
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: () => (
"mst-kruskal": { title: "Краскал и DSU", Component: KruskalSimul
"mst-prima": { title: "Прим", Component: KruskalSimul
"mst-boruvka": { title: "Борувка", Component: KruskalSimul
"string-kmp": { title: "Префикс-функция (КМП)", Component: () => (
"string-z-func": { title: "Z-функция", Component: () => (
"aho-corasick": {
    title: "Ахо–Корасик",
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
        <div className="space-y-10">
            <WaterfallAnimationWidget />
            <SalmonAutomatonWidget />
        </div>
    )
},

```

```

// Ветка H -> E -> R -> S
1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, de
2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, d
3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360,
4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460,
// Ветка H -> I -> S
5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, d
6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360,
// Ветка S -> H -> E
7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, de
8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, d
9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360,
    = 2 (HE)
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]
  0: { H: 1, S: 7 },
  1: { E: 2, I: 5 },
  2: { R: 3 },
  3: { S: 4 },
  4: {},
  5: { S: 6 },
  6: {},
  7: { H: 8 },
  8: { E: 9 },
  9: {},
};

```

```

export const WaterfallAnimationWidget: React.FC = () =>
  // Переключатель режимов: 'training' (Полный BFS обхо
  const [activeMode, setActiveMode] = useState<'trainin

  // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
  const [textToScan, setTextToScan] = useState<string>(
  const [currentIndex, setCurrentIndex] = useState<numb
  const [currentNode, setCurrentNode] = useState<number>
  const [isSearchDone, setIsSearchDone] = useState<bool

```

```

let jumpType: 'normal' | 'fail' | null = null;
void jumpType;
let originalCurr = curr;

if (TRIE_TRANSITIONS[curr][char] !== undefined) {
  const nextNode = TRIE_TRANSITIONS[curr][char];
  logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}). `;
  setJumpPath({ from: curr, to: nextNode, type: 'normal' });
  curr = nextNode;
  setActiveSearchCodeLine(3);
} else {
  setActiveSearchCodeLine(2);
  let jumps = 0;
  while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
    curr = WATERFALL_NODES[curr].fail;
    jumps++;
  }
  const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
  if (originalCurr === 0) {
    logMsg += `В корне нет перехода по '${char}'. Логично.`;
    setJumpPath(null);
  } else {
    logMsg += `Поток по '${char}' у вершины #${originalCurr} не имеет  

    рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}). `;
    setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
  }
  curr = nextNode;
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
  if (WATERFALL_NODES[temp].is terminal) {
    const word = [...currentMatches].reverse().map(m => m.word).join('');
    matchedWordsForLog.push(word);
    currentMatches = [];
  }
  temp = WATERFALL_NODES[temp].parent;
}

```

```

        просто не существует. fail[H] автоматически нап
codeLine: 5,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'H',
checkingBranchesFrom: null,
},
{
  targetNodeId: 7,
  title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
  desc: 'Обрабатываем дочернюю вершину #7 (S) на De
    ,
codeLine: 5,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: null,
},
{
  targetNodeId: 2,
  title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
  desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
    а-разведчик плывет в ROOT и ищет ветку по "E".
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'E',
checkingBranchesFrom: 0,
},
{
  targetNodeId: 5,
  title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
  desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
    H" и "S" не подходят. Ветки нет, fail[HI] = ROOT
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'I',

```

```

        title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
        desc: '  МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
        ба плывет в #1 (H) и ищет ветку "E". У вершины
        codeLine: 4,
        scoutPos: 1,
        inspectedParentFail: 1,
        checkTargetChar: 'E',
        checkingBranchesFrom: 1,
    },
    {
        targetNodeId: 4,
        title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
        desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
        #7 (S). fail[HERS] = #7 (S). Автомат полностью
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNode];

```

```

return (
  <div className="bg-slate-800/95 rounded-2xl p-6 border"
    {/* Шапка и переключатель режимов */}
    <div className="flex flex-wrap items-center justify-between"
      <div className="flex items-center space-x-3">
        <span className="text-3xl"> </span>
        <div>
          <h4 className="text-2xl font-bold text-white">
            Анимация водопада: Полное обучение автомата
          </h4>
          <p className="text-sm text-slate-400">
            Визуализация пошагового построения всех ф

```

```

    <h5 className="text-indigo-300 font-bold">
      <span> </span> Полный обход в ширину (BFS)
    </h5>
    <p className="text-xs text-slate-300 mb-3">
      Показаны все шаги очереди BFS (начиная с
      ar 8 (SHE → HE)</strong>, где наглядно видно
    </p>
    <div className="space-y-1.5 mb-6 overflow-hidden">
      {TRAINING_STEPS_INFO.map((step, idx) =>
        <button
          key={idx}
          onClick={() => setTrainStep(idx)}
          className={`w-full text-left px-3 py-2
            ${
              idx === trainStep
                ? 'bg-indigo-600 text-white font-medium'
                : 'bg-slate-800 text-slate-400'
            }`}
        >
          <span className="line-clamp-1">{step.description}</span>
          {idx === trainStep && <span className="font-medium">
            </button>
        </div>
      </div>

    <div className="flex gap-2">
      <button
        onClick={() => setTrainStep((prev) => Math.max(0, prev - 1))}
        disabled={trainStep === 0}
        className="flex-1 bg-slate-700 hover:bg-slate-600 text-white font-medium"
      >
        Назад
      </button>
      <button
        onClick={() => setTrainStep((prev) => Math.min(
          TRAINING_STEPS.length - 1, prev + 1
        ))}
        disabled={trainStep === TRAINING_STEPS.length - 1}
        className="flex-1 bg-slate-700 hover:bg-slate-600 text-white font-medium"
      >
        Вперед
      </button>
    </div>
  </div>
</div>

```

```

        т, возврат в корень</span>}
      </div>
      <div className={`p-1.5 rounded flex items-center justify-between border-emerald-500 text-emerald-300 font-mono`} >
        <span>4. if (trie[v][char] !== undefined) {
          {currentTrainInfo.codeLine === 4 && <span>ан IF!</span>}
        }
      </div>
      <div className={`p-1.5 rounded flex items-center justify-between border-blue-500 text-blue-300 font-bold`} >
        <span>5. else fail[u] = root; // Бегу к корню</span>
        {currentTrainInfo.codeLine === 5 && <span>root</span>}
      </div>
    </div>
  </div>
</div>

<div>
  <h5 className="text-slate-200 font-bold mb-2">
    <Lightbulb className="w-4 h-4 text-amber-400"/>
  </h5>
  <div className="bg-slate-800 p-4 rounded-lg w-full min-h-[72px] flex items-center">
    {currentTrainInfo.desc}
  </div>
</div>
</div>
) : (
  /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
  <div className="grid grid-cols-1 lg:grid-cols-3">
    <div className="bg-slate-900/60 p-5 rounded-xl">
      <div>
        <h5 className="text-white font-bold mb-2">
          <span>»</span> Текст для сканирования
        </h5>
        <p className="text-xs text-slate-400 mb-4">

```

```

        onClick={stepSearchSimulation}
        disabled={isSearchDone || textToScan.length === 0}
        className={`w-full py-2.5 rounded-lg font-medium transition-colors duration-200
        ${isSearchDone || textToScan.length === 0
            ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
            : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
        `}
    >
        <span>{currentIndex === 0 ? 'Начать план' : 'Продолжить'}</span>
        <ArrowRight className="w-4 h-4" />
    </button>
</div>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl" >
    <div>
        <h5 className="text-white font-bold mb-3">Текущее действие в коде</h5>
        <div className="bg-slate-950 p-4 rounded-2xl" >
            <div className={`p-1.5 rounded flex items-center border border-blue-500 text-blue-300 font-bold`} >
                <span>1. curr = state; char = text[i]
                {activeSearchCodeLine === 1 && <span>инициализация</span>}
            </div>
            <div className={`p-1.5 rounded flex items-center border border-rose-500 text-rose-300 font-bold`} >
                <span>2. while (curr !== root && !trie[curr].fail)
                {activeSearchCodeLine === 2 && <span>проверка по fail!</span>}
            </div>
            <div className={`p-1.5 rounded flex items-center border border-blue-400 text-blue-200 font-bold`} >
                <span>3. curr = trie[curr].get(char)
                {activeSearchCodeLine === 3 && <span>переход по течению</span>}
            </div>
        </div>
    </div>
</div>

```



```

</h5>
<div className="flex flex-wrap gap-4 text-xs">
  <span className="flex items-center gap-1 text-blue-500">
    <span className="w-3 h-0.5 bg-blue-500 inline-block"></span>
  </span>
  <span className="flex items-center gap-1 text-blue-500">
    <span className="w-3 h-0.5 border-t border-blue-500 inline-block"></span>
  </span>
</div>
</div>

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar">
  <svg viewBox="0 0 800 520" className="w-full h-full">

    {/* Горизонтальные линии и подписи уровней */}
    {[
      { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
      { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
      { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
      { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
      { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' }
    ].map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line
          x1="20"
          y1={level.y}
          x2="780"
          y2={level.y}
          stroke="#334155"
          strokeWidth="1"
          strokeDasharray="4,4"
          opacity="0.4"
        />
        <rect x="25" y={level.y - 12} width="140" height="24" />
        <text x="32" y={level.y + 4} fill="#94a3b8">
          {level.label}
        </text>
      </g>
    })}
  </svg>
</div>

```

```

        fontWeight="bold"
        textAnchor="middle"
    >
        {char}
    </text>

    {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
    {isTrainingExamined && (
        <g transform={`translate(${(fromN
.y) / 2)}`}>
            <rect x="-15" y="-12" width="30
strokeWidth="2" />
            <text x="0" y="5" fontSize="14"
                {isMatchBranch ? ' ' : ' '}
            </text>
        </g>
    )}
    </g>
    );
    });
    }}}

{/* Рендер fail-ссылок (пунктирные водопадн
{Object.values(WATERFALL_NODES).map((node) :
    if (node.id === 0) return null;
    const targetNode = WATERFALL_NODES[node.f

    // В режиме обучения показываем fail-ссыл
    // Найдем индекс шага, на котором обработ
    const stepIdxForNode = TRAINING_STEPS_INF
    if (activeMode === 'training' && trainStep

    let isHighlighted = activeMode === 'search
e === 'fail';
    let isJustEstablished = activeMode === 't
-ссылку

    // Вычисляем изогнутую кривую для красоты

```

```

    {isTargetChild && (
      <circle cx={node.x} cy={node.y} r="
    )}
  )}

```

```

    {/* Основной круг вершины */}
    <circle
      cx={node.x}
      cy={node.y}
      r={isCurrent || isTargetChild ? '24' : '18'}
      fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : '#f8d7da'}
      stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : '#000000'}
      strokeWidth={isCurrent || isTargetChild ? '2' : '1'}
      className="transition-all duration-300"
    />
  )}

```

```

    {/* Текст внутри вершины */}
    <text
      x={node.x}
      y={node.y + 5}
      fill={isCurrent || isTargetChild ? 'white' : 'black'}
      fontSize={isCurrent || isTargetChild ? '14' : '12'}
      fontWeight="bold"
      textAnchor="middle"
    >
      {node.label}
    </text>
  )}

```

```

    {/* Метка искомой вершины в режиме обхода */}
    {isTargetChild && (
      <g transform={`translate(${node.x}, ${node.y})`}
        <rect x="-35" y="-12" width="70" height="24" fill="#f8d7da" stroke="#000000" strokeWidth="1"/>
        <text x="0" y="2" fontSize="10" fill="white" font-weight="bold">
          ИЩЕМ FAIL
        </text>
      </g>
    )}
  )}

```

```

    {/* Индикатор словарных слов */}
  )}

```

```

        </svg>
      </div>
    </div>

    { /* Список найденных совпадений (только в режиме поиска)
    {activeMode === 'search' && {
      <div className="mt-6 bg-slate-900/50 p-5 rounded" >
        <h5 className="text-white font-bold mb-3 text" >
          <span> </span> Улов лосося (Найденные слова)
        </h5>
        {foundMatches.length === 0 ? {
          <p className="text-sm text-slate-400 italic" >
            Пока ничего не поймали. Запустите шаги симуляции
          </p>
        } : {
          <div className="flex flex-wrap gap-2">
            {foundMatches.map((match, idx) => {
              <div
                key={idx}
                className="bg-emerald-950 border border-emerald-500
                text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]" >
                <CheckCircle2 className="w-4 h-4 text-emerald-500" />
                <span>{match.word}</span>
                <span className="text-[10px] bg-emerald-950/50 px-2" >
                  Индекс {match.index}
                </span>
              </div>
            })}
          </div>
        )}
      </div>
    )}
  </div>
) }
</div>
};

```

```
// Вкладка 2D = своё демо (свой скелет): компилятор п
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, mode === "2d"
}, [chapterId, mode]);

return (
  <div className="w-full bg-slate-950 rounded-2xl bor
    <div className="flex gap-2 mb-4 overflow-x-auto n
      {{
        [
          ["1d", "1. Одномерные суммы"],
          ["2d", "2. Двумерные суммы"],
        ] as const
      }).map(({key, label}) => {
        <button
          key={key}
          onClick={() => setMode(key)}
          className={`px-3 py-2 rounded-lg text-xs sm
            mode === key ? "bg-indigo-600 text-white"
          }`
        >
          {label}
        </button>
      })}
    </div>
    {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
  </div>
);
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {
  const pref = useMemo(
    () => A1.reduce<number[]>((acc, v) => [...acc, acc[
    []
  );
};
```

```

const isDrivenAdd = driven !== null && driven < i;
return (
  <div
    key={i}
    onMouseEnter={() => setHover({ kind: "add", i })}
    onMouseLeave={() => setHover(null)}
    onClick={() => setRange((r) => (i < r ? i : r))}
    className={` ${cellBase} cursor-pointer ${
      isDrivenAdd
        ? "bg-amber-500 text-black ring-2 ring-black"
        : inCover
        ? "bg-indigo-500 text-white ring-2 ring-black"
        : inRange
        ? "bg-indigo-900/70 text-indigo-300"
        : "bg-slate-800 text-slate-300"
      }`}
    title={`a[${i}] = ${v}`}
  >
    {v}
  </div>
);
}}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
  <div className="w-[46px] shrink-0 text-right">
    {pref.map((v, i) => {
      const active = hover?.kind === "pref" && i === pref.length - 1;
      const isL = i === range.l;
      const isR = i === range.r + 1;
      const masked = driven !== null && i > driven;
      const isDrivenCur = driven !== null && i === driven;
      return (
        <div
          key={i}
          onMouseEnter={() => setHover({ kind: "pref", i })}
          onMouseLeave={() => setHover(null)}

```

```

    ) : (
      <p className="text-slate-300">
        <b className="text-emerald-400">
           $P[\{hover.i\}] = \{pref[hover.i]\}$ 
        </b>{" "}
        – это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
           $a[0..\{hover.i - 1\}] = \{A1.slice(0, hove$ 
        </span>
      </p>
    )
  ) : hover?.kind === "a" ? (
    <p className="text-slate-300">
      <b className="text-indigo-400">
         $a[\{hover.i\}] = \{A1[hover.i]\}$ 
      </b>{" "}
      входит во все префиксы, начиная с{" "}
      <span className="font-mono text-emerald-300">
        подвинуть границу запроса.
      </span>
    </p>
  ) : (
    <p className="text-slate-500">
      Наведите курсор на любое число: у <b classN
      у <b className="text-indigo-400">a</b> – ку
    </p>
  )
})
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
  <div className="text-xs text-slate-400 mb-2">
    Запрос суммы на отрезке (кликайте по массиву
  </div>
  <p className="font-mono text-sm sm:text-base te
     $sum(a[\{range.l\}..\{range.r\}]) = P[\{range.r + 1$ 
    <span className="text-emerald-400">{pref[rang
    <span className="text-rose-400">{pref[range.l
    <span className="text-indigo-300 font-bold">{

```

```

    return null;
  };

  return (
    <div className="space-y-5">
      <div className="grid gap-5 lg:grid-cols-2">
        {/* Исходная матрица */}
        <div className="overflow-x-auto">
          <div className="text-xs text-slate-400 mb-2">
            <div className="inline-block">
              {A2.map((row, i) => {
                <div key={i} className="flex gap-1.5 mb-1">
                  {row.map((v, j) => {
                    const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                    return (
                      <div
                        key={j}
                        onClick={() => setRect((r) => {
                          i < r.r1 || j < r.c1 ? { r1: i, r2: i, c1: j, c2: j } : r;
                        })}
                        className={` ${cellBase} cursor-pointer ${
                          inRect ? "bg-indigo-500 text-white" : ""
                        }`}
                      >
                        {v}
                      </div>
                    );
                  })}
                </div>
              })}
            </div>
          </div>
          <div className="text-xs text-slate-400 mb-2">
            <div className="inline-block">
              {A2.map((row, i) => {
                <div key={i} className="flex gap-1.5 mb-1">
                  {row.map((v, j) => {
                    const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                    return (
                      <div
                        key={j}
                        onClick={() => setRect((r) => {
                          i < r.r1 || j < r.c1 ? { r1: i, r2: i, c1: j, c2: j } : r;
                        })}
                        className={` ${cellBase} cursor-pointer ${
                          inRect ? "bg-indigo-500 text-white" : ""
                        }`}
                      >
                        {v}
                      </div>
                    );
                  })}
                </div>
              })}
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
  <div className="text-xs text-slate-400 mb-2">
    <div className="inline-block">
      {A2.map((row, i) => {
        <div key={i} className="flex gap-1.5 mb-1">
          {row.map((v, j) => {
            const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
            return (
              <div
                key={j}
                onClick={() => setRect((r) => {
                  i < r.r1 || j < r.c1 ? { r1: i, r2: i, c1: j, c2: j } : r;
                })}
                className={` ${cellBase} cursor-pointer ${
                  inRect ? "bg-indigo-500 text-white" : ""
                }`}
              >
                {v}
              </div>
            );
          })}
        </div>
      })}
    </div>
  </div>
  <div className="text-xs text-slate-400 mb-2">
    <div className="inline-block">
      {A2.map((row, i) => {
        <div key={i} className="flex gap-1.5 mb-1">
          {row.map((v, j) => {
            const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
            return (
              <div
                key={j}
                onClick={() => setRect((r) => {
                  i < r.r1 || j < r.c1 ? { r1: i, r2: i, c1: j, c2: j } : r;
                })}
                className={` ${cellBase} cursor-pointer ${
                  inRect ? "bg-indigo-500 text-white" : ""
                }`}
              >
                {v}
              </div>
            );
          })}
        </div>
      })}
    </div>
  </div>

```



```

    <div className="min-h-[54px] bg-slate-900 border
      {hover ? (
        <p className="text-slate-300">
          <b className="text-amber-400">
             $P[\{hover.i\}][\{hover.j\}] = \{P[\{hover.i\}][\{hover.j\}] +$ 
          </b>{" "}
          – сумма всего прямоугольника от левого верхнего угла
        </p>
      ) : (
        <p className="text-slate-500">Наведите курсор на элемент
      </p>
      )}
    </div>

    <div className="bg-slate-900 border border-indigo-500"
      <p className="font-mono text-sm sm:text-base text-slate-300">
        сумма = <span className="text-emerald-400">A</span> + <span
        <span className="text-rose-400">C</span> + <span
        <span className="text-indigo-300 font-bold">{
      </p>
      <p className="text-[11px] text-slate-500 mt-1">
        Вычли два «хвоста» и вернули дважды вычитенный
      </p>
    </div>
  </div>
);
};

```

ФАЙЛ 75/87: src/components/visualizers/SparseTableViz.tsx
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 724 | РАЗМЕР: 10.5 КБ

```

import React, { useContext, useEffect, useState } from "react";
import { emitVizDemo, useVizStepSync, VizChapterContext } from "viz";
import { motion, AnimatePresence } from "motion/react";
import {

```

```

    )
  );

  for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
      ST2D[r][c][0][0] = val2D[r][c];
    }
  }

  for (let kx = 0; kx <= 3; kx++) {
    for (let ky = 0; ky <= 3; ky++) {
      if (kx === 0 && ky === 0) continue;
      for (let r = 0; r + (1 << kx) <= 8; r++) {
        for (let c = 0; c + (1 << ky) <= 8; c++) {
          if (kx > 0) {
            ST2D[r][c][kx][ky] = Math.min(
              ST2D[r][c][kx-1][ky],
              ST2D[r + (1 << (kx-1))][c][kx-1][ky]
            );
          } else {
            ST2D[r][c][kx][ky] = Math.min(
              ST2D[r][c][kx][ky-1],
              ST2D[r][c + (1 << (ky-1))][kx][ky-1]
            );
          }
        }
      }
    }
  }
}

export const SparseTableViz: React.FC = () => {
  const [tab, setTab] = useState<"1d" | "2d_build" | "2d_build2"&> "1d";
  const chapterId = useContext(VizChapterContext);

  // Каждая вкладка = своё демо с собственным скелетом:
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab === "2d_build" ? "2d_build" : "1d", [chapterId, tab]);
  }, [chapterId, tab]);
}
```

```
};
```

```
const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  useVizStepSync(step, setStep, maxStep);
  const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

  return (
    <div className="flex flex-col gap-5">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="text-lg font-bold text-indigo-600">
            Построение Sparse Table (Min)
          </h3>
          <p className="text-sm text-slate-400">
            Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$ 
          </p>
        </div>
      </div>
      <div className="flex gap-2">
        <button
          onClick={() => setStep(Math.max(0, step - 1))}
          className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
          disabled={step === 0}
        >
          <ChevronLeft size={20} />
        </button>
        <button
```

```

        <b className="text-sky-400">
            ST[{hover.i}][{hover.j}] = {stID[hover.i]}
        </b>{" "}
        – это минимум на отрезке{" "}
        <span className="font-mono text-sky-300">
            [{covered.from}, {covered.to}]
        </span>{" "}
        длины  $2^{\{hover.j\}}$  = {1 << hover.j}:{ " "}
        <span className="font-mono text-slate-400">
            min({arrID.slice(covered.from, covered.to)})
        </span>
    </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор (или тапните) на любое число
        </p>
    )}
</div>
</div>

<div className="overflow-x-auto pb-2">
    <div className="inline-block min-w-full bg-slate-50">
        <div className="grid grid-cols-[auto_repeat(4,1fr)]">
            {/* Headers */}
            <div className="text-slate-500 font-mono text-sm">
                i \ j
            </div>
            <div className="text-center font-bold text-sm">
                 $2^0$  (L=1)
            </div>
            <div className="text-center font-bold text-sm">
                 $2^1$  (L=2)
            </div>
            <div className="text-center font-bold text-sm">
                 $2^2$  (L=4)
            </div>
        </div>
    </div>
</div>

```

```

        if (currentStep.j - 1 === j) {
          if (currentStep.i === i) isSrc1 = true
          if (currentStep.i + 1 << (currentStep.j - 1)) {
            isSrc2 = true;
          }
        }
      }

      return (
        <div key={j} className="relative flex flex-col items-center justify-center"
        {!isValid ? (
          <div className="w-[clamp(30px,9vw,17px)] font-bold transition-colors"
        ) : (
          <motion.div
            onMouseEnter={() => isVisible}
            onMouseLeave={() => setHover(false)}
            onClick={() => isVisible && setSrc1(i, j)}
            initial={{ opacity: 0, scale: 0.8 }}
            animate={{
              opacity: isVisible ? 1 : 0,
              scale: isVisible ? 1 : 0.8,
            }}
            className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors
              hover && hover.i === i && hover.j === j
                ? "bg-sky-500 text-white"
                : isActive
                ? "bg-indigo-500 text-white"
                : isSrc1
                ? "bg-emerald-500 text-white"
                : isSrc2
                ? "bg-amber-500 text-white"
                : isVisible
                ? "bg-slate-800 text-white"
                : "bg-transparent text-white"}
          >
            {st1D[i][j]}
          </motion.div>
        )}
      )
    )
  }
}

```

```

        const half = 1 << (c.j - 1);
        return {
          <p className="text-lg text-slate-300">
            <span className="text-white font-bold">
              ST[{c.i}][{c.j}]
            </span>{" "}
            = min(
              <span className="text-emerald-400 font-b
                ST[{c.i}][{c.j - 1}]
              </span>
              ,
              <span className="text-amber-400 font-bo
                ST[{c.i + half}][{c.j - 1}]
              </span>
            ) &rarr; min(
              <span className="text-emerald-400">{st1
              <span className="text-amber-400">
                {st1D[c.i + half][c.j - 1]}
              </span>
            ) ={" "}
            <span className="text-indigo-400 font-b
              {st1D[c.i][c.j]}
            </span>
          </p>
        );
      })()
    ) : {
      <p className="text-slate-500 italic">
        Нажмите далее, чтобы увидеть, как блоки сво
      </p>
    })
  </div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
  const [k, setK] = useState(1);

```

```

rollbar">
{[...Array(k + 1)].map((_, i) => k - i).map((
  const stepL = 1 << step;
  const stepSize = 8 - stepL + 1;
  const isCurr = step === k;

  return (
    <React.Fragment key={step}>
      {idx > 0 && <div className="text-slate
      <div className={`flex flex-col items-c
      <h4 className={`font-bold mb-3 flex
    }>
      <span>Матрица блоков {stepL}x{step
      <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
      </h4>
      <div className="max-w-full overflow-
      <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
      {Array.from({length: stepSize * st
        const r = Math.floor(idx / step
        const c = idx % stepSize;

        let highlightClass = isCurr ? 'b
/50 cursor-pointer' : 'bg-slate-800/50 text
        let zIndex = 'z-0';

        const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

        if (isActive) {
          if (isCurr) {
            highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
            zIndex = 'z-10';
          } else {
            const prevL = 1 << (k - 1);

```



```

                <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
            </div>
            <div className="flex ite
                <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
                <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
            </div>
            <div className="flex ite
                <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
                <span className="text
[activeCell.r + prevL][activeCell.c + prevL
            </div>
        </>
    )
    }}({})
</div>

    <div className="pt-3 mt-3 border-t
    ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : {
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
            Наведите курсор на матрицу {L}x{L
        </div>
    }}
</div>
    )}
</div>
</div>
</div>

```

```

                {val2D[r][c]}
            </div>
        )
    }}}

    <div
        className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
        style={{
            top: `${(r1 / 8) * 100}%`,
            left: `${(c1 / 8) * 100}%`,
            marginTop: '8px',
            marginLeft: '8px',
            height: 'calc(' + (H/8*100) +
            width: 'calc(' + (W/8*100) +
        }}
    />

    /* Показываем 4 угла в самой матрице
    <div className="absolute pointer-e
        shadow-[0_0_10px_#f43f5e]" style={{ top: `
        lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
        <div className="absolute pointer-e
        shadow-[0_0_10px_#3b82f6]" style={{ top: `
        width: `calc(`${(Ly / 8) * 100}% - 16px)`,
        <div className="absolute pointer-e
        ld-400 shadow-[0_0_10px_#10b981]" style={{
        8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
        <div className="absolute pointer-e
        00 shadow-[0_0_10px_#f59e0b]" style={{ top:
        00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
        </div>

    <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
        <label className="flex items-cen
            <span className="text-slate-
                <div className="flex gap-2">

```

```

        <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
        <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
        <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

        <span className="text-slate-500">// 2.
        <span className="text-indigo-400">int<
        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
В запроса)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-rose-400">
r1</span>][<span className="text-white bg-r

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
во, чтобы правый край уперся в c2)</span><b
        &nbsp;&nbsp;&nbsp;<span className="text-blue-400">
r1</span>][<span className="text-white bg-b

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
px, чтобы нижний край уперся в r2)</span><b
        &nbsp;&nbsp;&nbsp;<span className="text-emerald-400">
nded">r2 - Lx + 1</span>][<span className="

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
и влево от c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-amber-400">
">r2 - Lx + 1</span>][<span className="text
        {'}}'}});
    </div>

    <div className="w-full flex justify-centr
    <div className="flex flex-col items-cent
    <h4 className="text-slate-400 font-bo
        Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{{k
    </h4>
    <div className="grid gap-[2px] p-2.5

```

```
        wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border
            <span className="text-slate-400 font
                <span className="text-rose-400">
                <span className="text-blue-400 m
            n>
                <span className="text-emerald-400
            span>
                <span className="text-amber-400 m
                &nbsp;<span className="text-slate-400
                <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
            </div>
        </div>
    </div>
</div>
    )
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 76/87: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```
        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').
            ? 'px-3 py-1 rounded text-sm font-bold
```

```

    });

    if (theme === 'dark') document.getElementById('dark-mode')?.classList.add('dark');
    if (theme === 'light') document.getElementById('dark-mode')?.classList.remove('dark');
    if (theme === 'terminal') document.getElementById('dark-mode')?.classList.add('terminal');
  });

  document.getElementById('btn-theme-dark').addEventListener('click', () => {
    document.getElementById('dark-mode')?.classList.add('dark');
    document.getElementById('btn-theme-light').classList.remove('active');
    document.getElementById('btn-theme-terminal').classList.remove('active');
    document.getElementById('btn-theme-notebook').classList.remove('active');
  });

  // Initialize theme and mode
  setTheme('dark');

  // Setup intersection observer for TOC highlighting
  document.addEventListener('DOMContentLoaded', () => {
    const mobileMenuBtn = document.getElementById('mobile-menu');
    const mobileDrawer = document.getElementById('mobile-drawer');
    const closeDrawerBtn = document.getElementById('close-drawer');
    const drawerOverlay = document.getElementById('drawer-overlay');

    function toggleDrawer() {
      const isClosed = mobileDrawer.classList.contains('closed');
      if (isClosed) {
        mobileDrawer.classList.remove('closed');
        if (drawerOverlay) {
          drawerOverlay.classList.remove('hidden');
          // Timeout allows display:block to take effect
          setTimeout(() => drawerOverlay.classList.remove('hidden'), 100);
        }
      } else {
        mobileDrawer.classList.add('closed');
        drawerOverlay?.classList.add('hidden');
      }
    }

    mobileMenuBtn.addEventListener('click', toggleDrawer);
    closeDrawerBtn.addEventListener('click', toggleDrawer);
    drawerOverlay?.addEventListener('click', toggleDrawer);
  });

```

```
        // Highlight active TOC link lo
        const id = entry.target.id;
        document.querySelectorAll('aside
            if (link.getAttribute('href
                link.classList.add('bg-
            } else {
                link.classList.remove('l
            }
        });
    }
    });
}, observerOptions);

document.querySelectorAll('section[id^="que
    observer.observe(section);
});
});
</script>
</body>
</html>
```

ФАЙЛ 77/87: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРOK: 1169 | РАЗМЕР: 1000

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
    <title>Пособие по алгебре: От пиццы к предикатам</t
    <script src="https://cdn.tailwindcss.com"></script>
    <script src="https://polyfill.io/v3/polyfill.min.js
    <script id="MathJax-script" async src="https://cdn.
    <script>
        window.MathJax = {
```



```
        color: #3b2818 !important; /* brown ink */
        background-image: repeating-linear-gradient(
        background-attachment: local !important;
        font-family: "Comic Sans MS", "Caveat", "Serif";
    }
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
    background-color: rgba(255, 255, 255, 0.7);
    border-color: #cbd5e1 !important;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1 !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Comic Sans MS", "Caveat", "Serif"; }
body.theme-notebook #top-controls { background-color: #f0f0f0; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1); }
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
```



```

        <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-pink" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-cyan" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4b1" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-indigo" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-lime" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-green" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="black" stroke-width="1" />
    </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">
        <rect x="0" y="0" width="40" height="20" fill="yellow" />
        <rect x="6" y="-5" width="8" height="6" fill="yellow" />
        <rect x="26" y="-5" width="8" height="6" fill="yellow" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <g id="lego-red">
        <rect x="0" y="0" width="40" height="20" fill="red" />
        <rect x="6" y="-5" width="8" height="6" fill="red" />
        <rect x="26" y="-5" width="8" height="6" fill="red" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <g id="lego-rem">
        <rect x="0" y="0" width="40" height="10" fill="red" />
        <rect x="6" y="-5" width="8" height="6" fill="red" />
        <rect x="26" y="-5" width="8" height="6" fill="red" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <!-- МЯСОРУБКА -->

```

```

<aside id="mobile-drawer" class="fixed inset-y-0
  nslate-x-full transition-transform duration-300
  w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
  <div class="h-full flex flex-col p-6 overflow-y-auto
    lg:rounded-2xl lg:border lg:border-slate-700
      <div class="flex justify-between items-center
        <h3 class="font-black text-transparent bg-clip-text
          ider text-sm">
            Содержание
          </h3>
          <button id="close-drawer-btn" class="text-slate-400
            <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6
              <path stroke-linecap="round stroke-width="2" d="M6 18
                </svg>
              </button>
            </div>
          <nav class="space-y-3 text-sm font-medium">
            <div>
              <details class="group" open>
                <summary class="list-none cursor-pointer text-slate-400
                  <a href="#question-1" class="text-indigo-500 pl-3 font-bold text-indigo-300">
                    1. Алгебраические структуры
                  </a>
                </summary>
                <div class="pl-6 space-y-2 text-sm">
                  <a href="#section-1-1" class="text-slate-400">1.1
                  <a href="#section-1-2" class="text-slate-400">1.2
                  <a href="#section-1-3" class="text-slate-400">1.3
                  <a href="#section-1-4" class="text-slate-400">1.4
                </div>
              </div>
            </details>
            <div>
              <details class="group">
                <summary class="list-none cursor-pointer text-slate-400
                  <a href="#question-2" class="text-indigo-500 pl-3 font-bold text-indigo-300">
                    2. Комплексные числа
                  </a>
                </summary>
                <div class="pl-6 space-y-2 text-sm">
                  <a href="#section-2-1" class="text-slate-400">2.1
                  <a href="#section-2-2" class="text-slate-400">2.2
                  <a href="#section-2-3" class="text-slate-400">2.3
                  <a href="#section-2-4" class="text-slate-400">2.4
                </div>
              </div>
            </div>
          </nav>
        </div>
      </div>
    </div>
  </div>

```

```
<details class="group">
  <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-500">
    5. Группа корней из единицы
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-fuchsia-500">
    <a href="#q5-def" class="block hover-fuchsia-500">
    <a href="#q5-group" class="block hover-fuchsia-500">
    <a href="#q5-prim" class="block hover-fuchsia-500">
    <a href="#q5-crit" class="block hover-fuchsia-500">
    <a href="#q5-cyclic" class="block hover-fuchsia-500">
  </div>
</details>
```

```
<details class="group">
  <summary class="list-none cursor-pointer-lime-500 pl-3 font-bold text-lime-300">
    6. Делимость, НОД и Евклид
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-lime-500">
    <a href="#q6-1" class="block hover-lime-500">
    <a href="#q6-2" class="block hover-lime-500">
    <a href="#q6-3" class="block hover-lime-500">
    <a href="#q6-4" class="block hover-lime-500">
    <a href="#q6-5" class="block hover-lime-500">
    <a href="#q6-6" class="block hover-lime-500">
  </div>
</details>
```

```
<details class="group">
  <summary class="list-none cursor-pointer-lime-500 pl-3 font-bold text-lime-300">
    7. Делимость, НОД и Евклид
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-lime-500">
    <a href="#q7-1" class="block hover-lime-500">
    <a href="#q7-2" class="block hover-lime-500">
    <a href="#q7-3" class="block hover-lime-500">
    <a href="#q7-4" class="block hover-lime-500">
    <a href="#q7-5" class="block hover-lime-500">
    <a href="#q7-6" class="block hover-lime-500">
  </div>
</details>
```

```

<div class="pl-6 space-y-2 text
  <a href="#q9-1" class="bloc
  <a href="#q9-2" class="bloc
  <a href="#q9-3" class="bloc
  <a href="#q9-4" class="bloc
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-10" clas
der-rose-500 pl-3 font-bold text-rose-400">
      10. Делимость и Схема Г
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q10-1" class="blo
    <a href="#q10-2" class="blo
    <a href="#q10-3" class="blo
a>
      <a href="#q10-4" class="blo
      <a href="#q10-5" class="blo
    </div>
  </details>

  <details class="group">
    <summary class="list-none curso
      <a href="#question-11" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
        11. НОД и НОК многочлен
      </a>
    </summary>
    <div class="pl-6 space-y-2 text
      <a href="#q11-1" class="blo
      <a href="#q11-2" class="blo
нственности НОД</a>
      <a href="#q11-3" class="blo
ПИРАЛЬ)</a>

```

```
14. Кратность корня. Ко  
    </a>  
  </summary>  
  <div class="pl-6 mt-2 space-y-2"  
    <a href="#q14-1" class="blo  
    <a href="#q14-2" class="blo  
    <a href="#q14-3" class="blo  
deg)</a>  
    <a href="#q14-4" class="blo  
ition-colors border border-rose-500/30"> K  
  </div>  
</details>  
  
<details class="group">  
  <summary class="list-none curso  
    <a href="#question-15" clas  
der-cyan-500 pl-3 font-bold text-cyan-400">  
    15. Производная. Критер  
    </a>  
  </summary>  
  <div class="pl-6 mt-2 space-y-2"  
    <a href="#q15-1" class="blo  
    <a href="#q15-2" class="blo  
    <a href="#q15-3" class="blo  
    <a href="#q15-4" class="blo  
  </div>  
</details>  
  
<details class="group">  
  <summary class="list-none curso  
    <a href="#question-16" clas  
der-red-500 pl-3 font-bold text-red-400">  
    16. Неприводимые многоч  
    </a>  
  </summary>  
  <div class="pl-6 mt-2 space-y-2"  
    <a href="#q16-1" class="blo  
    <a href="#q16-2" class="blo
```

```

        detail.addEventListener('toggle
            if (detail.open) {
                document.querySelectorA
                    if (other !== detail
                        other.removeAtt
                    }
            }
        });
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('clie
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersectin
                visibleId = entry.t

                // Highlight active
                document.querySelec
                    a.style.fontWeig
                    if (a.href.ends
                        a.style.fon

```

```
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей браузера  
некоторые веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';
```

```

        box.appendChild(btnContainer);
        modal.appendChild(box);
        document.body.appendChild(modal)
    }

```

```

function downloadMD() {
    let clone = document.querySelector('#clone')

    // Smart markdown parsing based on marked.js
    // Smart markdown parsing based on marked.js
    clone.querySelectorAll('h2').forEach(el => {
    clone.querySelectorAll('h3').forEach(el => {
    clone.querySelectorAll('.analog').forEach(el => {
    clone.querySelectorAll('li').forEach(el => {
    clone.querySelectorAll('svg').forEach(el => {
        let caption = '';
        if (el.nextElementSibling && el.nextElementSibling.tagName === 'CAPTION') {
            caption = el.nextElementSibling.textContent;
        }
        el.textContent = '\n\n[ Изображение: ' + caption + ' ]';
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        let text = clone.innerHTML;
        text = text.replace(/\n{3,}/g, '\n\n');

        if (window !== window.top) {
            fallbackModal(text, 'Markdown');
            return;
        }
        const blob = new Blob([text], { type: 'text/markdown' });
        const url = URL.createObjectURL(blob);
        const link = document.createElement('a');
        link.href = url;
        link.download = 'vyisshaya_algebra.md';
        document.body.appendChild(link);
        link.click();
    });
}

```



```

nt; }

body.bg-white [class*="]

nt; }

body.bg-white [class*="]
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur

```

```

                <span> </span> Word
            </button>
            <a href="algebra.pdf" target="_blank">
- sm font-bold flex items-center justify-center
                <span> </span> PDF
            </a>
            <button onclick="downloadMD()">
- bold flex items-center justify-center gap-2
                <span> </span> Markdown
            </button>
            <button onclick="downloadHTML()">
t-sm font-bold flex items-center justify-center
                <span> </span> HTML
            </button>
        </div>

        <!-- Смена темы -->
        <div>
            <div class="text-xs text-slate-500">
                <div class="flex items-center gap-2">
                    <button id="theme-dark" onclick="toggleTheme('dark')">
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
                    <button id="theme-light" onclick="toggleTheme('light')">
r justify-center text-amber-50 hover:text-white
">* </button>
                    <button id="theme-terminal" onclick="toggleTheme('terminal')">
- center justify-center text-green-400 hover:text-green-500
терминала"> </button>
                </div>
            </div>
        </div>
    </div>
</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12">

```

```

<!-- ===== WTF CONTAINER =====
<div id="wtf-container" class="hidden">
  <h2 class="text-3xl font-black text-cen

    <div class="overflow-x-auto w-full pb-4"
      <div class="grid grid-cols-3 gap-4"
        <!-- Column headers -->
        <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
        <div class="bg-orange-900/40 bo
ольцо полиномов</div>
        <div class="bg-cyan-900/40 bord
сные числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 2: Базовые операции / Евкли
        <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-10" onclick="setM

```

```

        <div class="font-bold text-slate-500">
    </a>

    <!-- Row 5: Корни / Уравнения -->
    <div class="hidden md:block"></div>
    <a href="#question-13" onclick="setMainQuestion(13)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <a href="#question-4" onclick="setMainQuestion(4)">
- 4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">
            <div class="font-bold text-slate-500">
    </a>

    <!-- Row 6: Кратность -->
    <div class="hidden md:block"></div>
    <a href="#question-14" onclick="setMainQuestion(14)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <div class="hidden md:block"></div>

    <!-- Row 7: Производная -->
    <div class="hidden md:block"></div>
    <a href="#question-15" onclick="setMainQuestion(15)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <div class="hidden md:block"></div>

    <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
    <div class="col-span-1 md:col-span-1">
b-2 border-b border-emerald-500/30 pb-2">Кл

```



```
Просто перепиши формулу с буквой $k$.</li>
      <li><span class="text-w
усок $k$, замени его на жареного карася из
      </ul>
```

```
</div>
```

```
<!-- Двойная делимость -->
<div class="bg-slate-800 p-6
  <h3 class="text-xl text-ora
  <p class="text-sm text-slate
  <ul class="list-disc pl-5 s
    <li><span class="text-w
вверх по уравнениям).</li>
    <li><span class="text-w
рху вниз, показывая, что $\delta \vdots r_i
    <li><span class="text-w
х. Твой $d$ больше.</li>
    </ul>
  </div>
```

```
<h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md
  <div class="bg-slate-800 p-
    <h4 class="text-red-400
    <p class="text-sm text-
    <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
  </div>
```

```
  <div class="bg-slate-800 p-
    <h4 class="text-red-400
    <p class="text-sm text-
    <p class="text-xs font-
$c$. Пока получается 0 – корень жив. Как то
корня минус один.</p>
```

```

        <h4 class="text-cyan">
        <p class="text-sm">
x двух, потом последних. <br>
        <code class="text-x">
    </div>
</li>

<li class="bg-slate-800/50">
    <span class="text-2xl">
    <div>
        <h4 class="text-cyan">
        <p class="text-sm">
есть сложение – есть ноль. Если есть умноже
        <code class="text-x">
    </div>
</li>

<li class="bg-slate-800/50">
    <span class="text-2xl">
    <div>
        <h4 class="text-cyan">
        <p class="text-sm">
авь их драться. Умножение всегда врывается
        <code class="text-x">
    </div>
</li>
</ul>
</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====>
```

```

-----
for (const sheet of Array.from(document.styleSheets))
  try {
    const rules = (sheet as CSSStyleSheet).cssRules;
    parts.push(Array.from(rules).map((r) => r.cssText));
  } catch {
    const href = (sheet as CSSStyleSheet).href;
    if (!href) continue;
    try {
      const res = await fetch(href);
      if (res.ok) parts.push(await res.text());
    } catch {
      /* чужой домен – пропускаем */
    }
  }
}
return parts.join("\n");
}

```

```

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
@media print {
  body { background: #fff; color: #0f172a; }

```



```

    host.querySelectorAll("details").forEach((d) => d.setOpen(true));

    return { body: host.innerHTML, termIds: ids };
}

```

```

function glossarySection(termIds: string[]): string {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escapeHtml(id)}">
        <h3>${escapeHtml(e.title)}</h3>
        <p>${escapeHtml(e.short)}</p>
        ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
        ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
      </div>`;
    })
    .join("\n");

  return `<section class="export-gloss">
    <h2>Словарик терминов из этой темы</h2>
    <p style="font-size:.8rem;color:#94a3b8;margin:0 0 10px 0">
      Перейдите по ссылкам, чтобы узнать больше о терминах, связанных с
      этой темой.
    </p>
    ${items}
  </section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; extraCss: string }): string {
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>

```

```
/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
  const num = title.match(/^\\s*(\\d+)/)?.[1];
  const base = id.replace(/^[^a-z0-9-]+/gi, "-").toLowerCase();
  return `${num ? String(num).padStart(2, "0") + "-": ""}${base}`;
}
```

```
/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
  const css = await collectCss();
  const { body, termIds } = prepareBody(chapter.content);
  const html = wrap({
    title: chapter.title,
    subtitle: chapter.category || "Универсальное пособие",
    css,
    body,
    gloss: glossarySection(termIds),
    origin: window.location.origin,
  });
  triggerDownload(html, safeName(chapter.title, chapter.id));
}
```

```
/** Выгрузить всё пособие одним файлом со сквозным оглавлением
export async function downloadBookHtml(chapters: Chapter[]) {
  const css = await collectCss();
  const allTerms: string[] = [];
  const bodies: string[] = [];
```

```
  const toc = chapters
    .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}>${c.title}</a>`
    .join("\\n");
```

```
  chapters.forEach((c) => {
    const { body, termIds } = prepareBody(c.content);
    termIds.forEach((t) => {
      if (!allTerms.includes(t)) allTerms.push(t);
    });
  });
```

```

*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\s{2}[a-zA-Z0-9_]+/g)].map(m => m[0].trim()));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
  };
});

```

}

ФАЙЛ 81/87: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

/**

* Проверка покрытия глав всплывающими подсказками:
* в каждой ли теме находятся термины из глоссария.
*

* node scripts/audit-terms.mjs

*/

import { build } from "esbuild";

import path from "node:path";

const ROOT = process.cwd();

await build({

entryPoints: ["src/data/content.ts", "src/data/glossa

bundle: true, format: "esm", platform: "node",

outdir: "tmp/audit-terms", outExtension: { ".js": ".m

external: ["react", "react-dom", "motion", "lucide-re

});

const { chapters } = await import(path.join(ROOT, "tmp/

const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*\+\?\^\\$\{\}\(\)|[\]\[\]]/g, "\\

const re = new RegExp(

`(?<![\\p{L}\\p{N}_-])(\${GLOSSARY_LABELS.map((l) => e

"giu"

);

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts

const MAX_PER_SURFACE = 2;

const MAX_PER_TERM = 8;

ФАЙЛ 82/87: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, }

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
  if (!fs.existsSync(PAGES_DIR)) {
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
  }

  const pages = fs
    .readdirSync(PAGES_DIR)
    .filter((f) => f.endsWith(".png"))
    .sort();

  if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

  ensureDir(OUT_DIR);

  const doc = new PDFDocument({
    size: [A4.width, A4.height],
    margin: 0,
    autoFirstPage: false,
```

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текст
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недо
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+\$/ , "Конфигурация проекта"],
];
```

```

    }

    return files
      .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
      .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
      .filter((rel) => CODE_EXT.has(path.extname(rel)))
      .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
      .sort((a, b) => {
        const ia = ORDER.indexOf(categoryOf(a));
        const ib = ORDER.indexOf(categoryOf(b));
        return ia !== ib ? ia - ib : a.localeCompare(b);
      });
  }

  /** Достаём заголовки глав из данных пособия (без запуска)
  function extractChapters(files) {
    const chapters = [];
    const seen = new Set();
    for (const rel of files.filter((f) => f.startsWith("src/"))) {
      const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
      const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
      let m;
      while ((m = re.exec(src)) !== null) {
        const [, id, title] = m;
        if (seen.has(id)) continue;
        seen.add(id);
        chapters.push({ id, title, file: rel });
      }
    }
    return chapters.sort((a, b) => {
      const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
      const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
      return na - nb;
    });
  }

  function main() {
    const files = listFiles();
  }

```

```

    }
    push(`  ${String(i + 1).padStart(3, " ")} . ${rel}`)
  });
  push();
  push();

  current = null;
  files.forEach((rel, i) => {
    const cat = categoryOf(rel);
    if (cat !== current) {
      current = cat;
      push(HR);
      push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
      push(HR);
      push();
    }
    const content = fs.readFileSync(path.join(ROOT, rel), "utf8");
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
  });

  push(HR);
  push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
  push(HR);

  const txt = out.join("\n") + "\n";
  fs.writeFileSync(TXT_PATH, txt, "utf8");

  console.log("[1/3] код -> txt");
  console.log(`файлов:  ${files.length}`);
  console.log(`глав:      ${chapters.length}`);

```



```
/** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
monochrome: process.env.EXPORT_GRAYSCALE !== "1",
size: "A4",
font: "DejaVu-Sans-Mono",
fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
pointsize: 8,
/** Максимум символов в строке (далее – жёсткий пере
cols: 138,
/** Строк содержимого на странице (+2 служебные: шапка
rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  for (const cmd of ["magick", "convert"]) {
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore" });
      return cmd;
    } catch {
      /** пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
  );
}
```

```

    rest = rest.slice(width);
    first = false;
  }
  return parts;
}

function paginate(txt) {
  const source = txt.replace(/\r\n/g, "\n").split("\n");
  const pages = [];
  for (let i = 0; i < source.length; i += PAGE.rows) {
    pages.push(source.slice(i, i + PAGE.rows));
  }
  return pages;
}

async function main() {
  if (!fs.existsSync(TXT_PATH)) {
    throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
  }

  const im = imageMagickCmd();
  const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

  fs.rmSync(PAGES_DIR, { recursive: true, force: true });
  ensureDir(PAGES_DIR);

  const total = pages.length;
  const jobs = pages.map((lines, idx) => {
    const num = String(idx + 1).padStart(4, "0");
    const header = `Универсальное пособие • полный исходный код
    Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
    const body = [header, "-".repeat(PAGE.cols), ...lines];
    const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
    const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
    fs.writeFileSync(txtFile, body + "\n", "utf8");
    return { txtFile, pngFile, num };
  });
});

```

```
-----
const bytes = jobs.reduce((s, j) => s + fs.statSync(j)
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAGE
}

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});
```

=====

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

ФАЙЛ 86/87: `.github/workflows/deploy-pages.yml`
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

name: Deploy app to GitHub Pages

```
# ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
# (у бота-интеграции нет admin-прав на них – API отвечает)
#
# 1. Settings → Pages → Build and deployment → Source
#    Если там «Deploy from a branch» (legacy) – шаг D
# 2. Settings → Environments → github-pages → Deployments
#    добавьте паттерн «arena/**» и ветку «main» (или
#    иначе деплой из новой рабочей ветки отклоняется.
```

on:

push:

branches:

- main

Позволяет проверить Pages прямо из рабочей ветки

- "arena/**"

```
  with:
    path: ./dist
```

```
deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    # Ранняя и понятная ошибка вместо невнятного «Failed»
    # когда в репозитории Pages не переключён на сбор
    - name: Preflight – Pages собирается через GitHub
      env:
        GH_TOKEN: ${ secrets.GITHUB_TOKEN }
      run: |
        bt=$(gh api "repos/${ github.repository }/pages" |
        echo "Pages build_type: ${bt}")
        if [ "${bt}" != "workflow" ]; then
          echo "::error title=Pages не настроен::Settings не
          те этот запуск. См. комментарий в начале deployment
          exit 1
        fi

    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
```

ФАЙЛ 87/87: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ

```
name: Rebuild code PDF
```

```
# Пайплайн пересборки PDF при каждом коммите:
```

steps:

- name: Checkout
uses: actions/checkout@v6
with:
 fetch-depth: 0
 persist-credentials: true
- name: Setup Node.js
uses: actions/setup-node@v7
with:
 node-version: "24"
 cache: npm
- name: Install ImageMagick + DejaVu fonts
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends
 # На Ubuntu политика ImageMagick иногда запрещает
 # разрешаем то, что нужно пайплайну (text -> pdf)
 for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
 if [-f "\$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{PATTERN\}" /rights="write" pattern="\{PATTERN\}" /'
 fi
 done
 convert -version
- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${GITHUB_EVENT_INPUTS.DENSITY:-100}
run: npm run export:png

```
run: |
  git config user.name "github-actions[bot]"
  git config user.email "41898282+github-action@users.noreply.github.com"
  git add -- public/export/full_code_all_pages.*
  if git diff --cached --quiet; then
    echo "Экспорт не изменился – коммит не нужен"
    exit 0
  fi
  git commit -m "chore(export): пересборка full_code_all_pages.*"
  git push origin "HEAD:${GITHUB_REF_NAME}"
```